18-740: Modern Computer Architecture & Design

Lecture 11: "Cache Coherence & Memory Consistency"

Olivia Hsu

September 29, 2026

- From **Synthesis Lectures on Computer Architecture**:
- Core Reading:
  - Sec. 2, 3 & 8: Sorin et al, "A Primer on Memory Consistency and Cache Coherence," 2011.
  - Sec. 1 & 2: Balasubramonian et al, "Multi-Core Cache Hierarchies," 2011.
- Elective Reading:
  - Sec. 5: Jain Akanksha, Lin Calvin, "Cache Replacement Policies", 2019.

Electrical & Computer ENGINEERING



Memory Data Flow : Roadmap

- Cache Coherence
- Memory Consistency
- Synchronization & OS

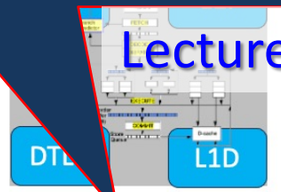
Lectures 9, 10, 11

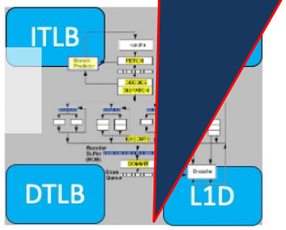
- Memory Technology
- Cache µArch
- Prefetching

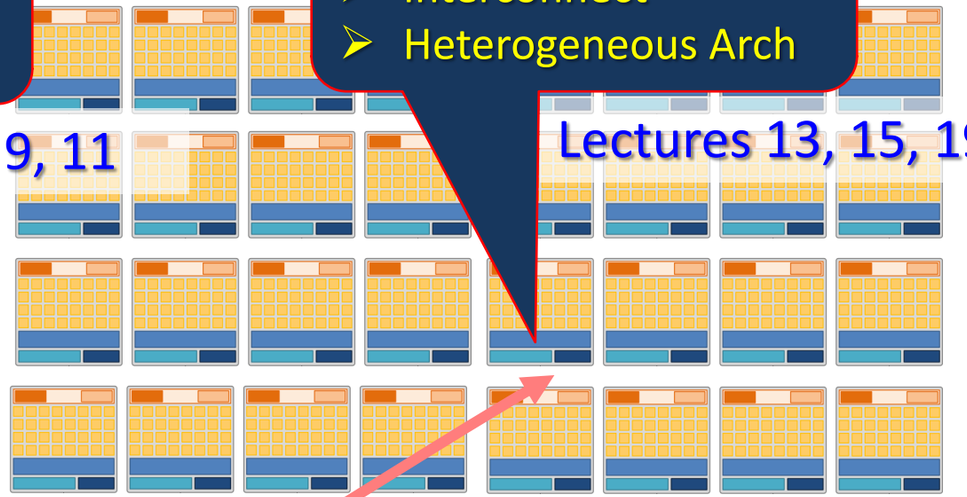
Lectures 8, 9, 11

- Unified Memory
- Interconnect
- Heterogeneous Arch

Lectures 13, 15, 19







Unified Private Cache (SRAM)

Unified Private Cache (SRAM)

Multicore

Unified Shared Cache (SRAM)

GPU caches

Interconnect

Multiagent

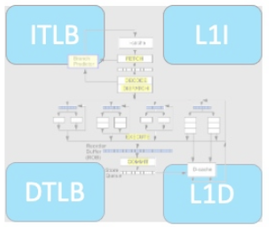
Singlecore

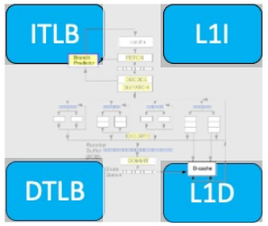
Main Memory (DRAM)

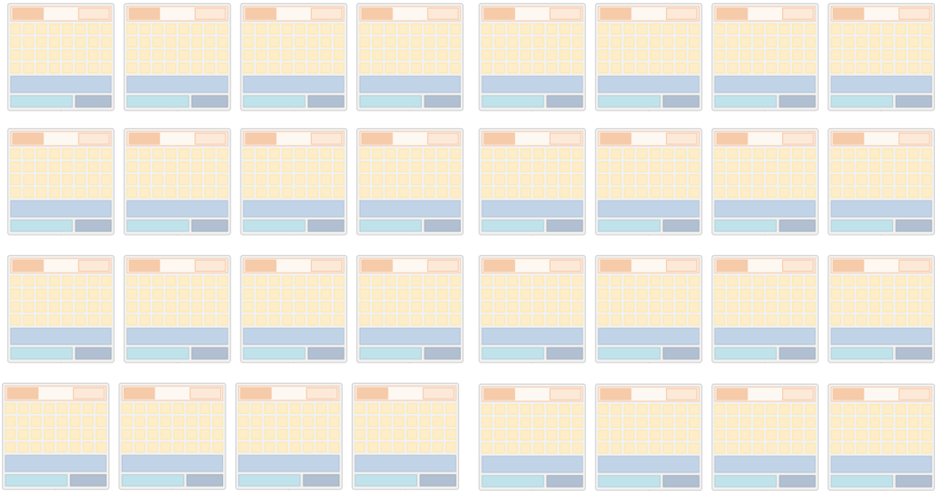
Potentially HBM or other memory

Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

Memory Data Flow : Split Top-level Caches







Unified Private Cache (SRAM)

Unified Private Cache (SRAM)

Unified Shared Cache (SRAM)

GPU caches

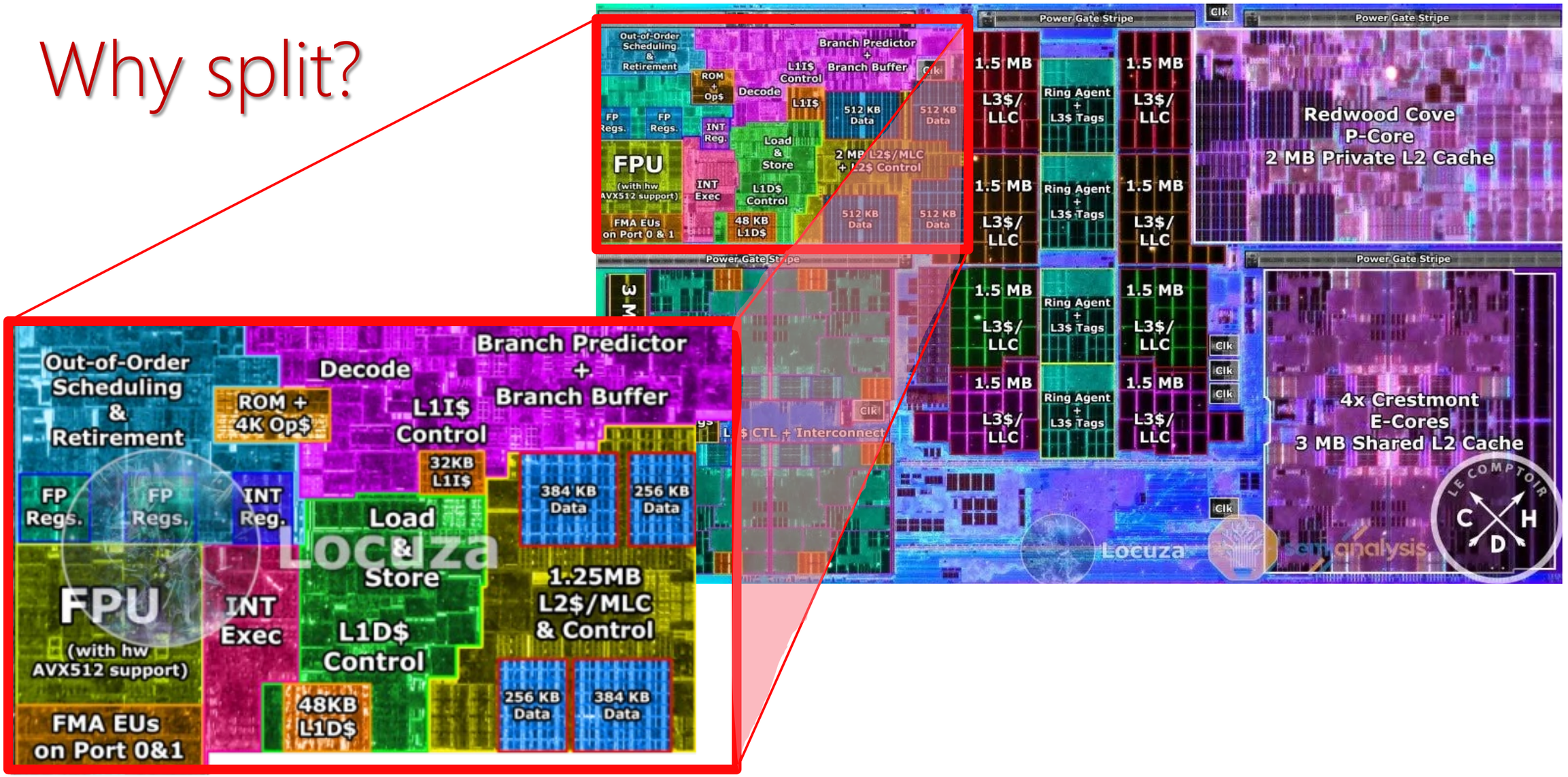
Interconnect

Main Memory (DRAM)

Potentially HBM or other memory

Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

Why split?



Meteor Lake die shot by Nicolas Derumigny / comptoir-hardware.com - www.comptoir-hardware.com
Die shot interpretation by Locuza - https://twitter.com/Locuza_

Why split?

- Besides physical layout constraints, several algorithmic arguments for splitting
- Instruction/Data/MMU memory all exhibit different access patterns
  - Cache set/way organization
  - Replacement policy
  - Writeback/through policy
- It may make sense to handle ***Coherency*** differently
  - I.e. software versus hardware coherence

Before we move on…
Things you should know

slides graciously borrowed from 18-447, James C. Hoe, 18-213, David O'Hallaron

Recall - aBCs of Caches

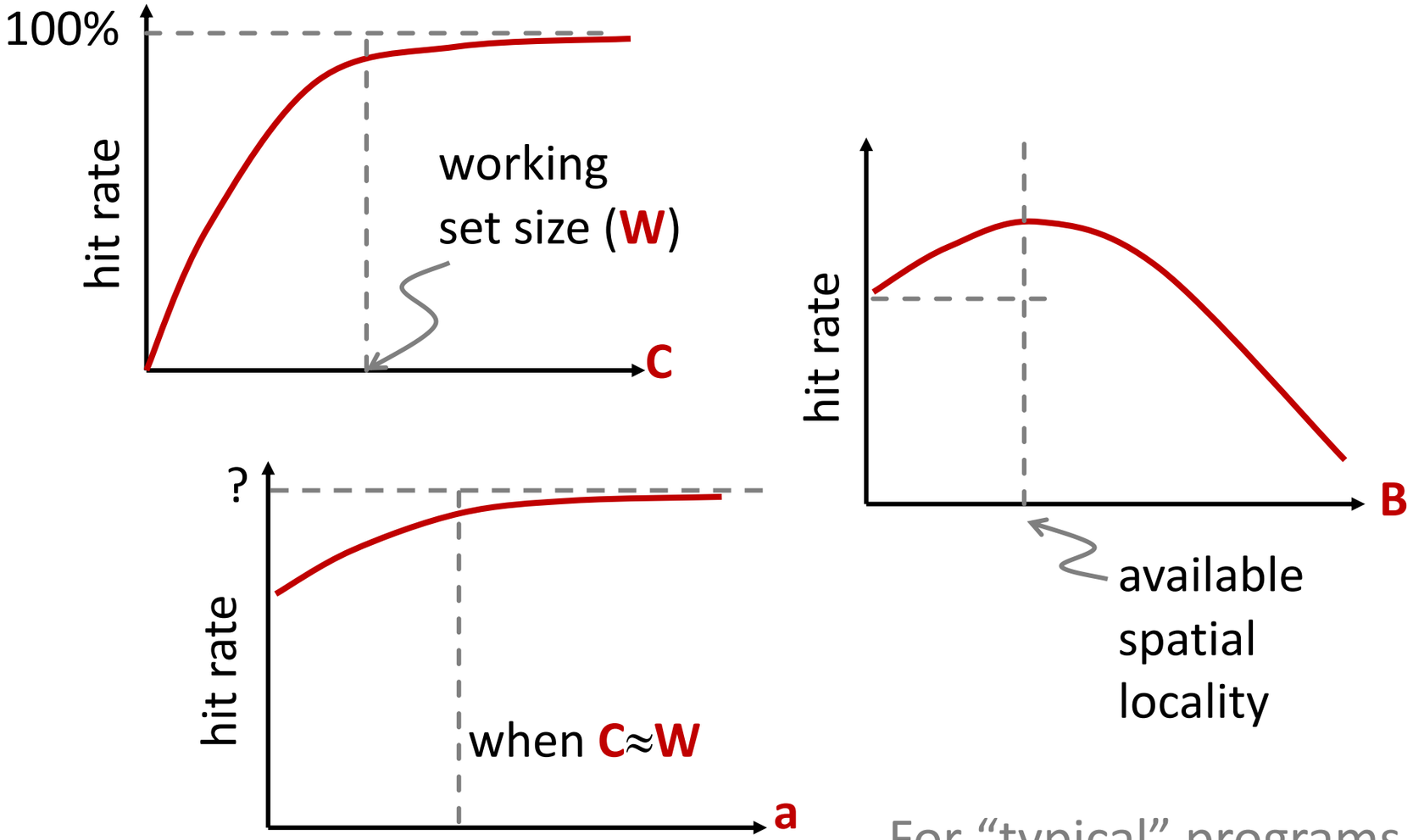
- **M = 2^m** : size of address space in bytes
  example values: 2^{32}, 2^{64}
- **G=2^g** : cache access granularity in bytes
  example values: 4, 8

- **C** : "capacity" of cache in bytes
  example values: 16 KByte (L1), 1 MByte (L2)
- **B = 2^b**: "block size" in bytes
  example values: 16 (L1), >64 (L2)
- **a**: "associativity" of the cache
  example values: 1, 2, 4, 5(?),... "C/B"

ISA

Implementation

Recall - aBCs of Caches

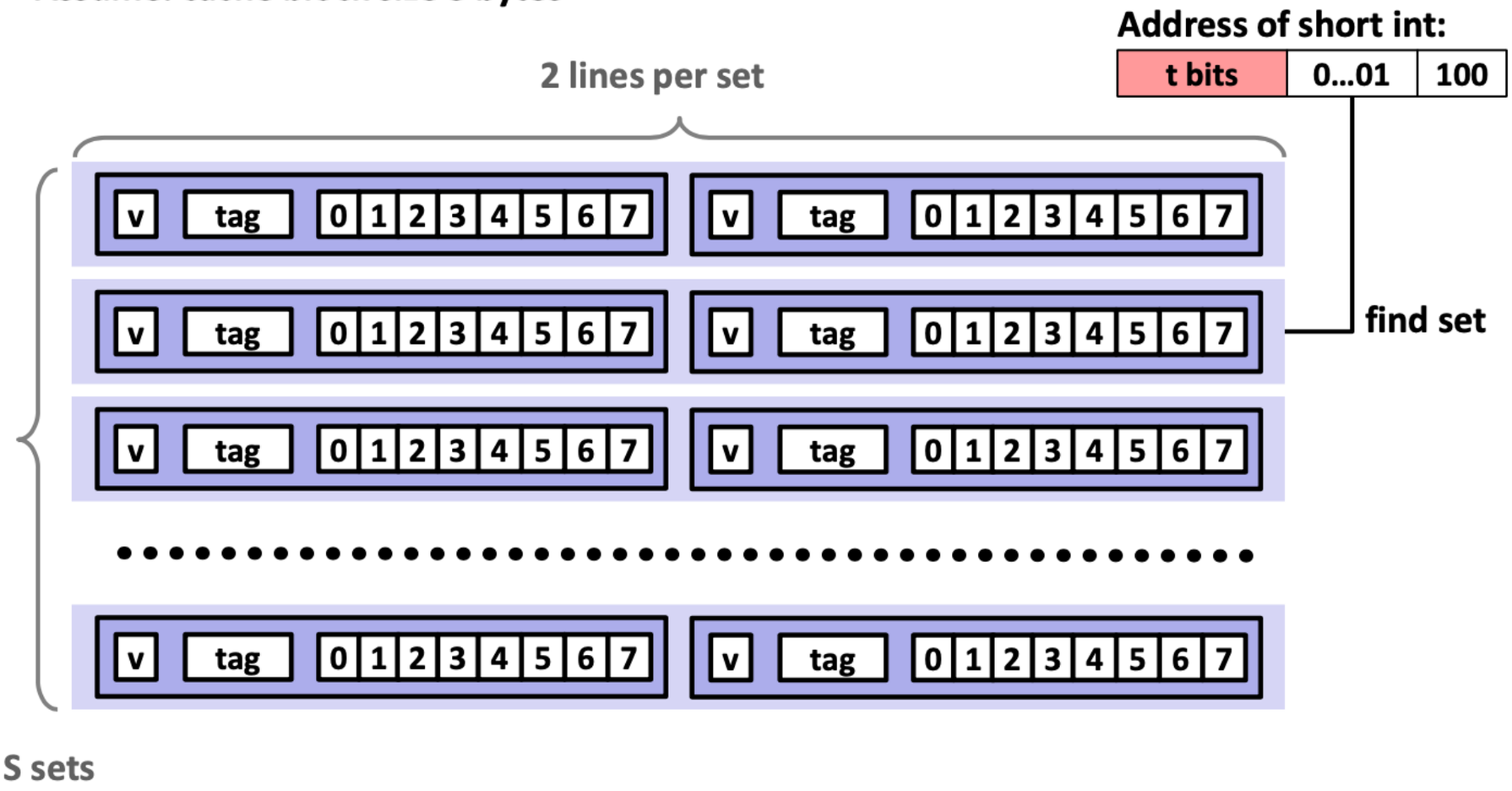


For "typical" programs

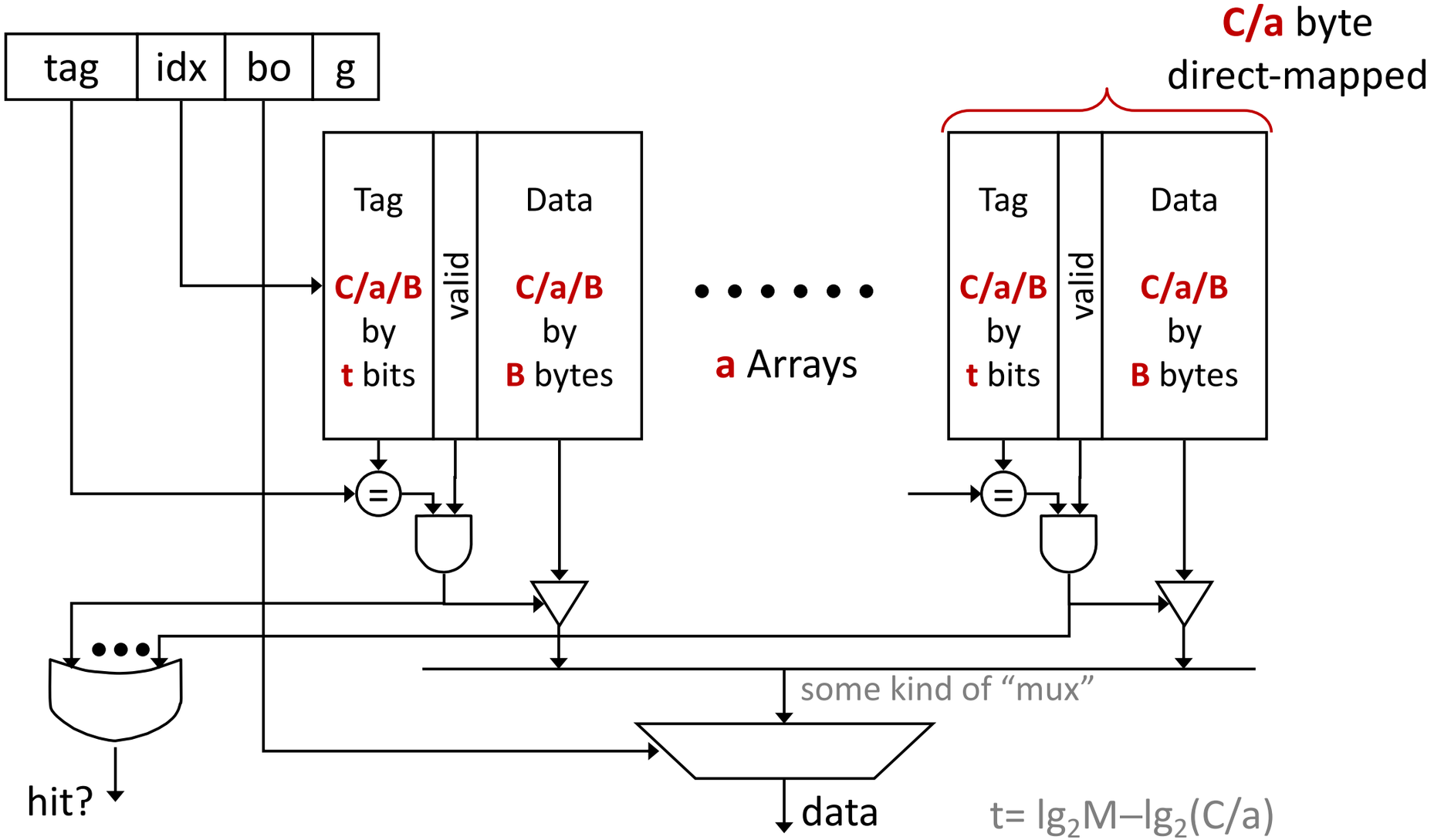
Recall - Abstract Set-Associative Cache (213)

a ~~E~~ = 2: Two lines per set

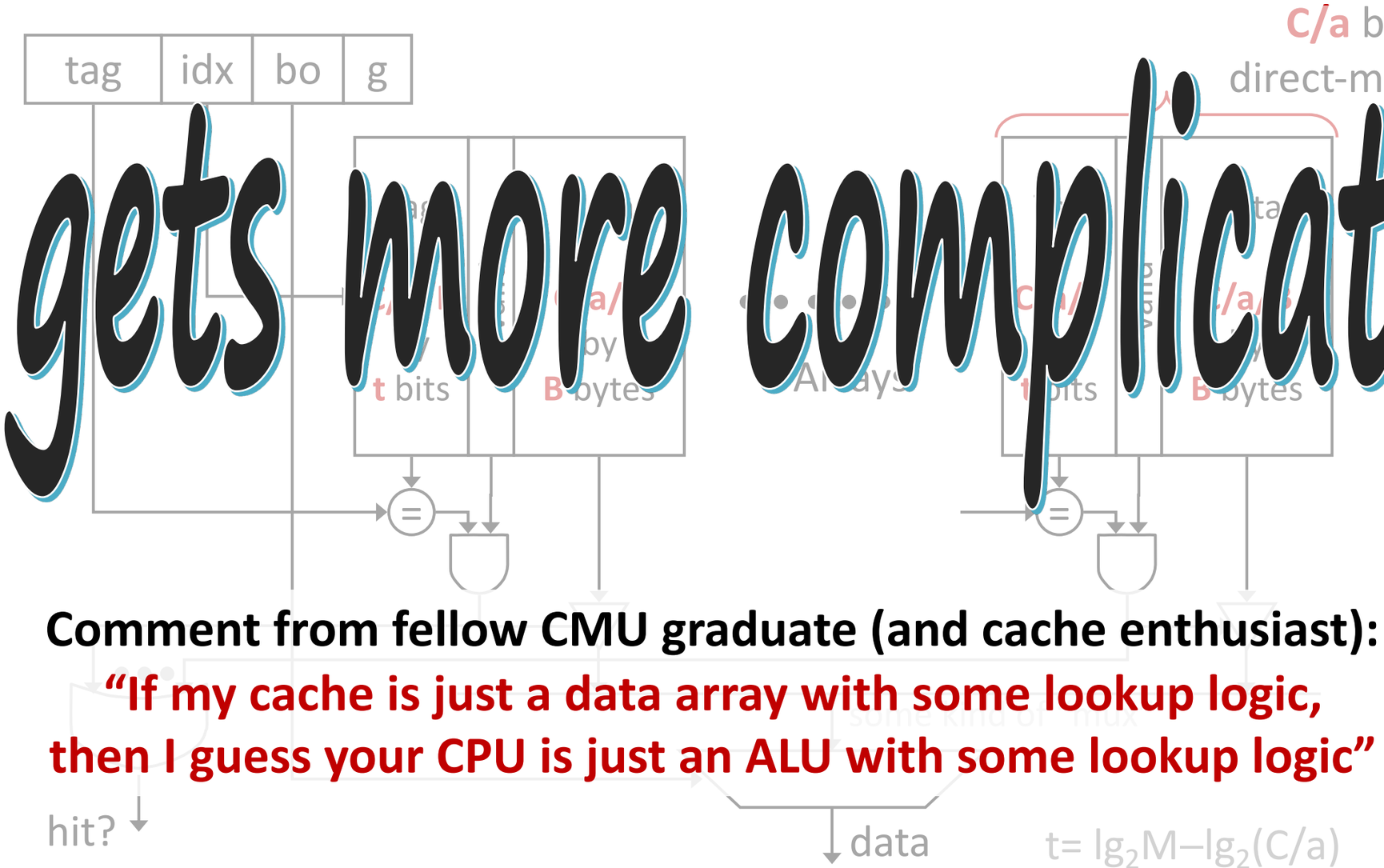
Assume: cache block size 8 bytes



Recall - Set-Associative Cache (447)



Recall - Set-Associative Cache (447)



It gets more complicated

Comment from fellow CMU graduate (and cache enthusiast):

"If my cache is just a data array with some lookup logic, then I guess your CPU is just an ALU with some lookup logic"

Other things to recall

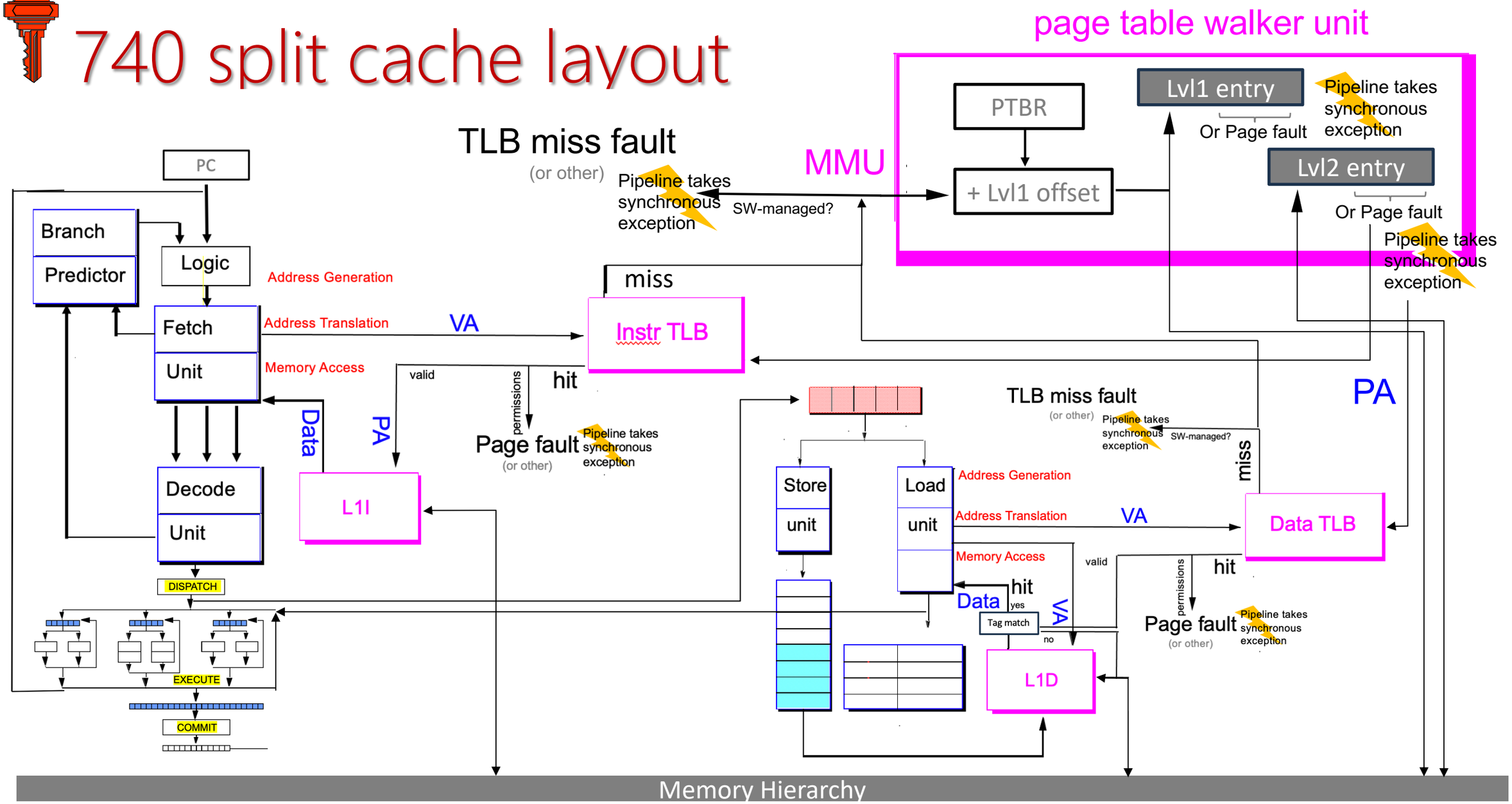
- Cold, Conflict, Capacity misses
- Basic replacement policies
  - LRU, MRU, Belady's
  - And why these only matter on non-direct-mapped caches
- Write-back vs write-through caches
  - And tradeoffs
- Inclusivity vs Non-Inclusivity vs Exclusivity
  - We will revisit this point
- **If these terms are unfamiliar**
  - Ask us (now or later). We can help!
  - Or (a good practice to get into) see if you can find the answer first

Moving on…

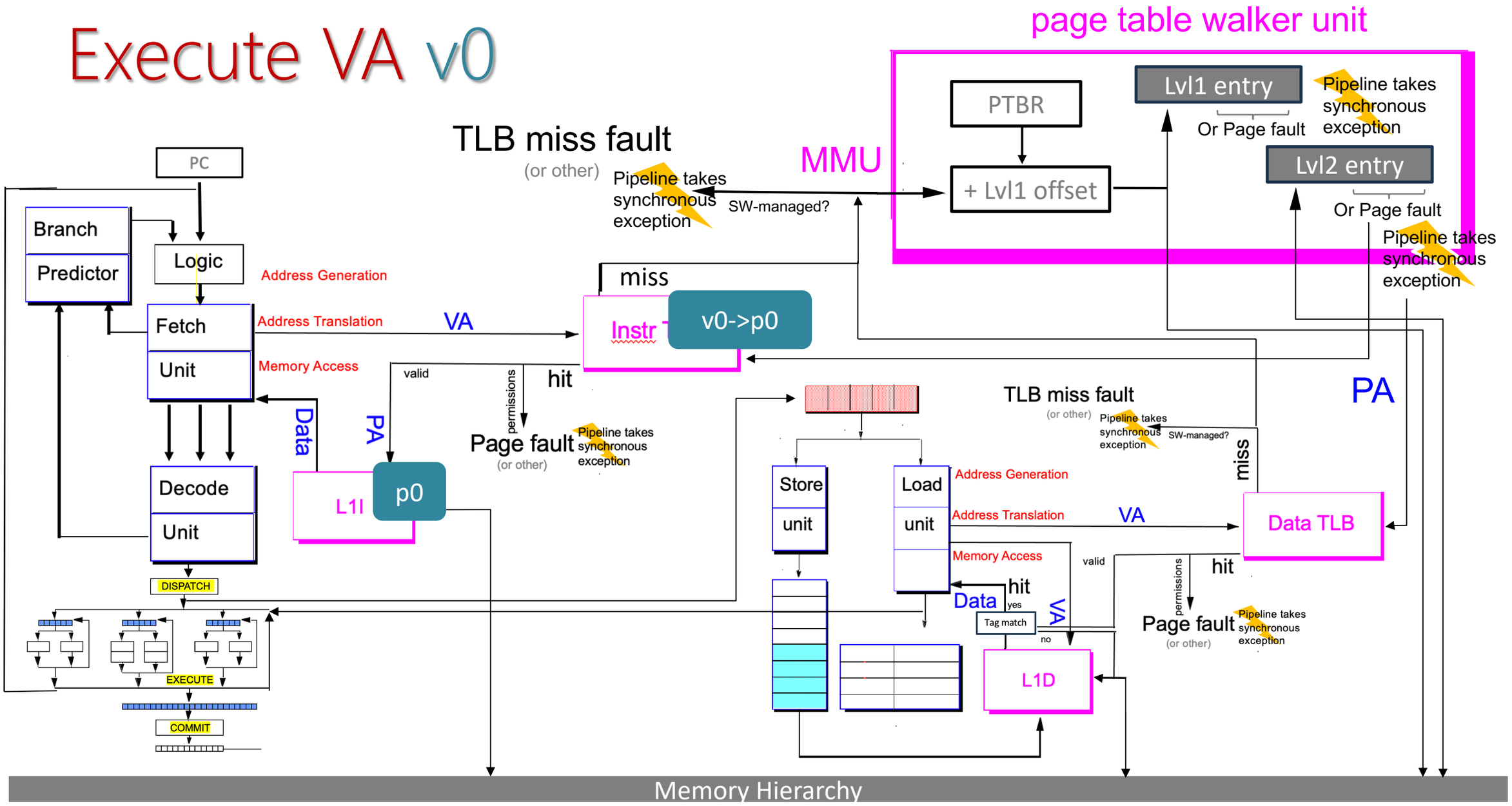
We left off Tuesday with two problems…

- **Writing MMU or Instruction cache lines as data**
- **Virtual indexing alias problem**
- **Our plan:**
  - Solve these problems… and work our way down the hierarchy

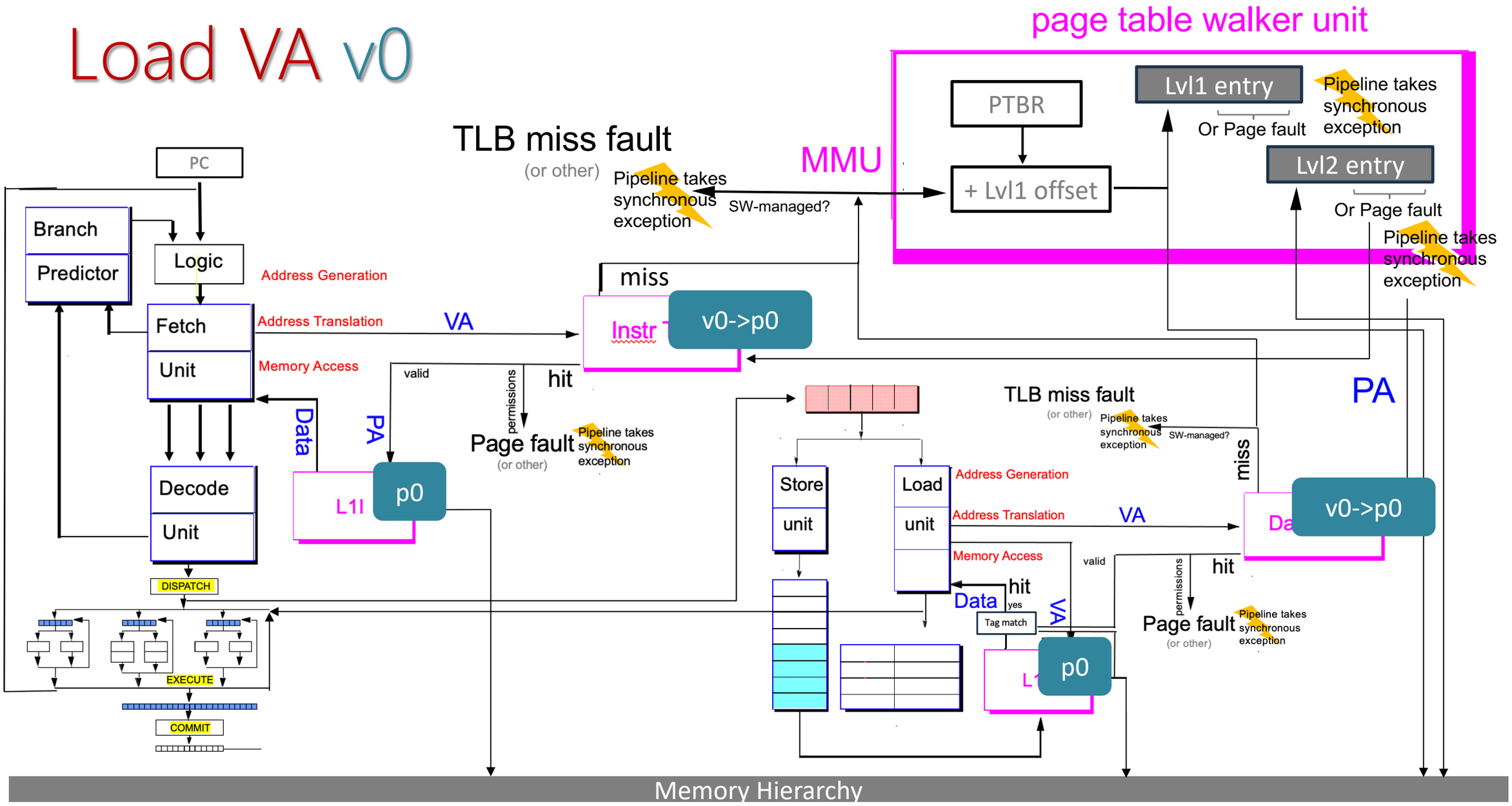
740 split cache layout



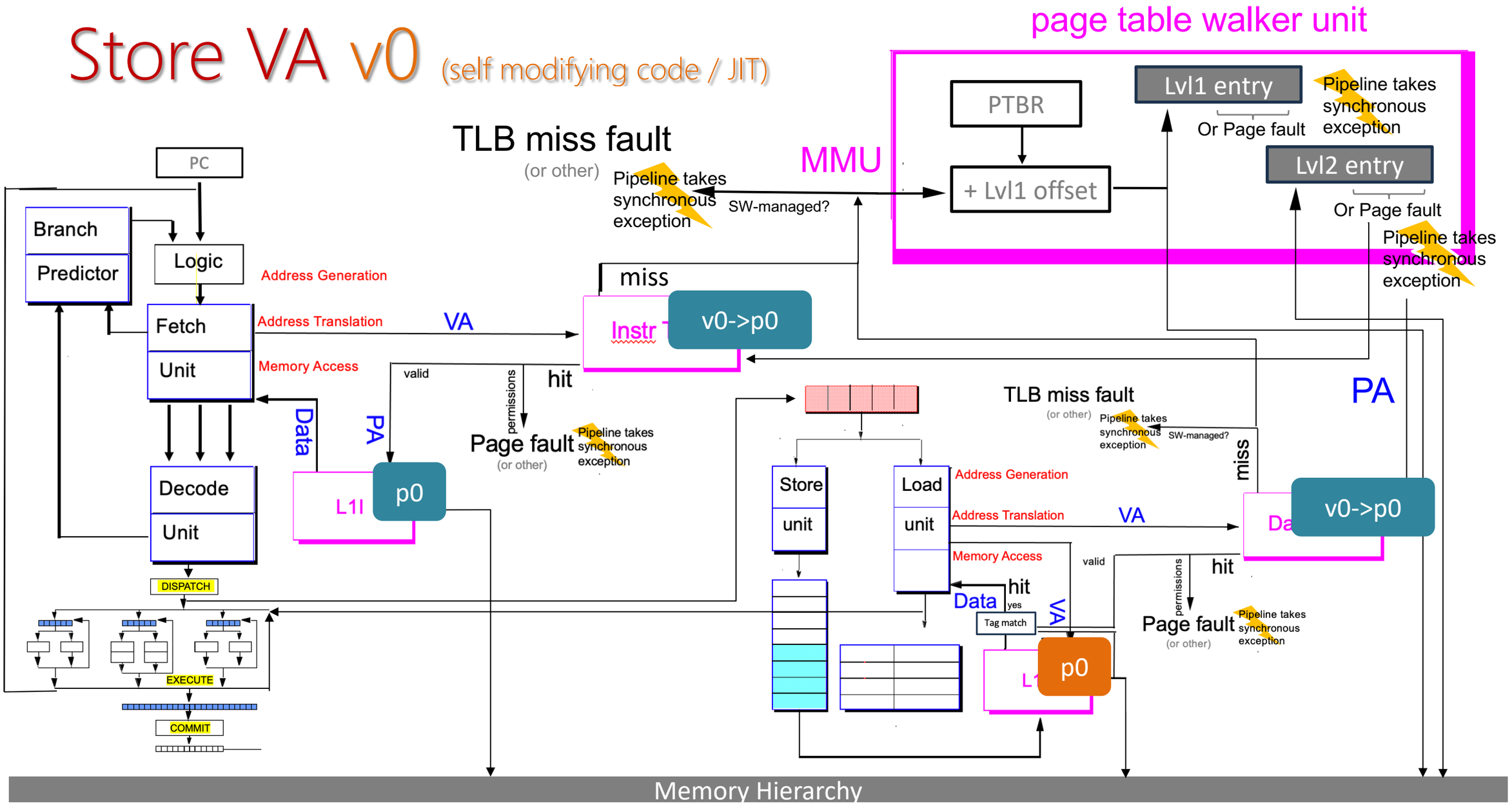
Execute VA v0



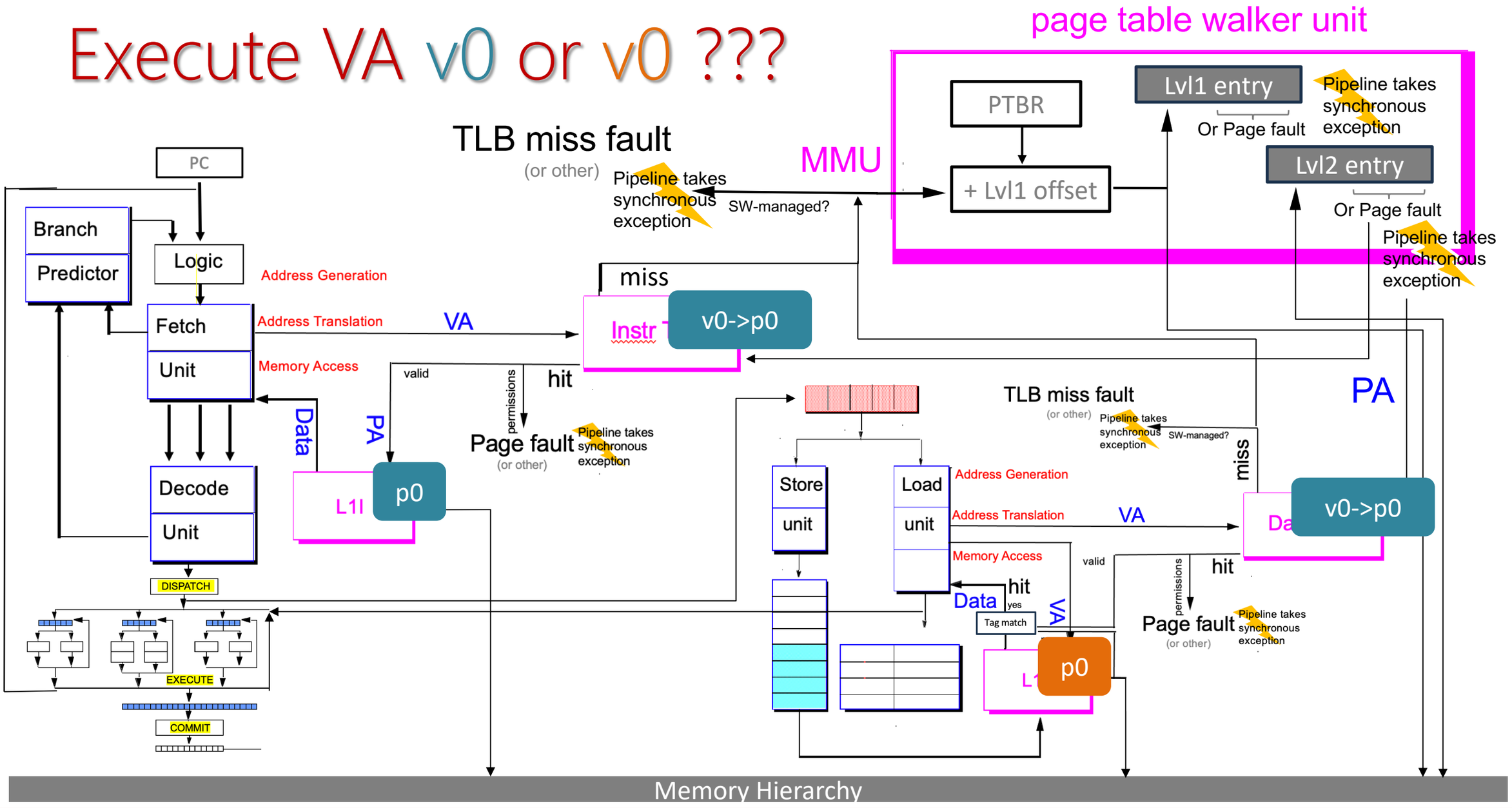
Load VA v0


page table walker unit
PTBR
Lvl1 entry
Pipeline takes synchronous exception
Or Page fault
Lvl2 entry
Or Page fault
Pipeline takes synchronous exception
+ Lvl1 offset
MMU
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
PC
Branch
Predictor
Logic
Address Generation
Fetch
Unit
Address Translation
VA
Memory Access
miss
Instr T
v0->p0
valid
permissions
hit
Page fault
(or other)
Pipeline takes synchronous exception
Data
PA
L1I
p0
Decode
Unit
DISPATCH
EXECUTE
COMMIT
Store
unit
Load
unit
Address Generation
Address Translation
VA
Memory Access
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
miss
PA
v0->p0
valid
permissions
hit
Page fault
(or other)
Pipeline takes synchronous exception
hit
Data
yes
Tag match
VA
p0
Memory Hierarchy

Store VA v0 (self modifying code / JIT)



Execute VA v0 or v0 ???



For instructions written as data

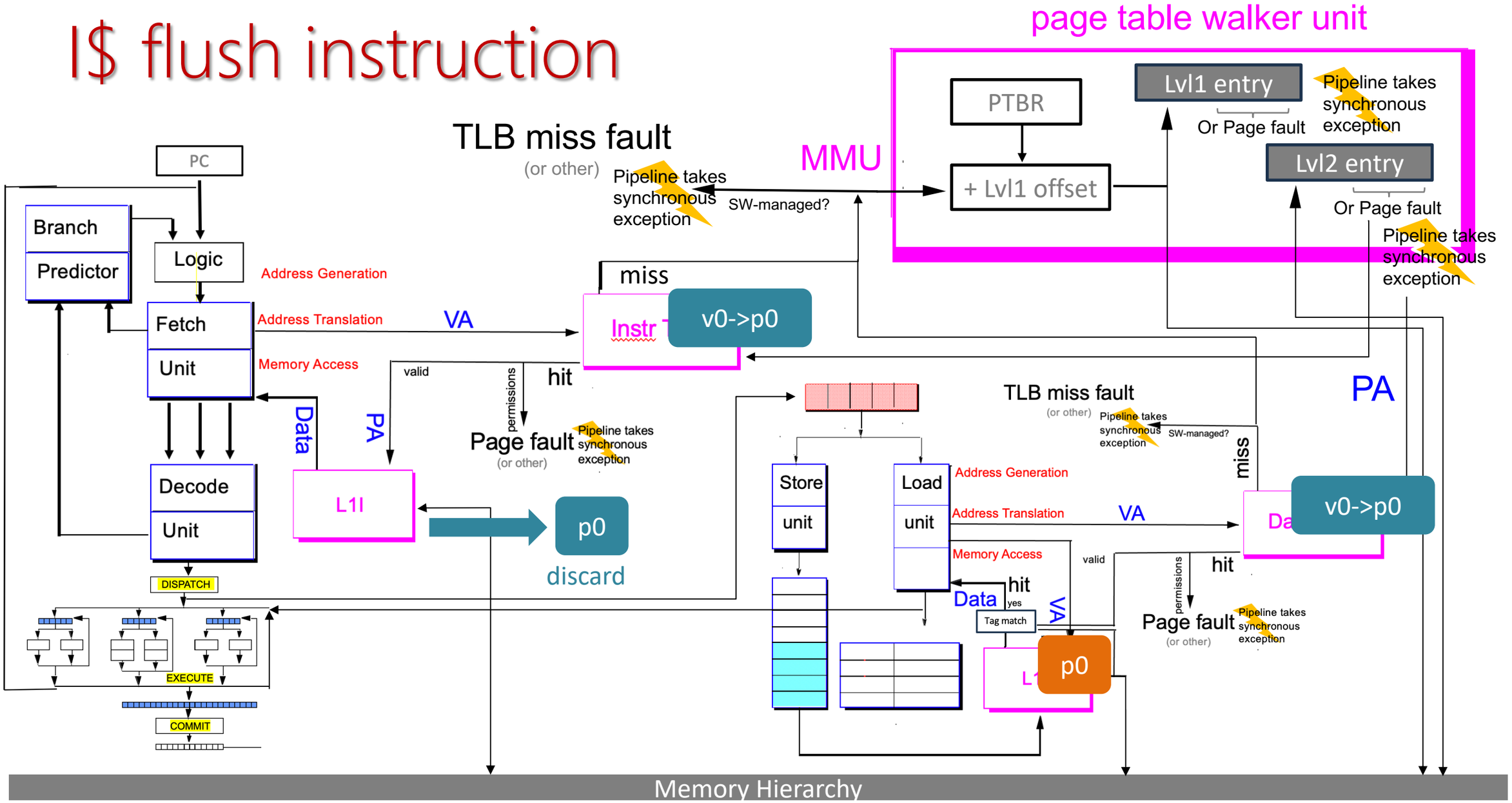
- Could solve with hardware coherence
  - Extra hardware in the core dedicated to detecting aliasing (shared bus, broadcast, etc…)
  - Extra hardware somewhere else in the memory hierarchy (such as a lower cache)
- But this is ***usually infrequent***
  - Read that not as "we don't have to solve" but as "we can solve cheaply"
  - Arm/x86 leave I-Cache coherence up to ***Software***

If write data you intend to execute, Programmer calls something like

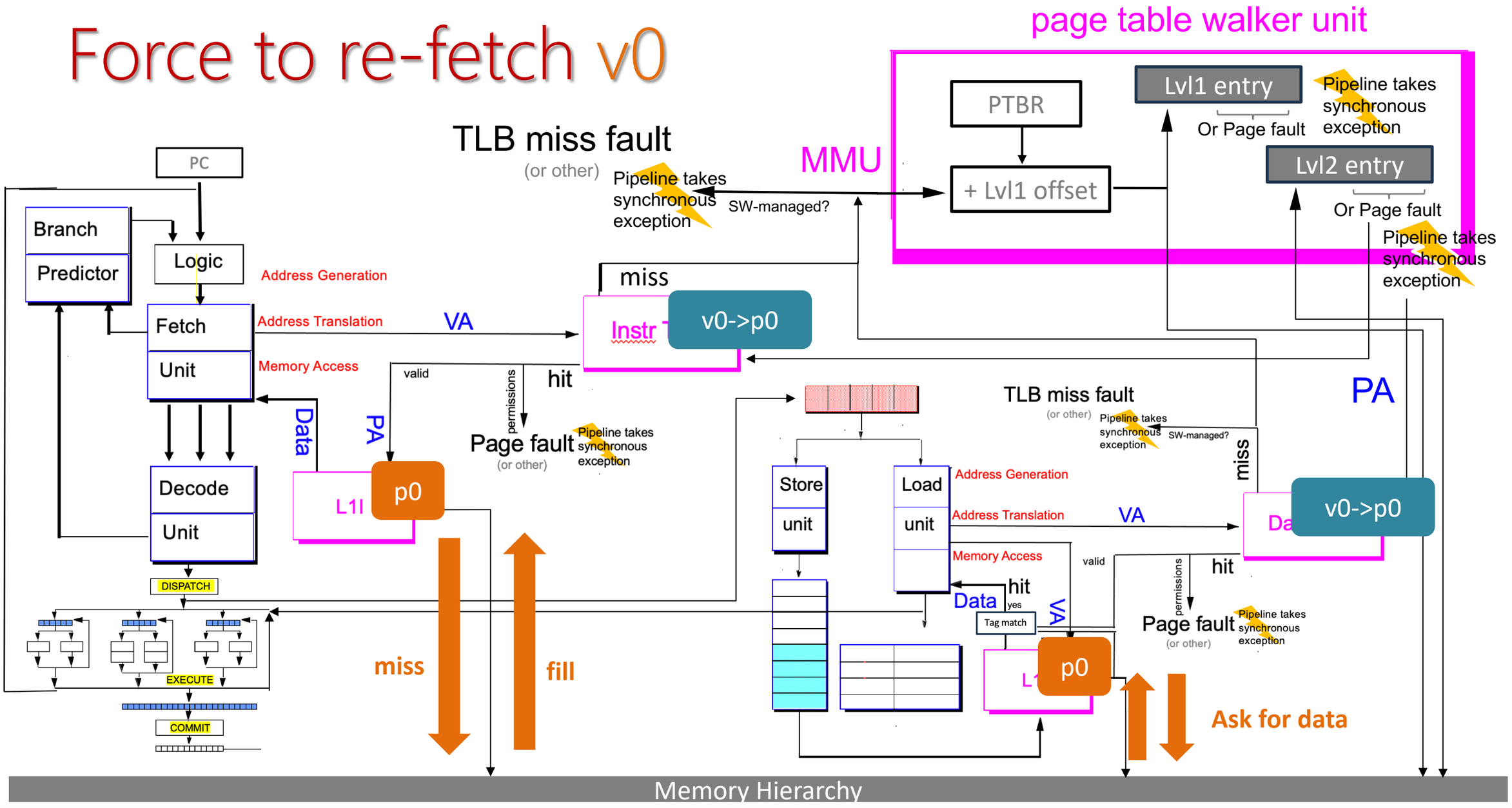
```
_platform_pre_stackheap_init:
    dsb ish      // ensure all previous stores have completed
                 // before invalidating
    ic ialluis   // I cache invalidate all inner shareable to PoU
                 // (which includes secondary cores)
    dsb  ish     // ensure completion on inner shareable domain
                 // (which includes secondary cores)
    isb
    b InvalidateUDCaches    // only needed if the MMU is on at this point
```

From Arm developer website - https://developer.arm.com/documentation/dui0808/j/the-c-and-c---library-functions-reference/-platform-pre-stackheap-init--

I$ flush instruction



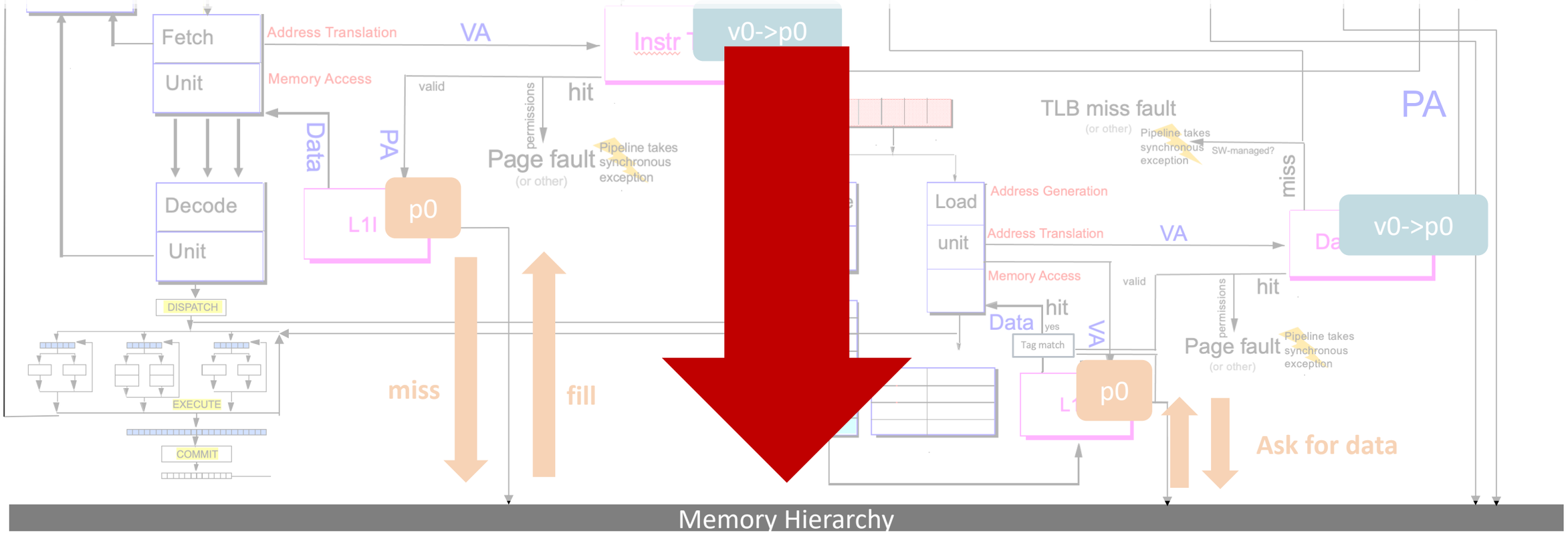
Force to re-fetch v0



Must know to ask L1D for the dirty data

Some amount of state tracking…

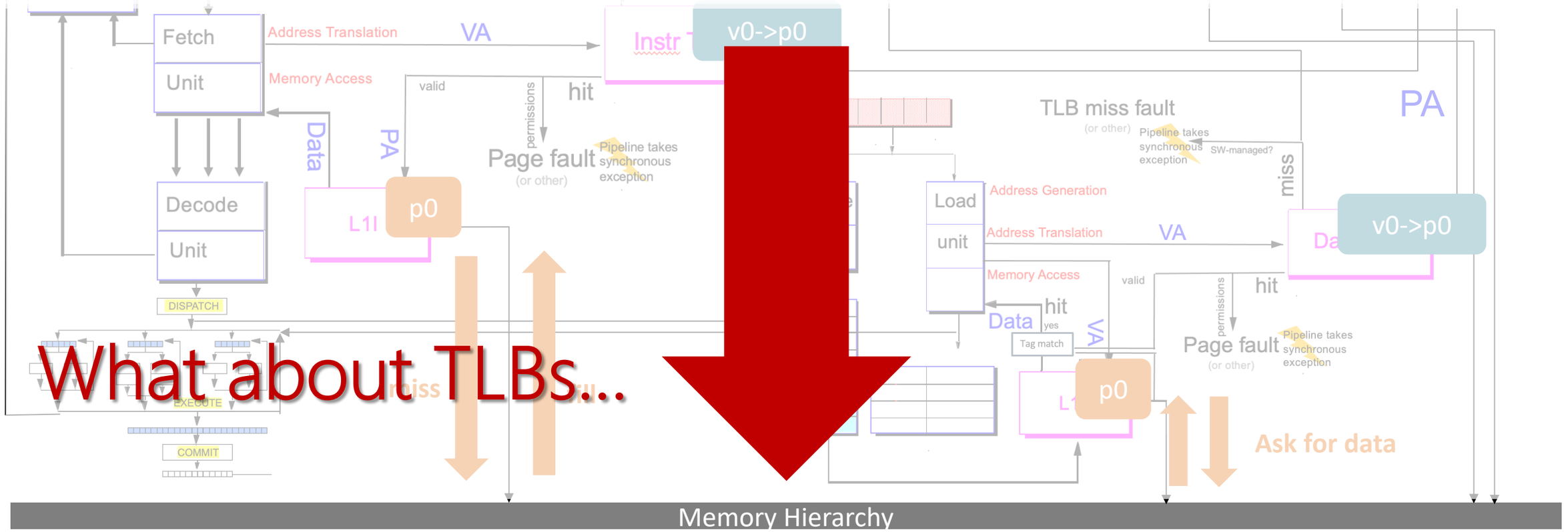
Revisit with directory coherence



Must know to ask L1D for the dirty data

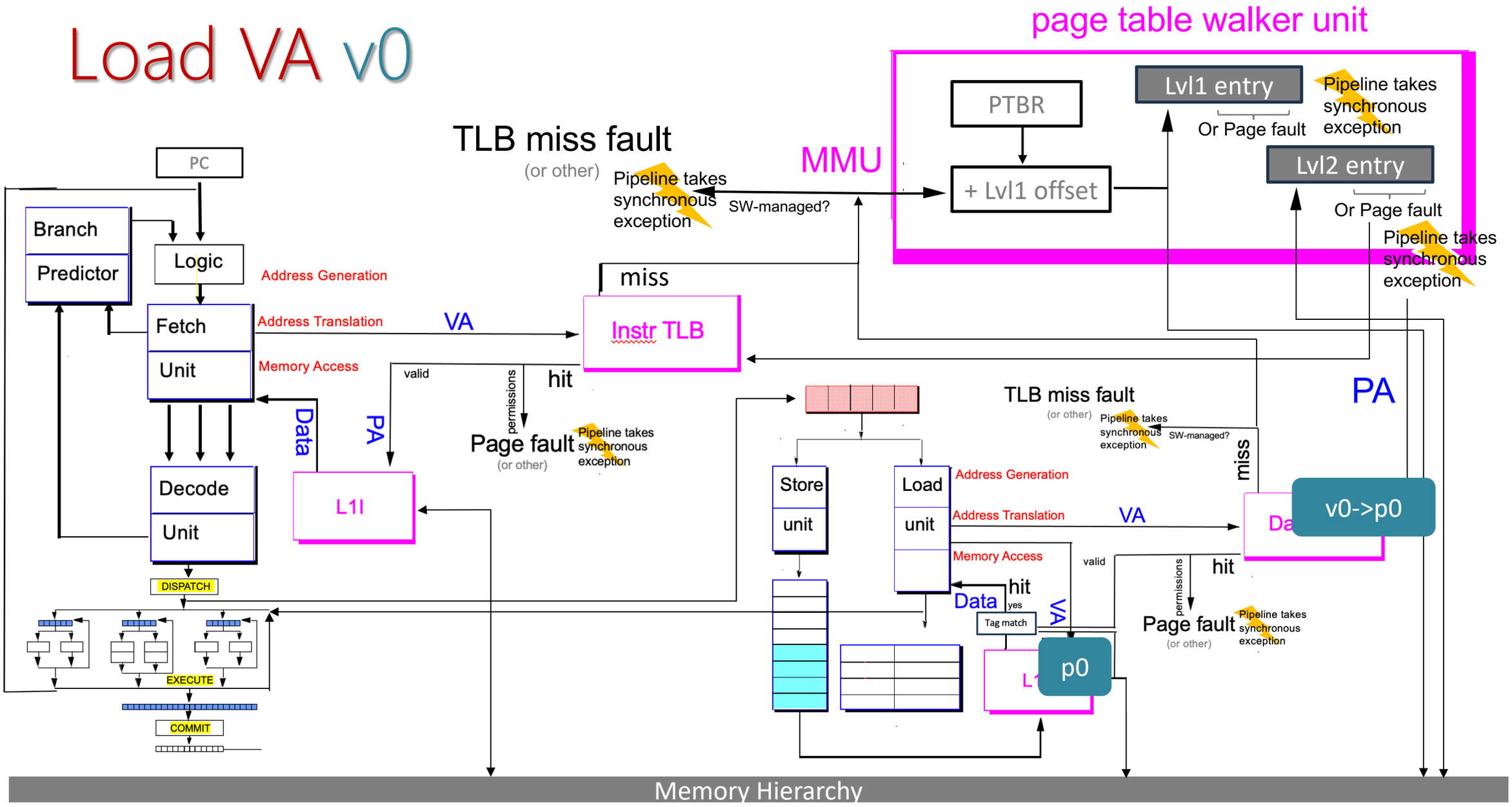
Some amount of state tracking…

Revisit with directory coherence

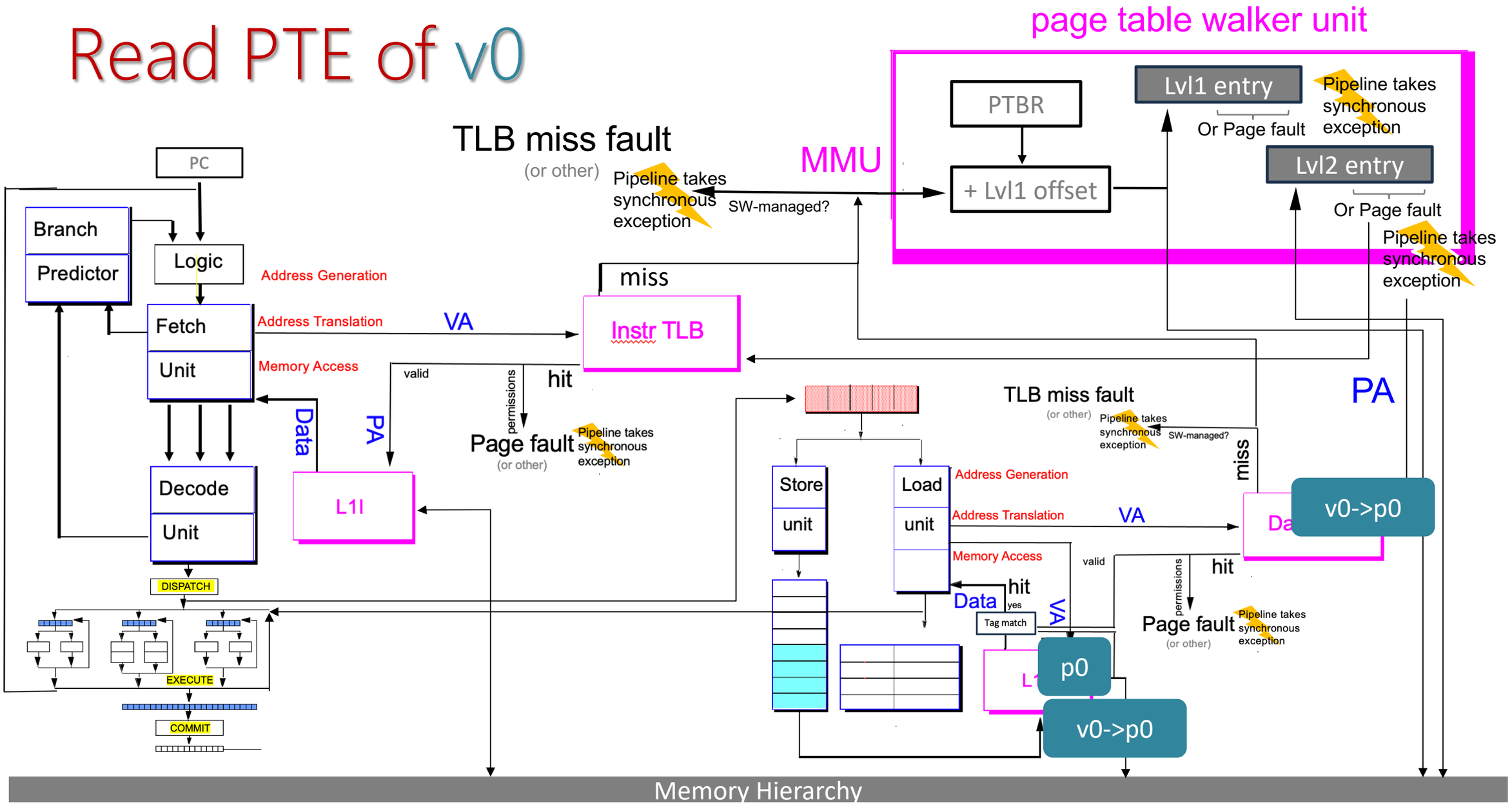


What about TLBs…

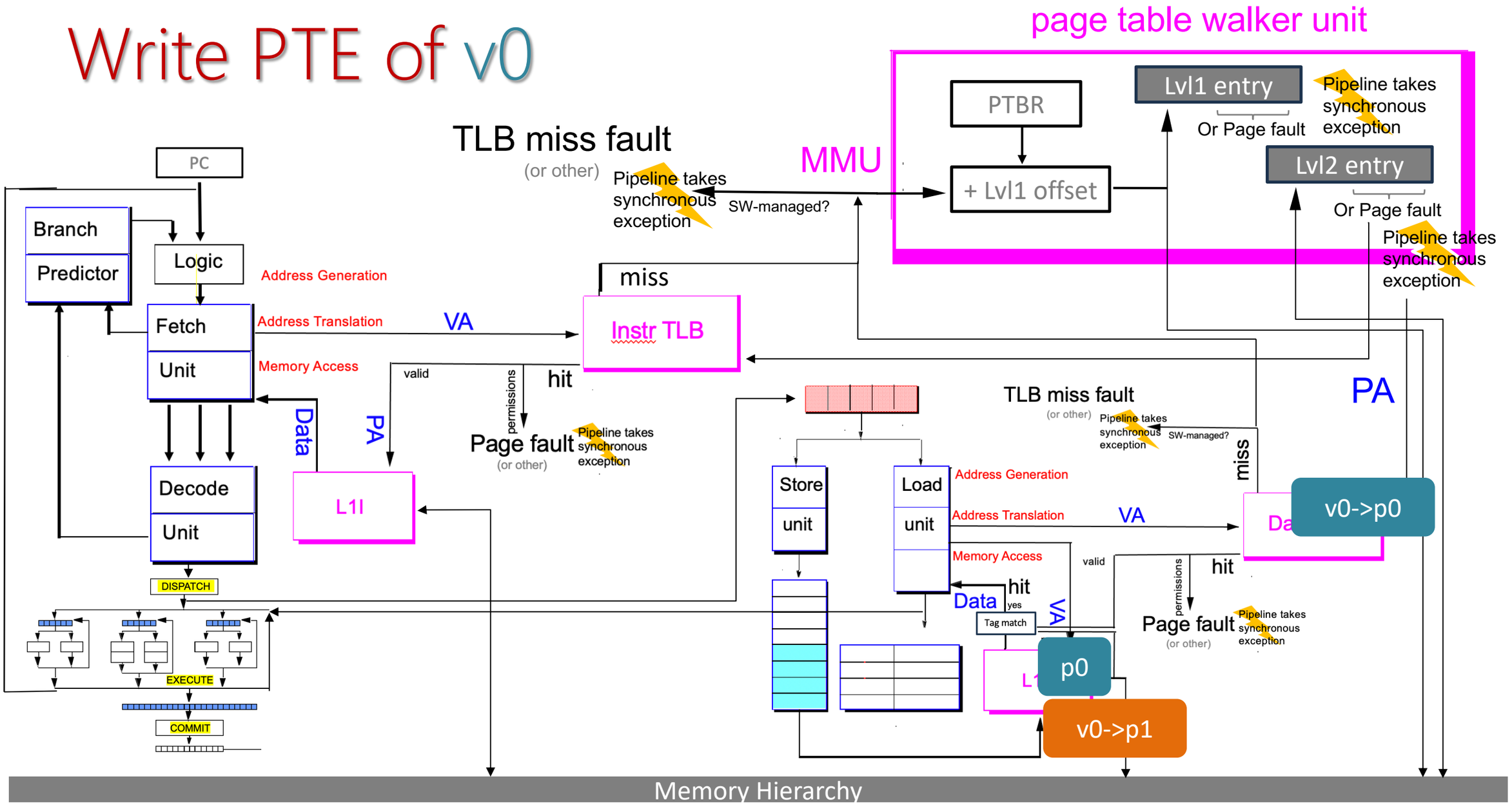
Load VA v0


page table walker unit
PTBR
Lvl1 entry
Pipeline takes synchronous exception
Or Page fault
Lvl2 entry
Or Page fault
Pipeline takes synchronous exception
+ Lvl1 offset
MMU
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
PC
Branch
Predictor
Logic
Address Generation
Fetch
Unit
Address Translation
VA
Memory Access
miss
Instr TLB
valid
permissions
hit
Page fault
(or other)
Pipeline takes synchronous exception
Data
PA
L1I
Decode
Unit
DISPATCH
EXECUTE
COMMIT
Store
unit
Load
unit
Address Generation
Address Translation
VA
Memory Access
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
miss
PA
v0->p0
valid
permissions
hit
Data
hit
yes
Tag match
VA
p0
Page fault
(or other)
Pipeline takes synchronous exception
Memory Hierarchy

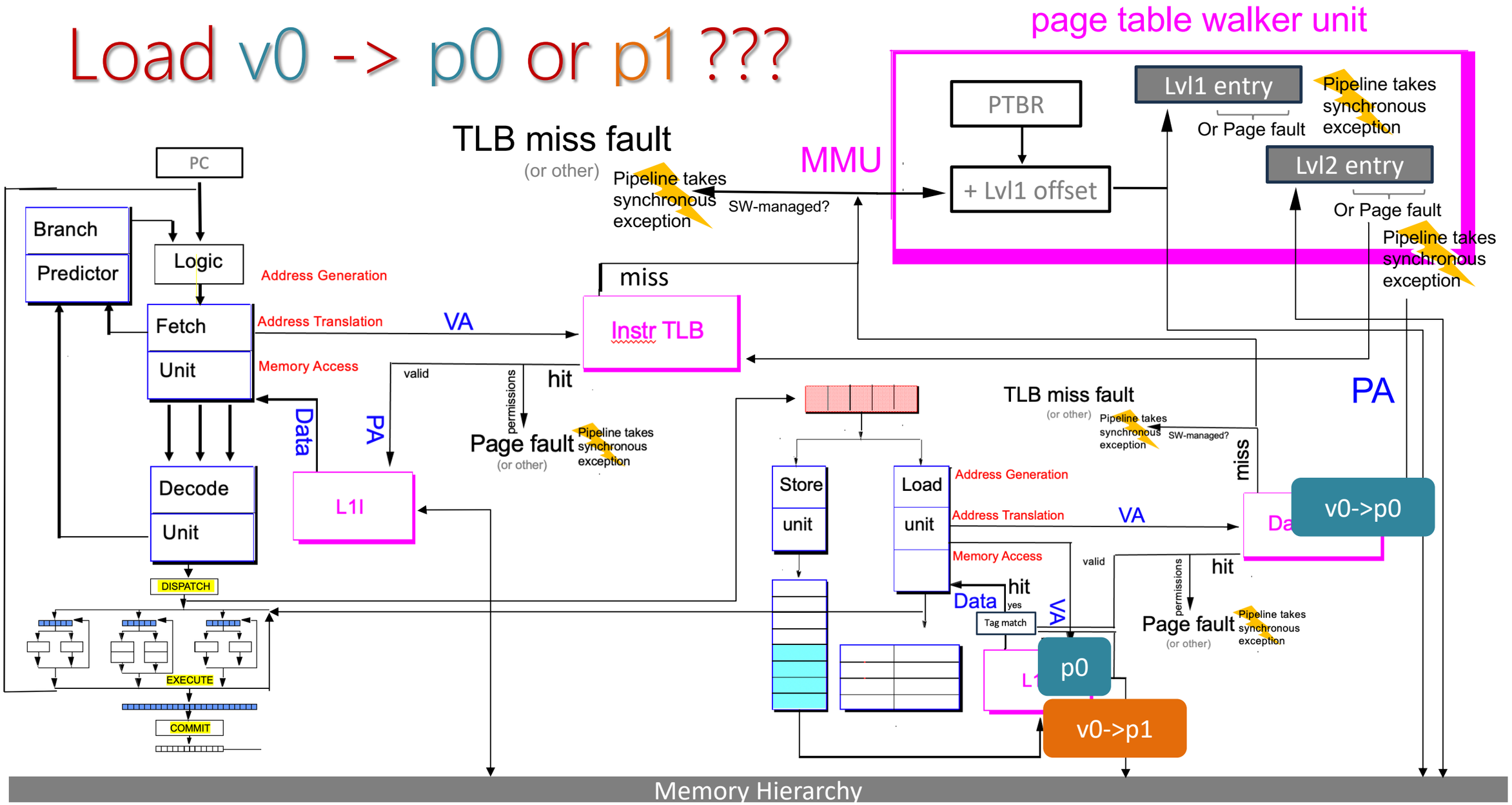
Read PTE of v0



Write PTE of v0



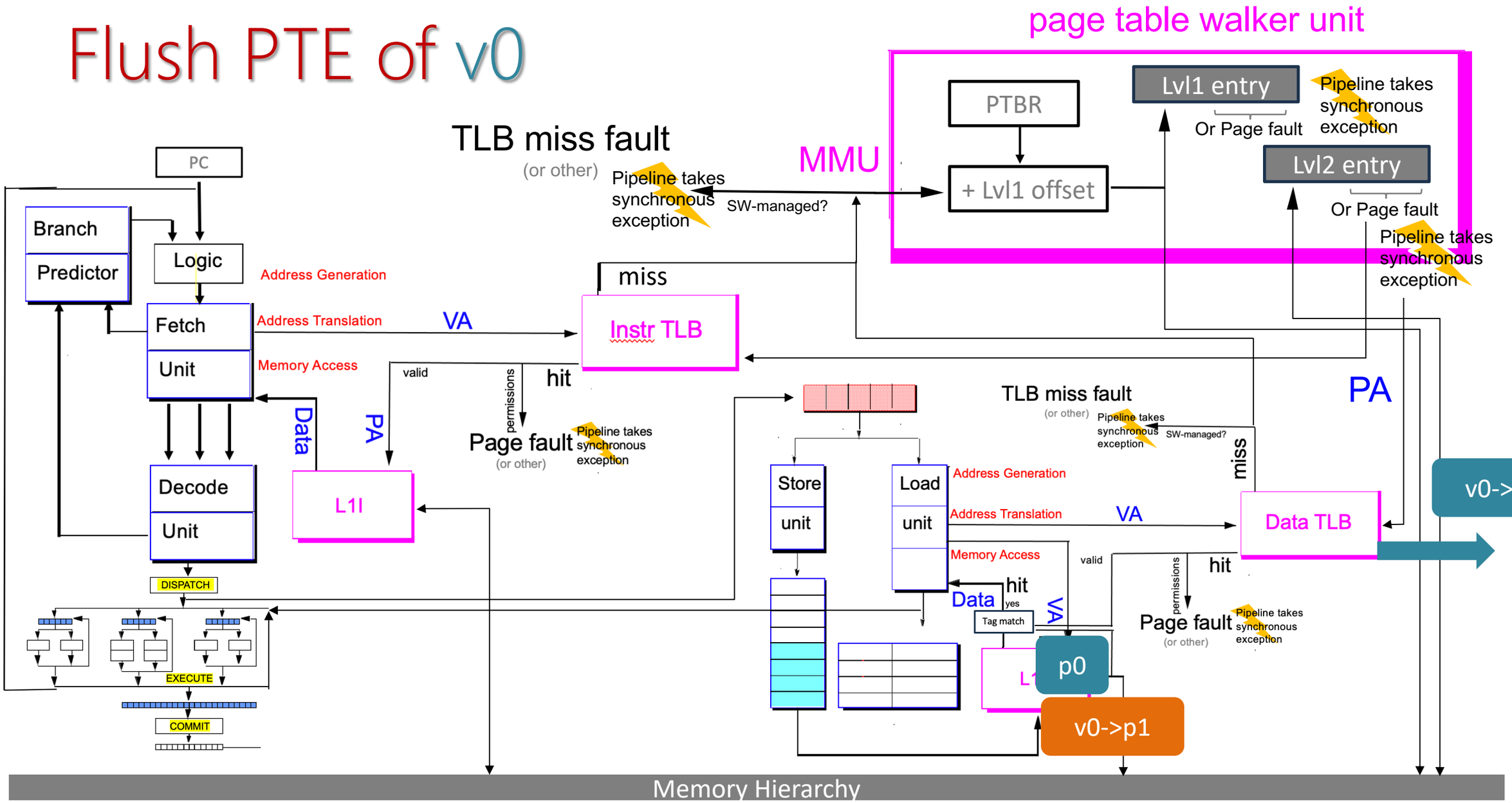
Load v0 -> p0 or p1 ???



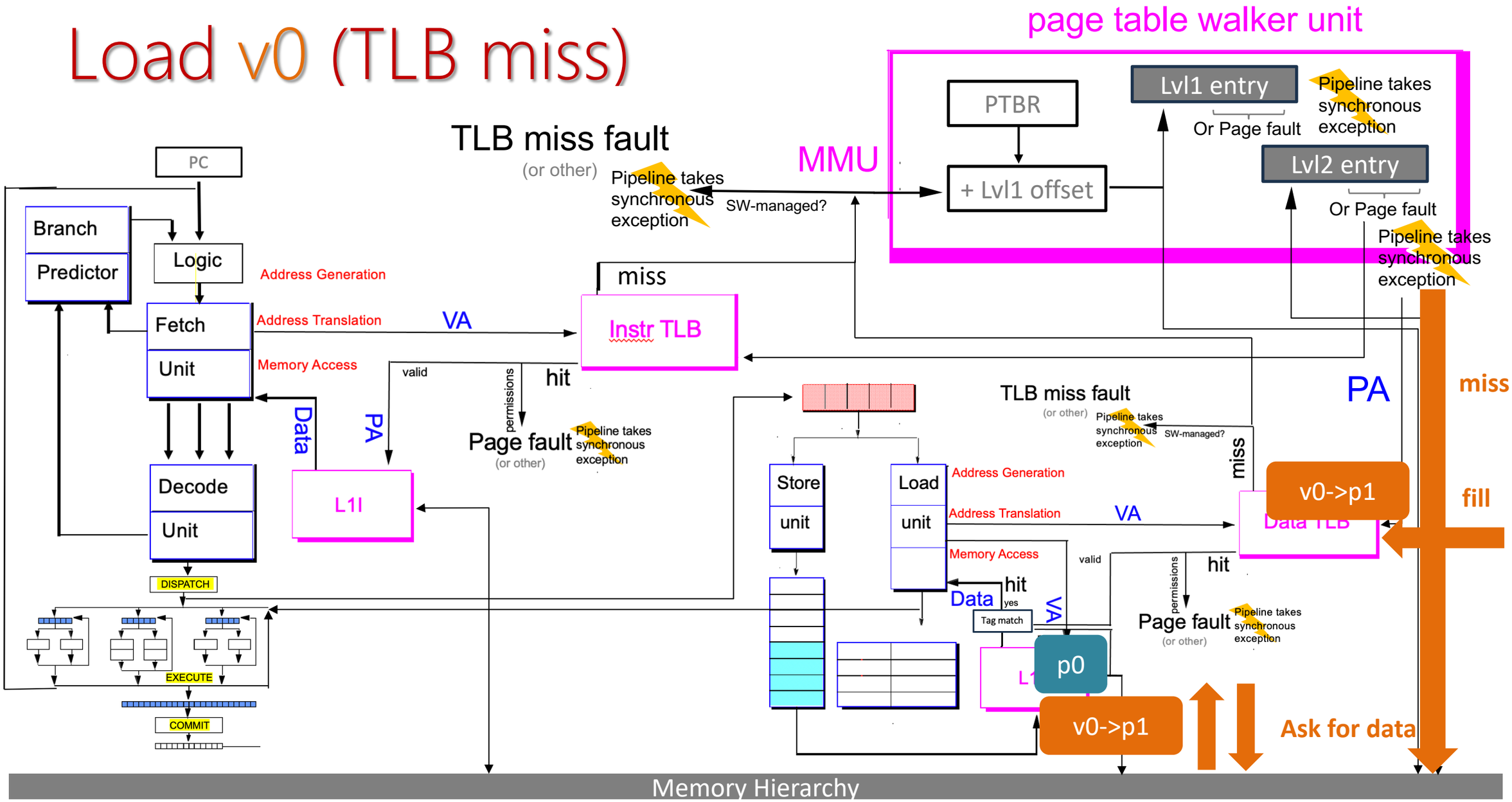
For Page Table Entries written as data

- Could ***also*** solve with hardware coherence
  - Extra hardware in the core dedicated to detecting aliasing (shared bus, broadcast, etc…)
  - Extra hardware somewhere else in the memory hierarchy (such as a lower cache)
- But this is ***also usually "infrequent"*** (in the loose sense)
  - fork/exec new process, thread spawn, heap extension (OS may have many reasons)
  - Arm/x86 also require explicit flush in ***Software***
- There exist a large array of various **TLB-invalidation instructions**
  - But usually need to be used in conjunction with barriers
  - **i.e. Arm break-before-make sequence**

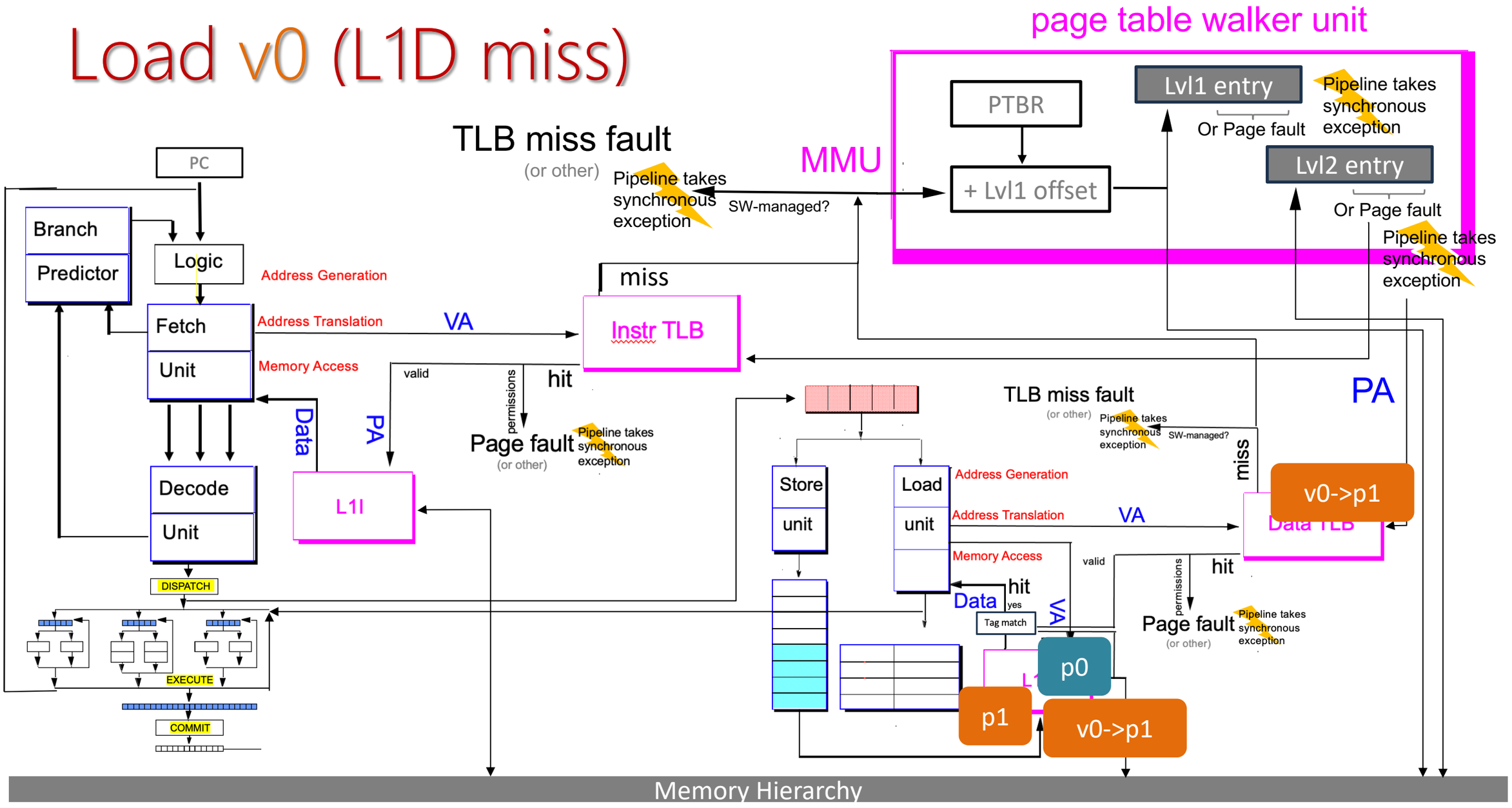
Flush PTE of v0



Load v0 (TLB miss)



Load v0 (L1D miss)

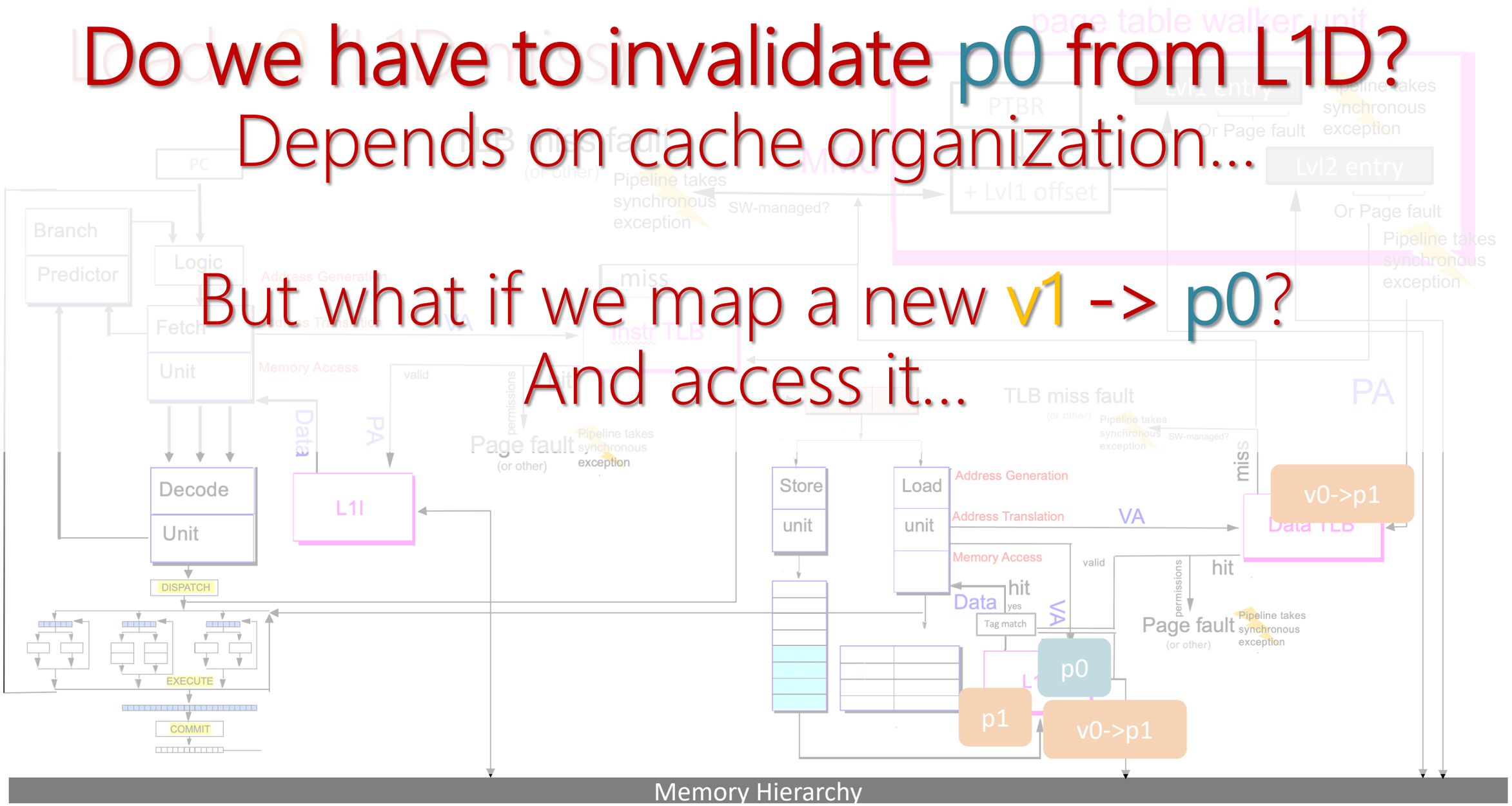


Do we have to invalidate p0 from L1D?

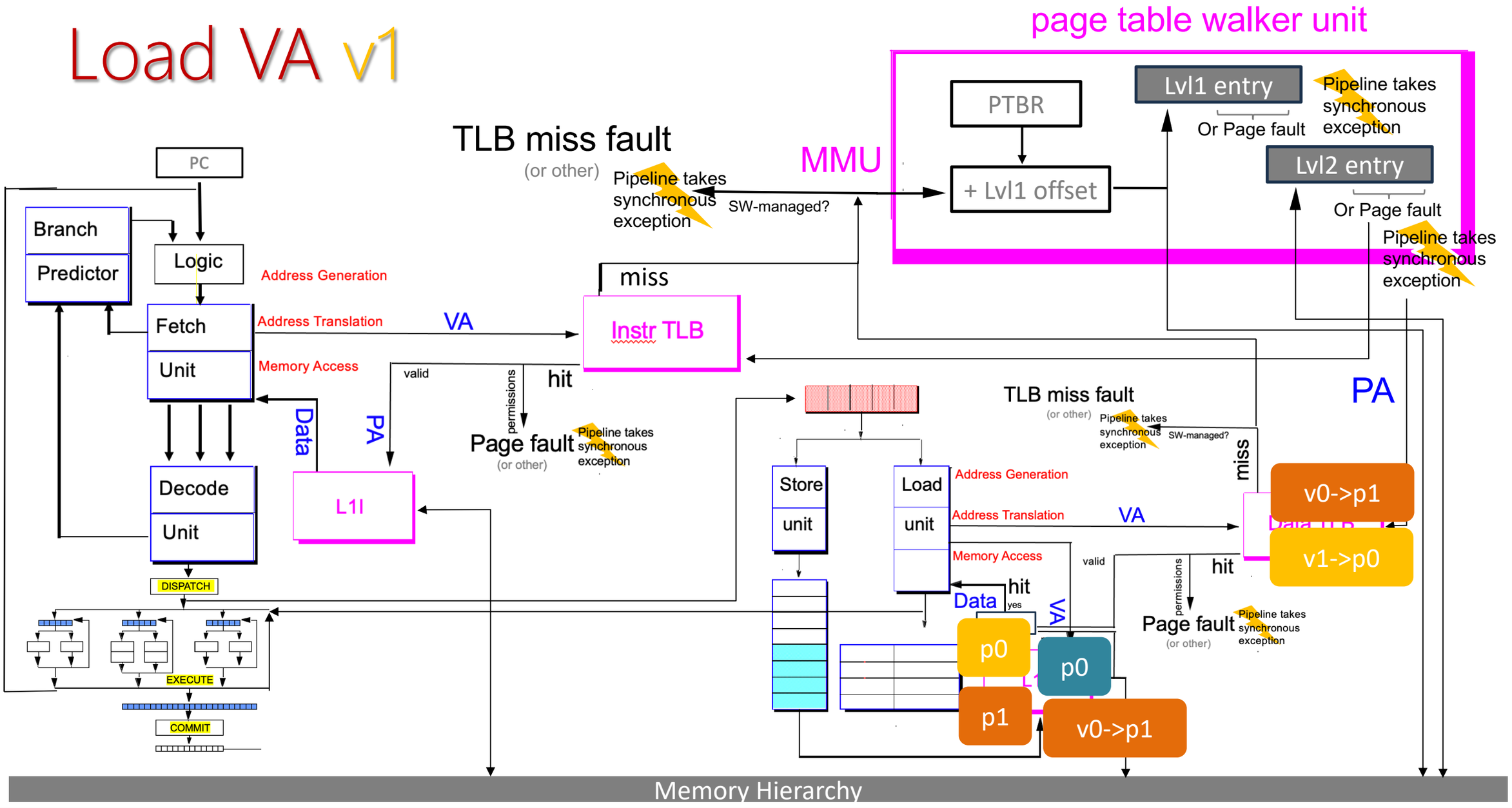
Depends on cache organization…

But what if we map a new v1 -> p0?

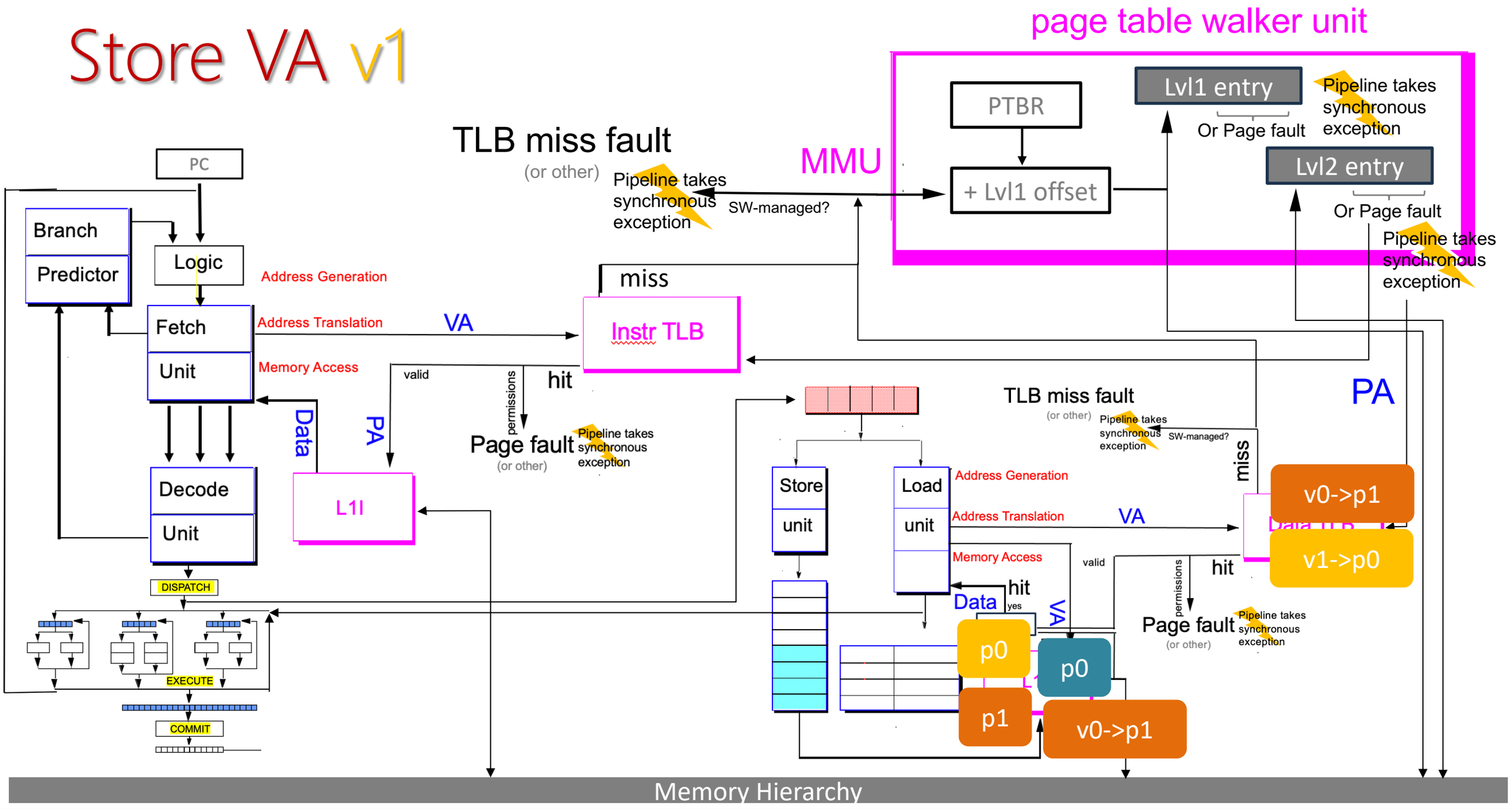
And access it…



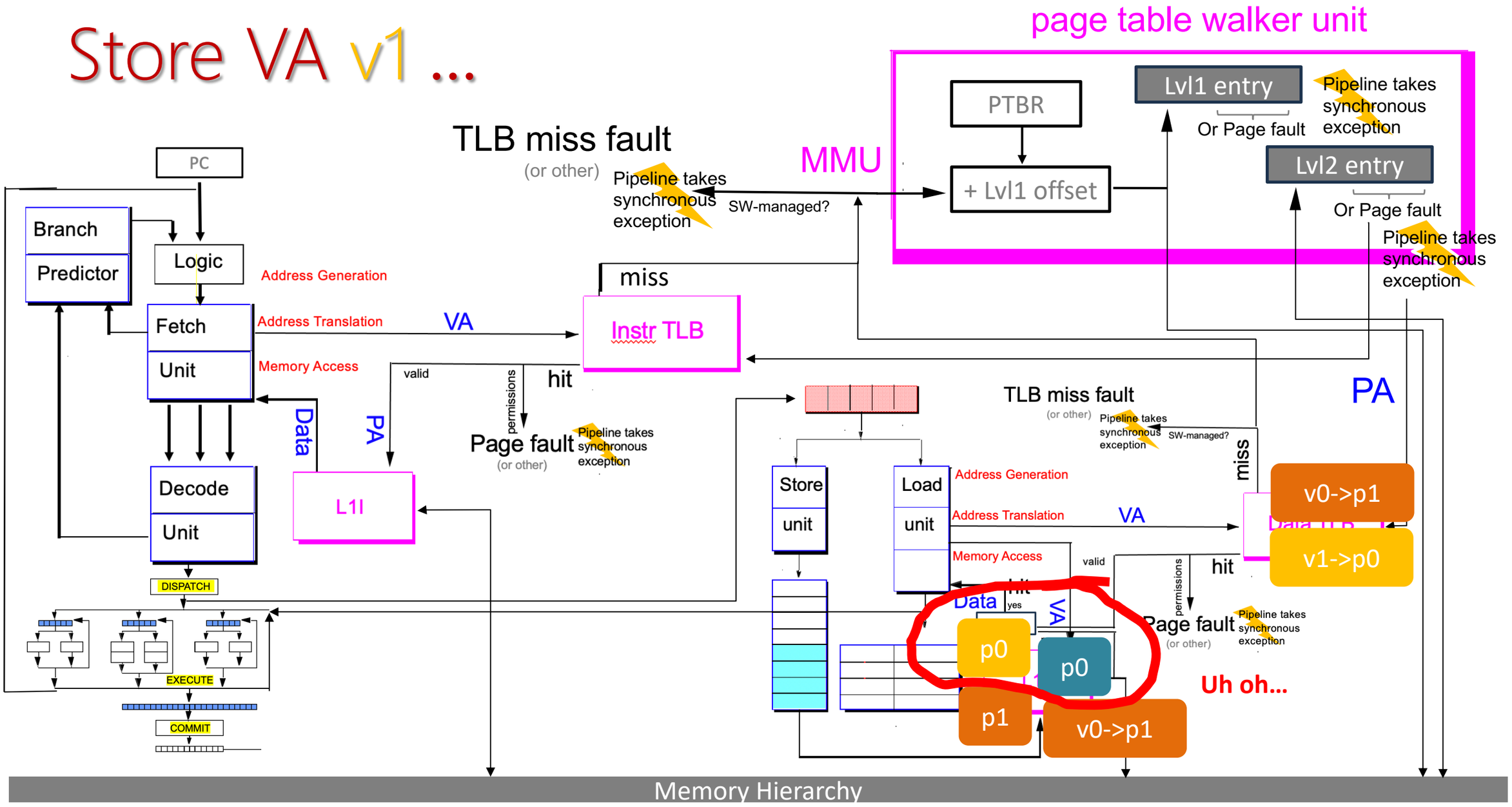
Load VA v1


page table walker unit
PTBR
Lvl1 entry
Pipeline takes synchronous exception
Or Page fault
Lvl2 entry
Or Page fault
Pipeline takes synchronous exception
+ Lvl1 offset
MMU
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
PC
Branch
Predictor
Logic
Address Generation
Fetch
Unit
Address Translation
VA
Memory Access
miss
Instr TLB
valid
permissions
hit
Page fault
(or other)
Pipeline takes synchronous exception
Data
PA
L1I
Decode
Unit
DISPATCH
EXECUTE
COMMIT
Store
unit
Load
unit
Address Generation
Address Translation
VA
Memory Access
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
miss
PA
v0->p1
v1->p0
valid
permissions
hit
Data
hit
yes
VA
Page fault
(or other)
Pipeline takes synchronous exception
p0
p0
p1
v0->p1
Memory Hierarchy

Store VA v1

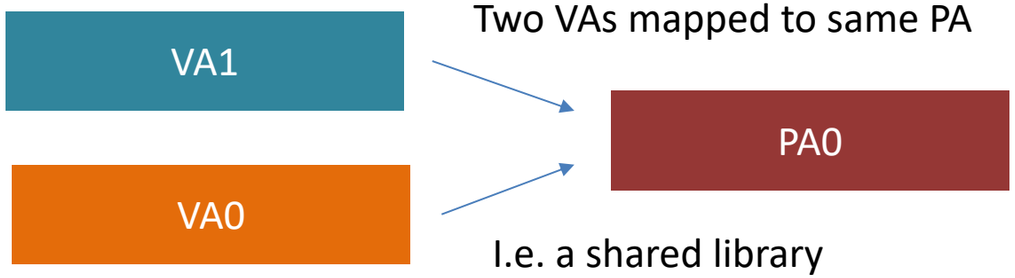

page table walker unit
PTBR
Lvl1 entry
Pipeline takes synchronous exception
Or Page fault
Lvl2 entry
Or Page fault
Pipeline takes synchronous exception
MMU
+ Lvl1 offset
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
PC
Branch
Predictor
Logic
Address Generation
Fetch
Unit
Address Translation
VA
Memory Access
miss
Instr TLB
valid
permissions
hit
Page fault
(or other)
Pipeline takes synchronous exception
Data
PA
L1I
Decode
Unit
DISPATCH
EXECUTE
COMMIT
Store
unit
Load
unit
Address Generation
Address Translation
VA
Memory Access
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
miss
PA
v0->p1
v1->p0
valid
permissions
hit
hit
yes
Data
VA
Page fault
(or other)
Pipeline takes synchronous exception
p0
p0
p1
v0->p1
Memory Hierarchy

Store VA v1 …


page table walker unit
PTBR
Lvl1 entry
Pipeline takes synchronous exception
Or Page fault
+ Lvl1 offset
Lvl2 entry
Or Page fault
Pipeline takes synchronous exception
MMU
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
PC
Branch
Predictor
Logic
Address Generation
Fetch
Unit
Address Translation
VA
miss
Instr TLB
Memory Access
valid
permissions
hit
Page fault
(or other)
Pipeline takes synchronous exception
Data
PA
L1I
Decode
Unit
DISPATCH
EXECUTE
COMMIT
Store
unit
Load
unit
TLB miss fault
(or other)
Pipeline takes synchronous exception
SW-managed?
miss
PA
Address Generation
Address Translation
VA
Memory Access
valid
permissions
hit
Data
yes
VA
Page fault
(or other)
Pipeline takes synchronous exception
v0->p1
v1->p0
p0
p0
p1
v0->p1
Uh oh…
Memory Hierarchy

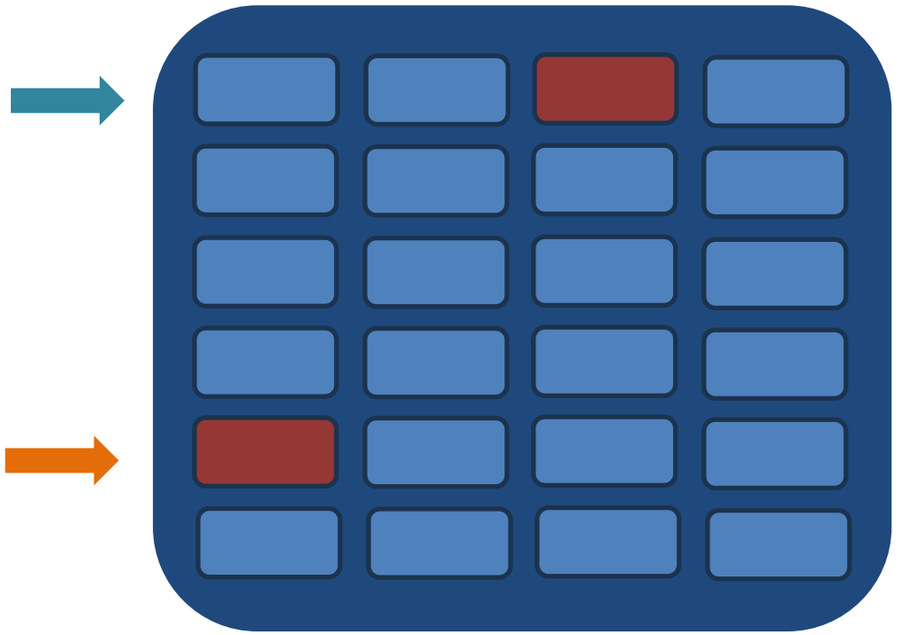
Recall - the problem with virtual indexing

- Can end up with two copies of the same data
- If any of those copies is written to they will get out of sync
- A coherence problem... for ***this*** time



Or that diabolical access pattern we just showed

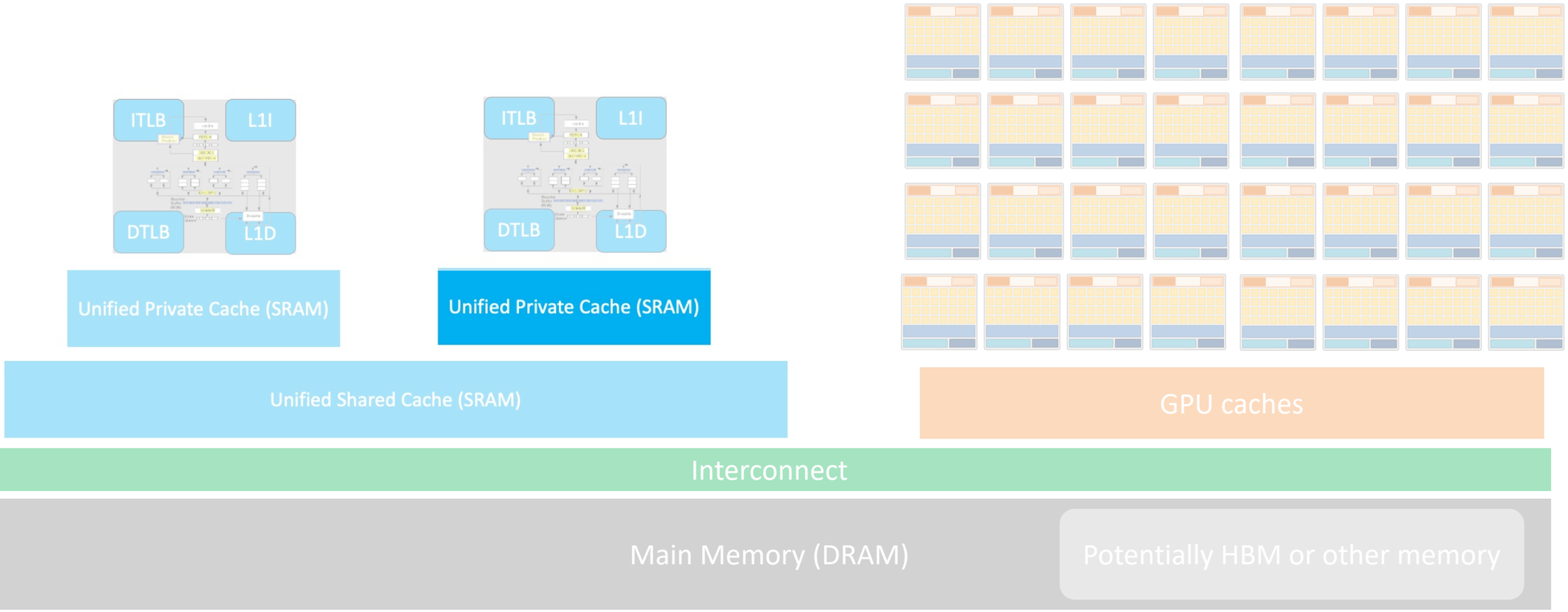
Traditional lookup only does tag match within cache set



Many proposed solutions

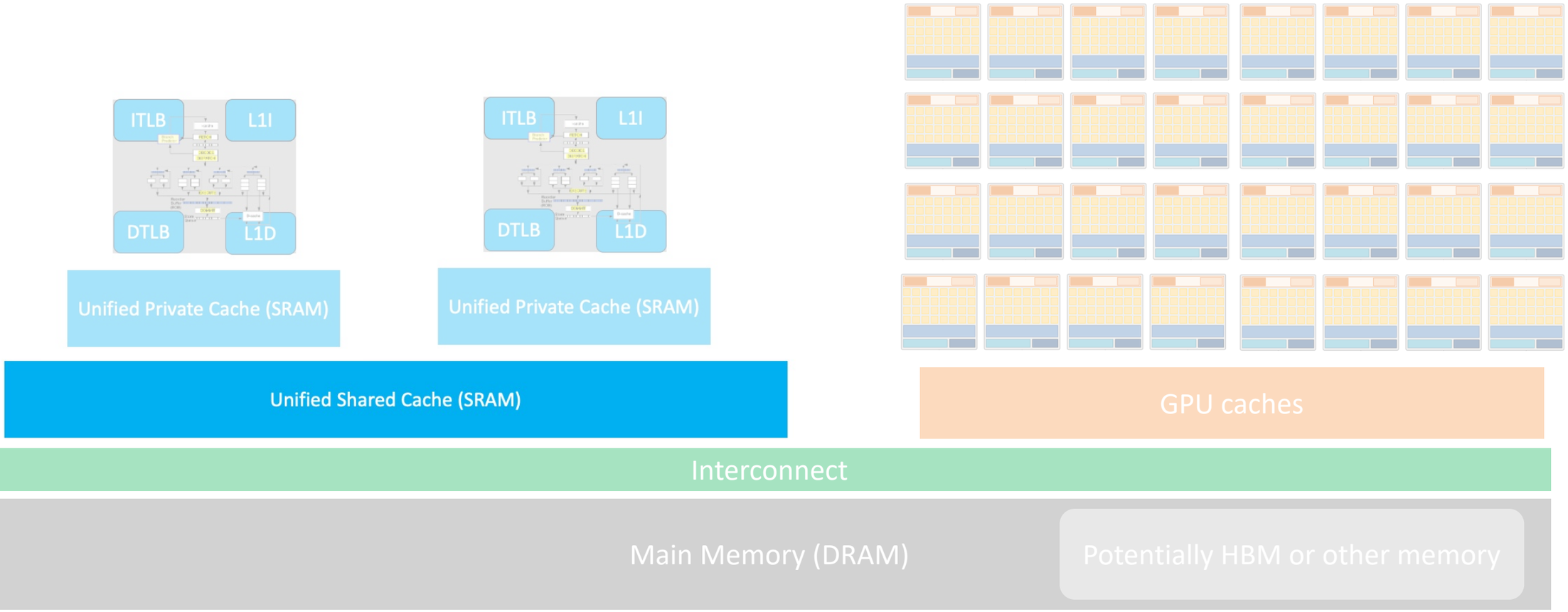
- Specialized hardware in the core to detect
- Software guarantees against the situation occurring
  - Force software to promise that p0 will only ever get mapped to VAs that index into set X
- But also resembles the more generic **coherence problem**

Memory Data Flow : (Unified) Private Cache



Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

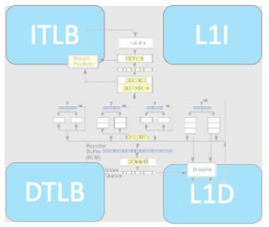
Memory Data Flow : (Unified) Shared LLC

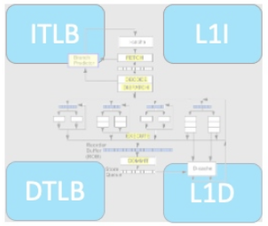


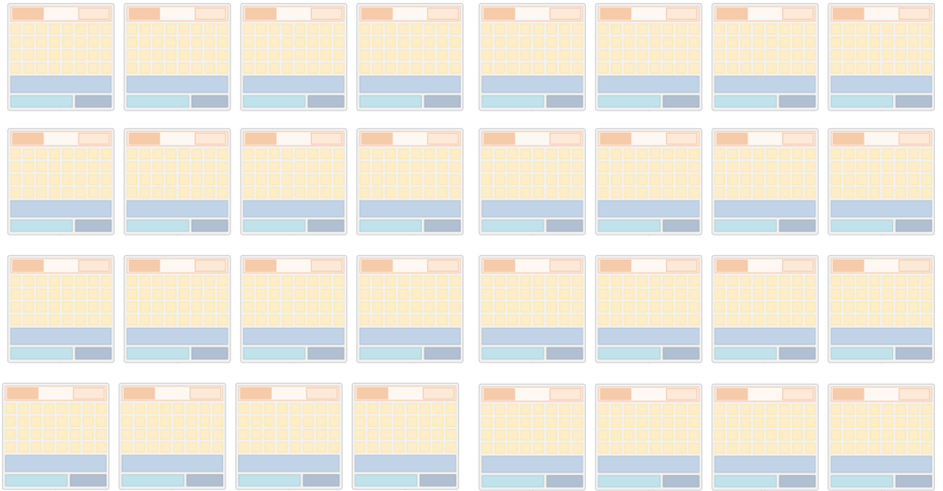
Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

Cache Coherence (example: Unified Caches)

Assuming all PA-based so we don't have to think about address translation







Unified Private Cache (SRAM)

Unified Private Cache (SRAM)

Unified Shared Cache (SRAM)

GPU caches

Interconnect

Main Memory (DRAM)

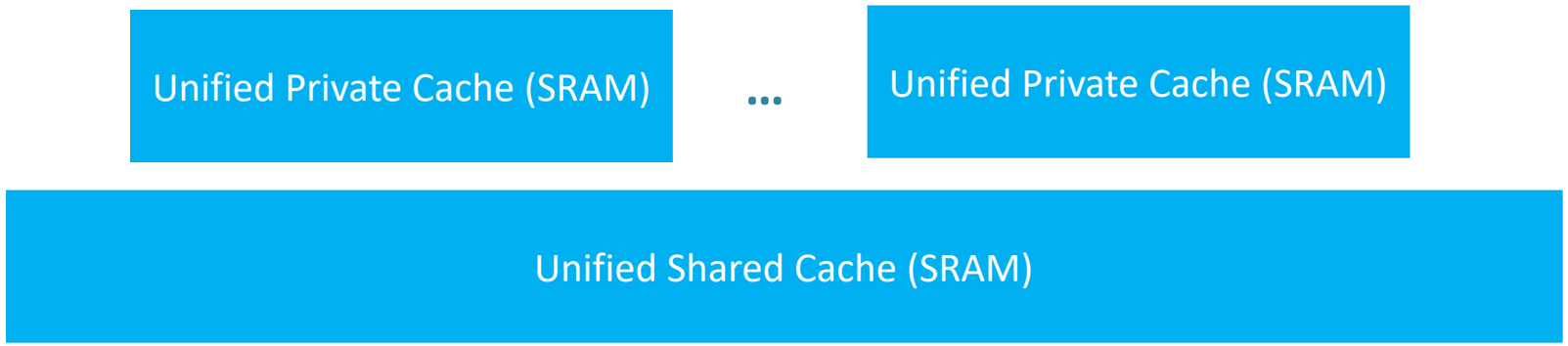
Potentially HBM or other memory

Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

Cache Coherence (example: Unified Caches)

Assuming inclusive for now – simpler than exclusive

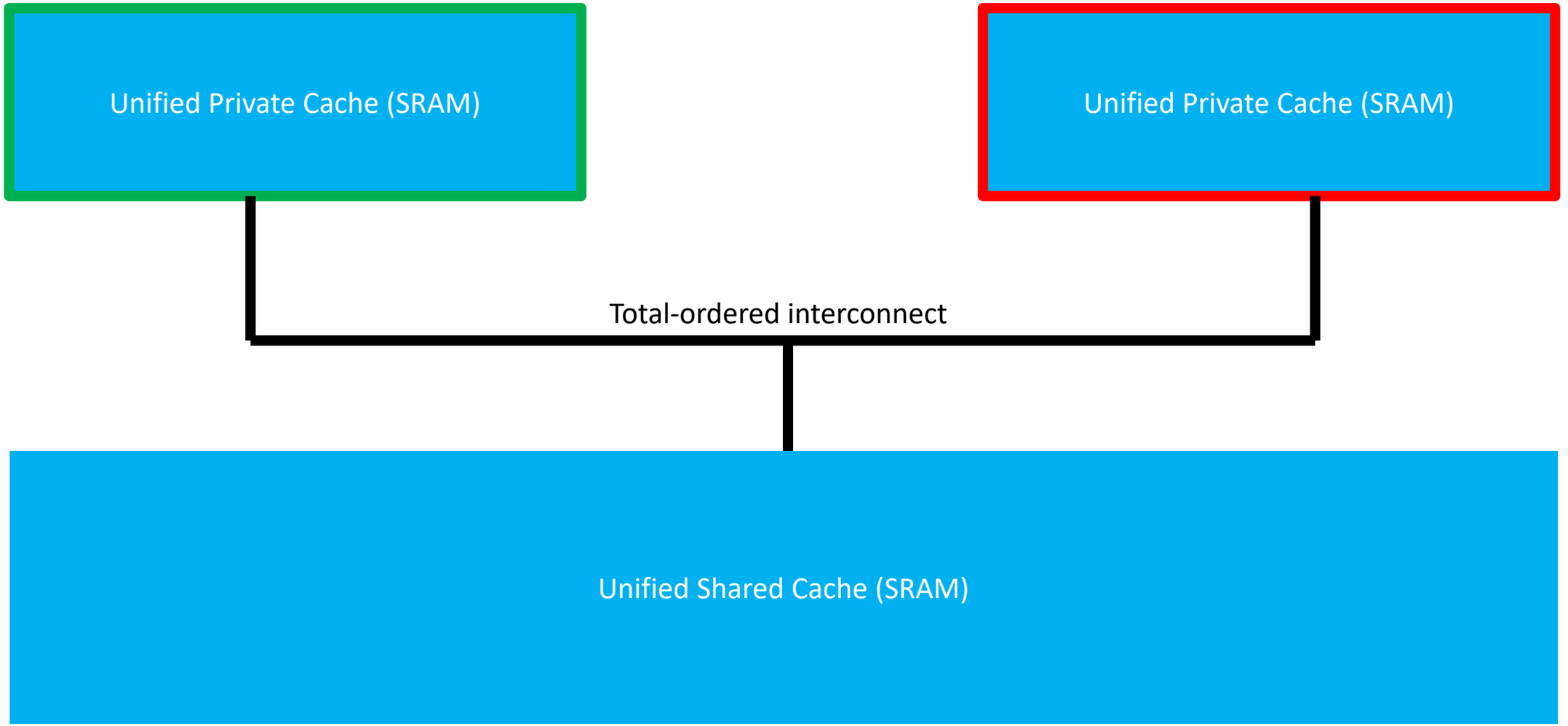
- In general, need a mechanism to keep N private caches coherent
- Can achieve "**Cache Coherence**" through a variety of implementations
  - Global, Synchronous implementations (Shared bus)
  - Local, Asynchronous implementations (Directory-based)



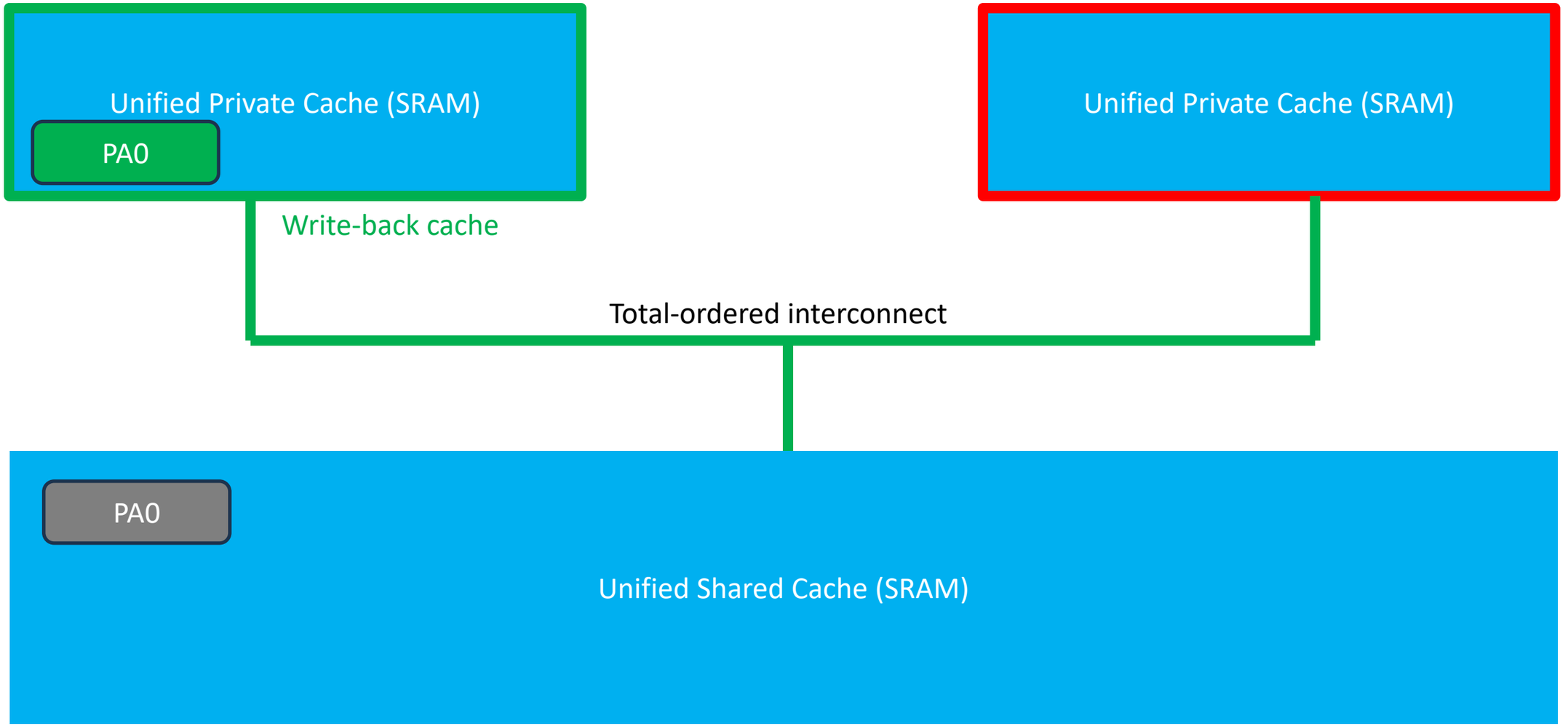
- **A lot of "First Core X requests A, Then Core Y requests B" statements to come**
  - Implicitly assuming a global time-ordering of events… which may not be the case (memory consistency)

Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

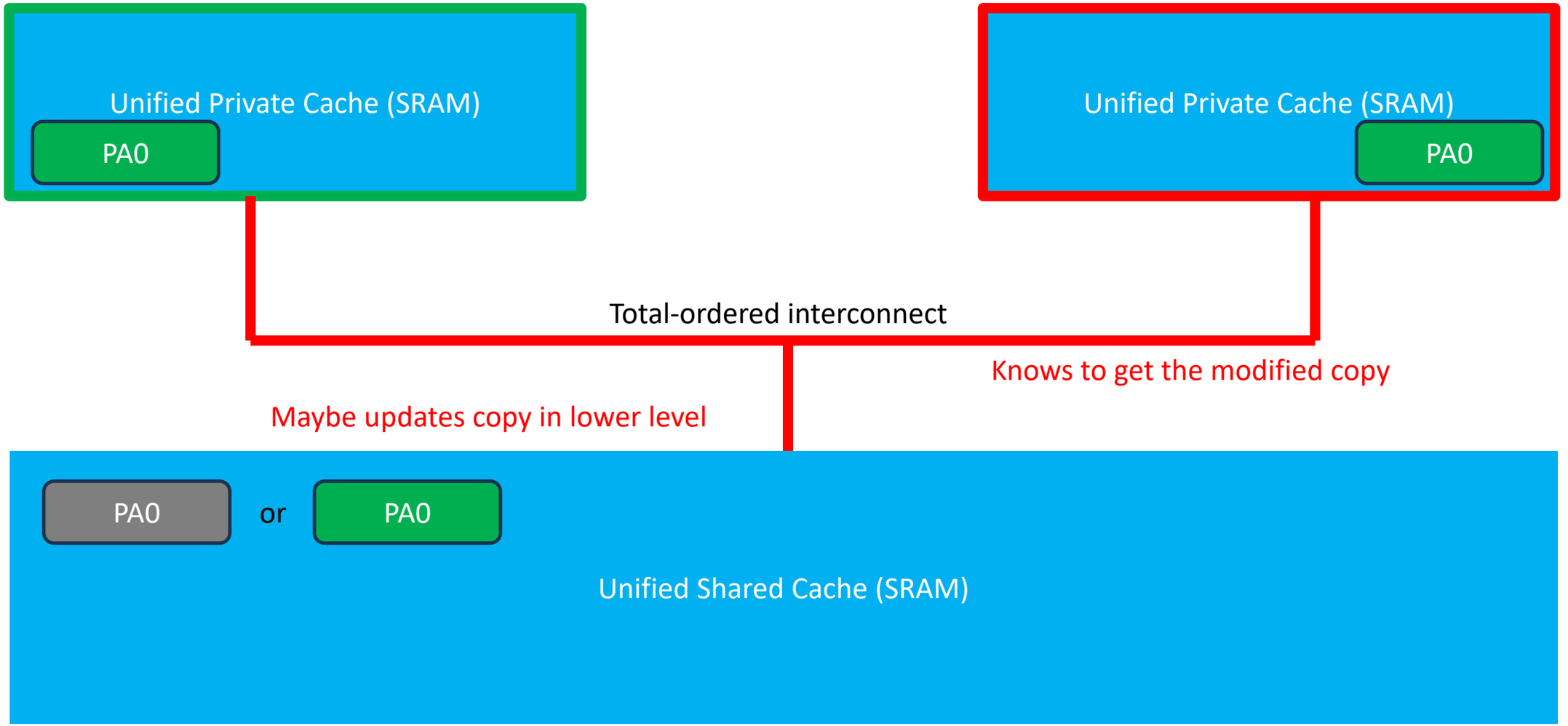
"Snoopy" Coherence (e.g. shared bus model)



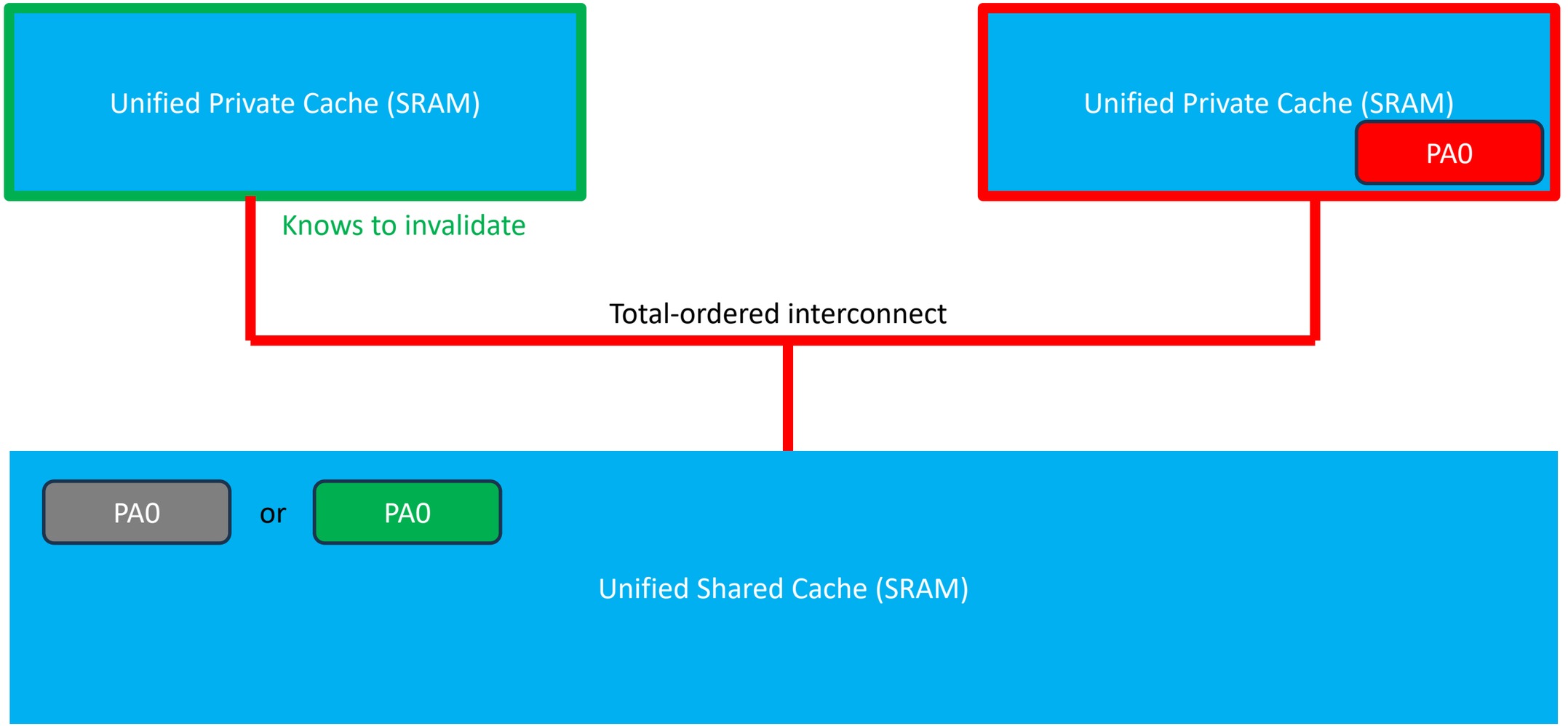
Something like - Read and modify PA0



Something like - Read PA0



Something like - Write PA0



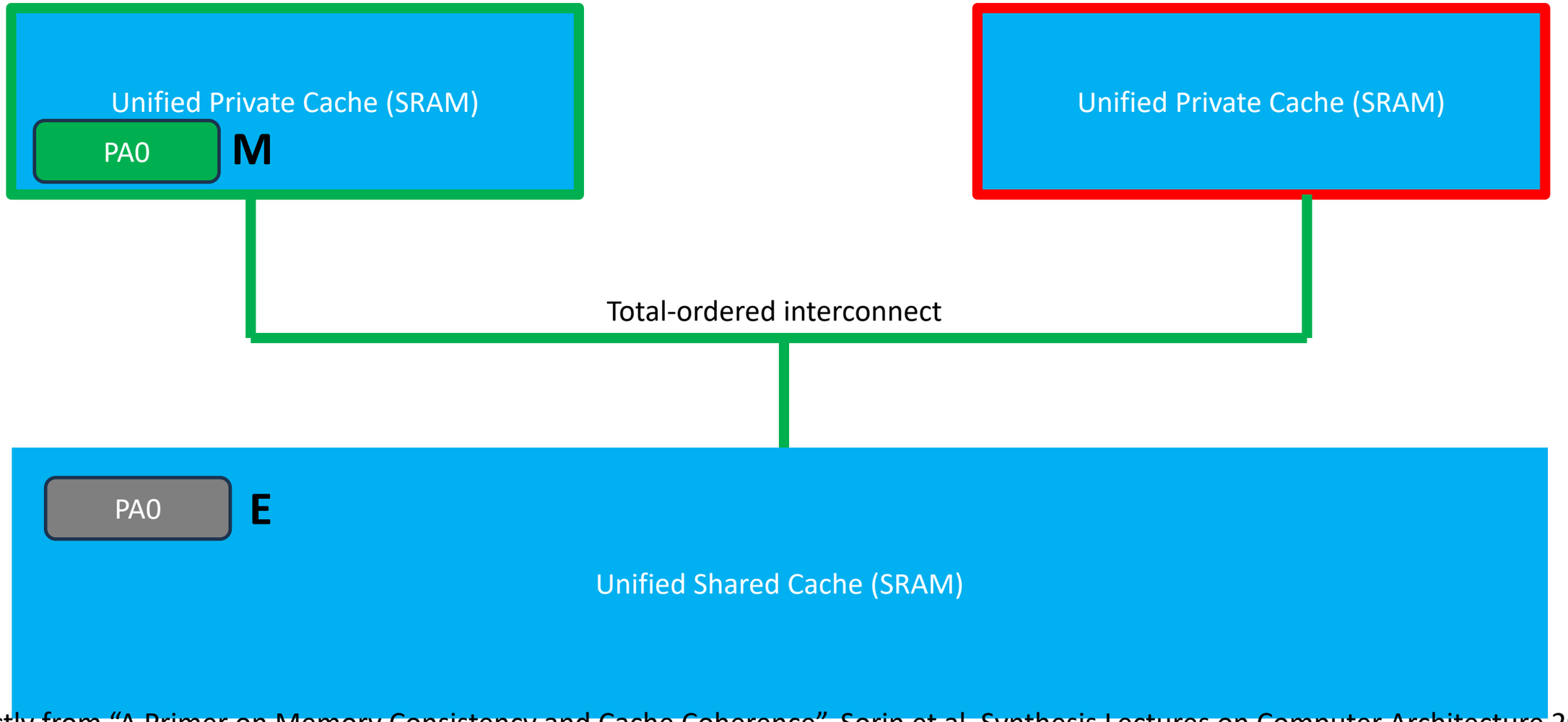
Finite, enumerable state transitions

- Typically the algorithm used to implement is described via the states supported
- ***MSI***
  - Modified
    - data is dirty and need to be written back on eviction
    - If another core requests this line, you must respond as the copy in lower level is stale
  - Shared
    - data is clean and copy in lower level is correct
    - If you try to write to it, must request line in M
  - Invalid
    - data is not present in the cache
    - If you try to write or read to it, must request in M or S

Finite, enumerable state transitions

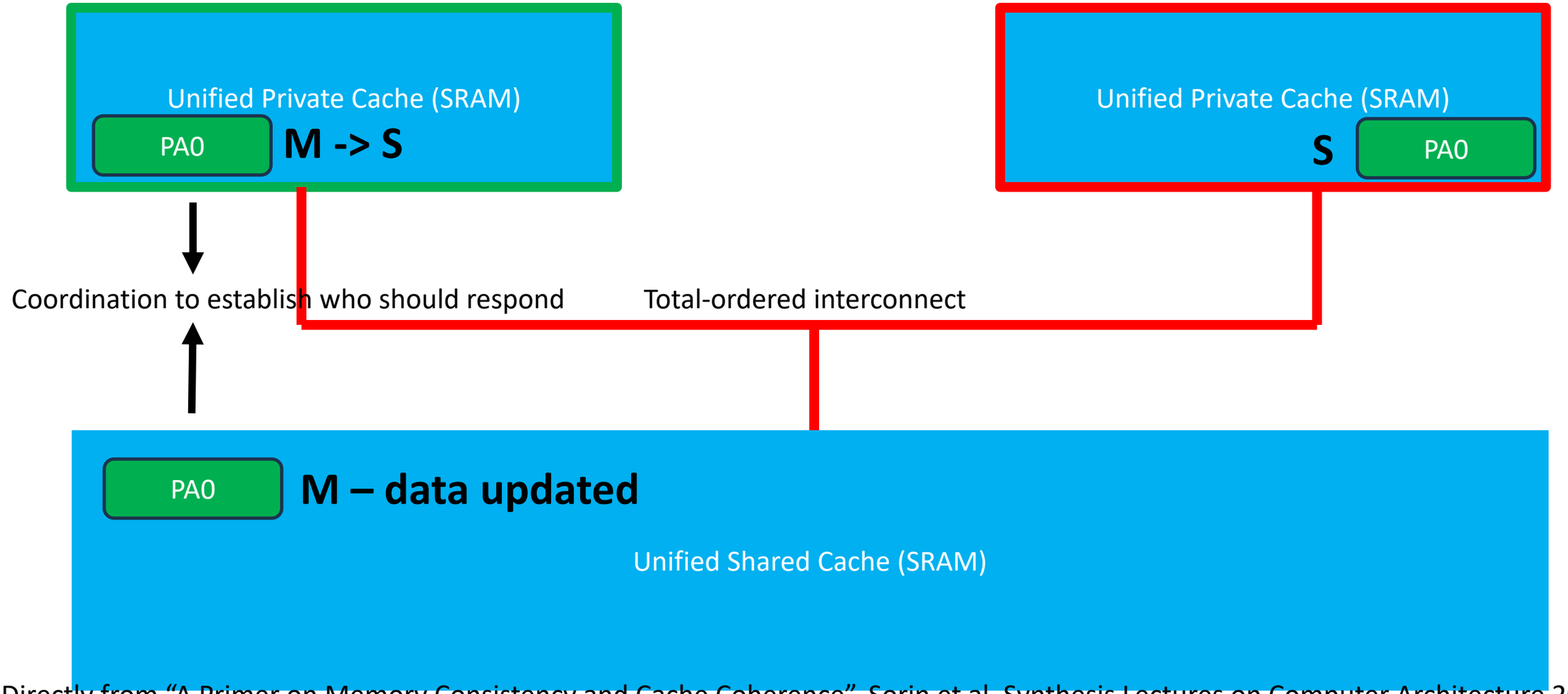
- Can expand to add more states
- ***MESI***
  - Exclusive
    - Data is clean but core still has write-permissions
    - I.e. does not need to broadcast-on-write like it would have to on an S->M transition
    - When evicted, and data is clean, data need not be transferred
- ***MO(E)SI***
  - Owned
    - Data is clean and core does not have write permissions
    - Core is responsible for responding with the data instead of the lower level
- Going quickly because assuming this is familiar from undergrad
- Total-ordered interconnect used in broadcast mode to manage transitions

MOESI – Read/Write PA0



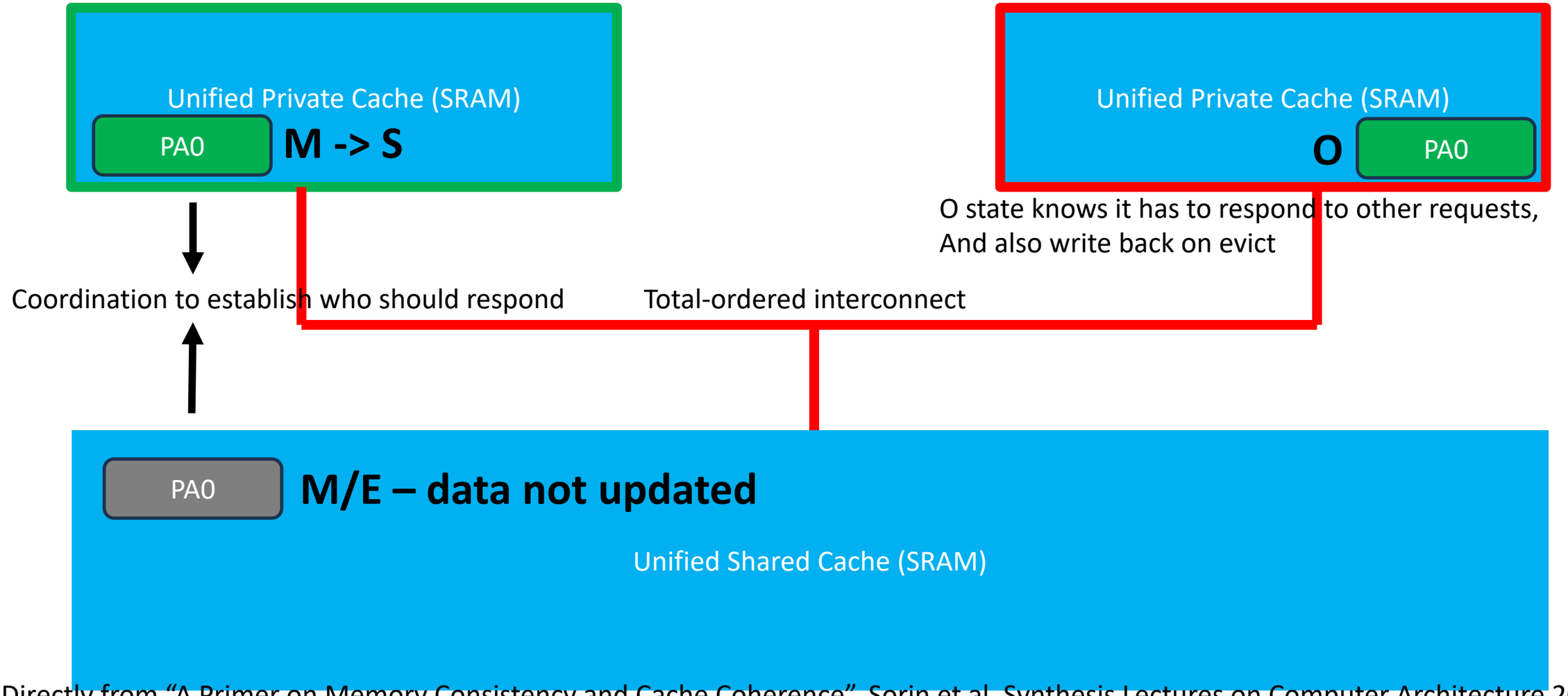
Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

MOESI - Read PA0 (option 1)



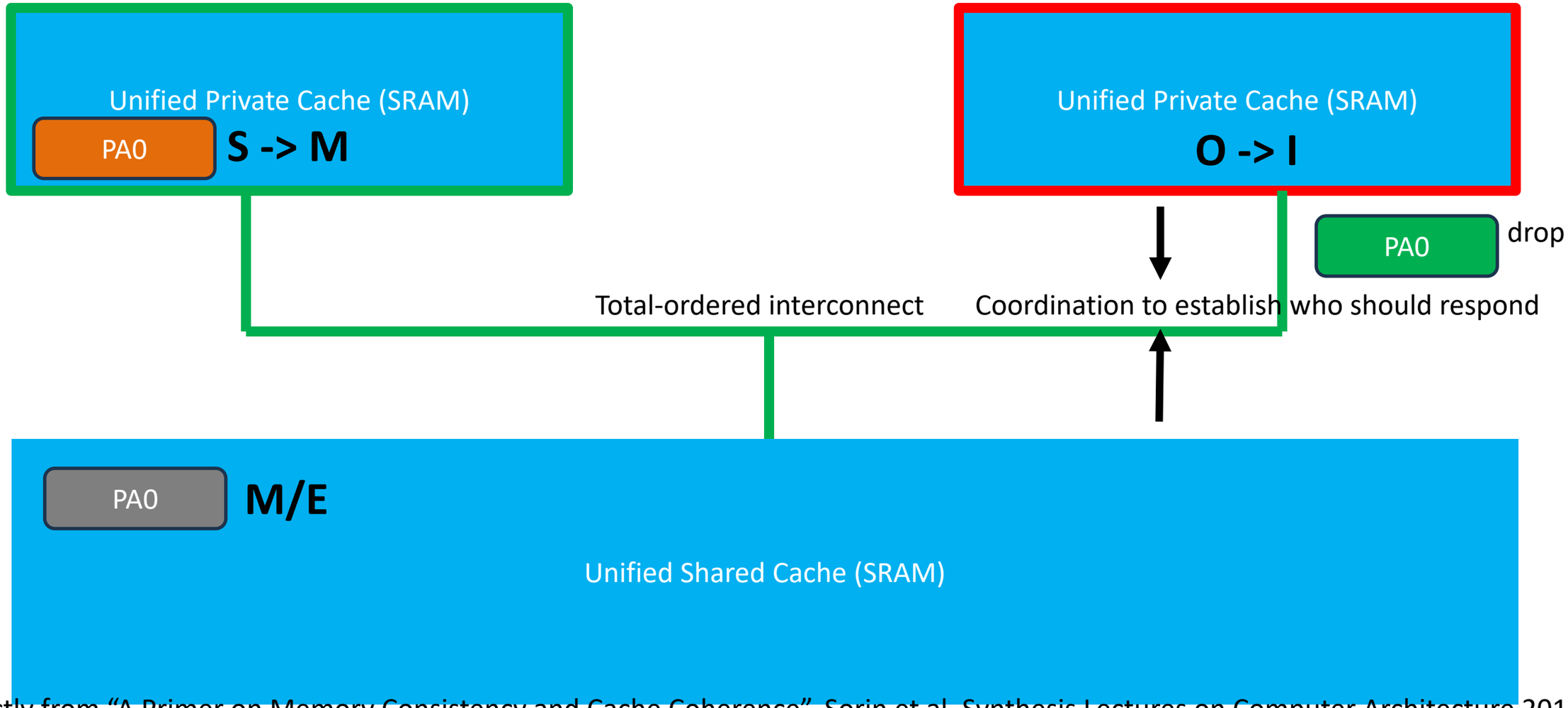
Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

MOESI - Read PA0 (option 2)



Directly from "A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

MOESI – Write PA0

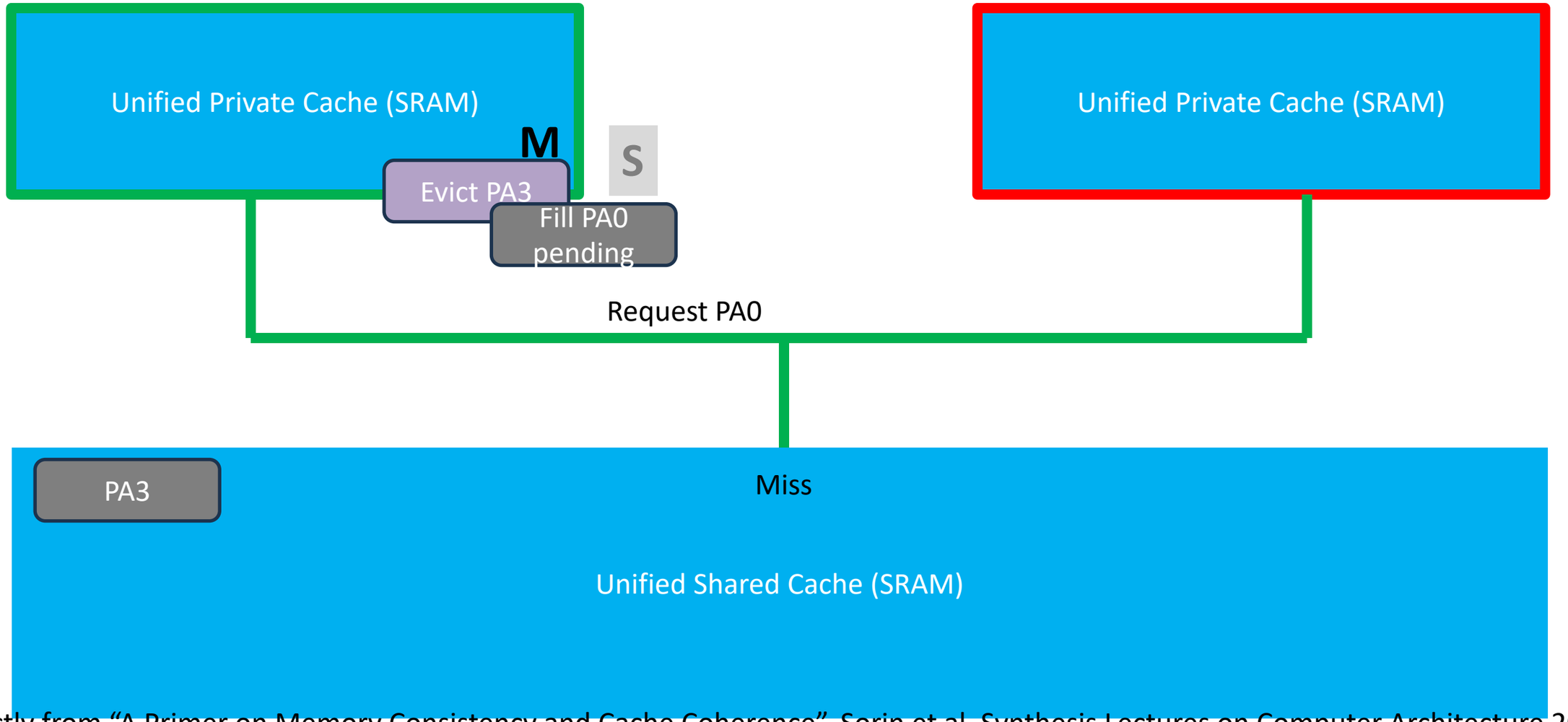


Directly from "A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

But this is a graduate class

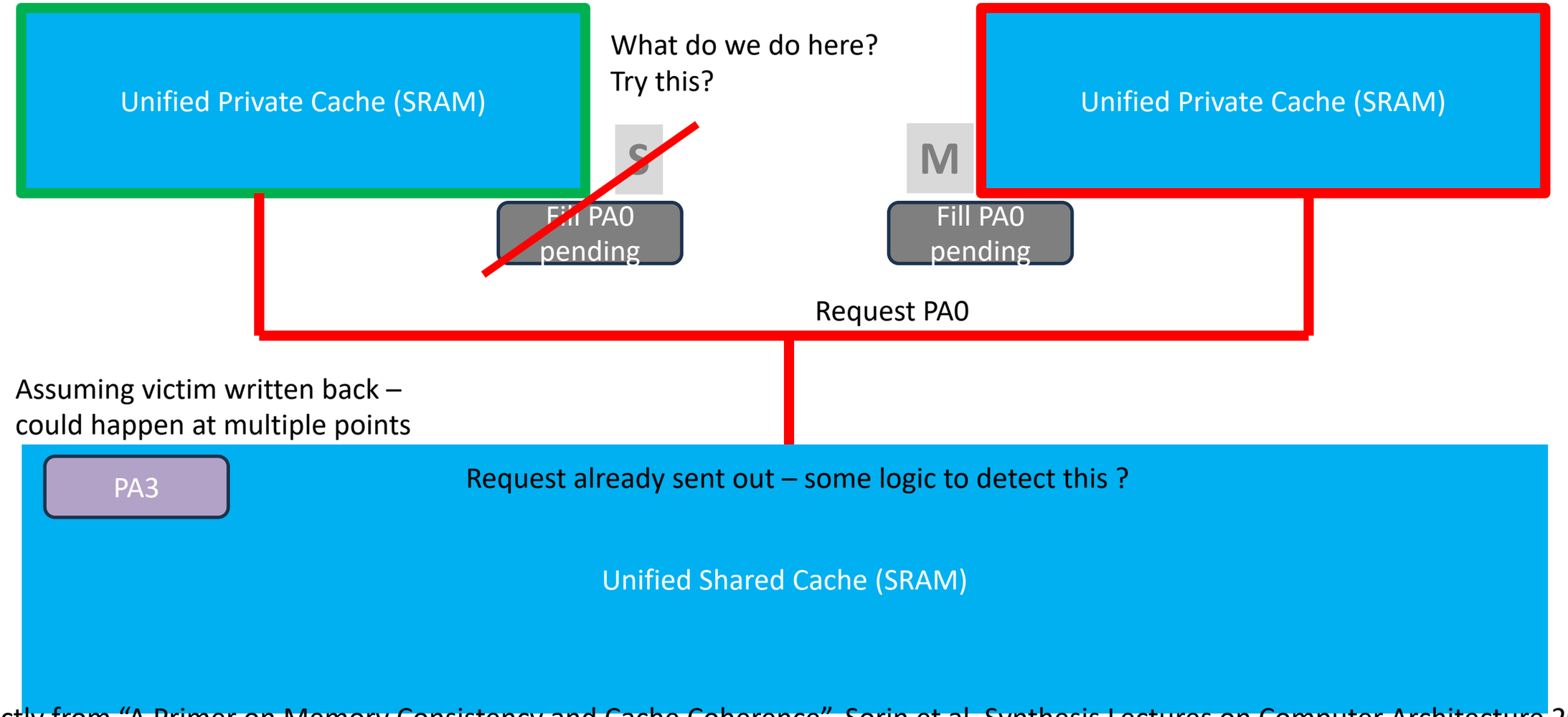
- The state machine is the boring part
- Atomic state transitions are good in textbooks
- 740 is dealing with ***non-blocking***, ***OOO*** caches with ***MSHRs***
- ***Request and Response are split and can be interleaved in nasty ways***

Example - Req PA0 for Reading



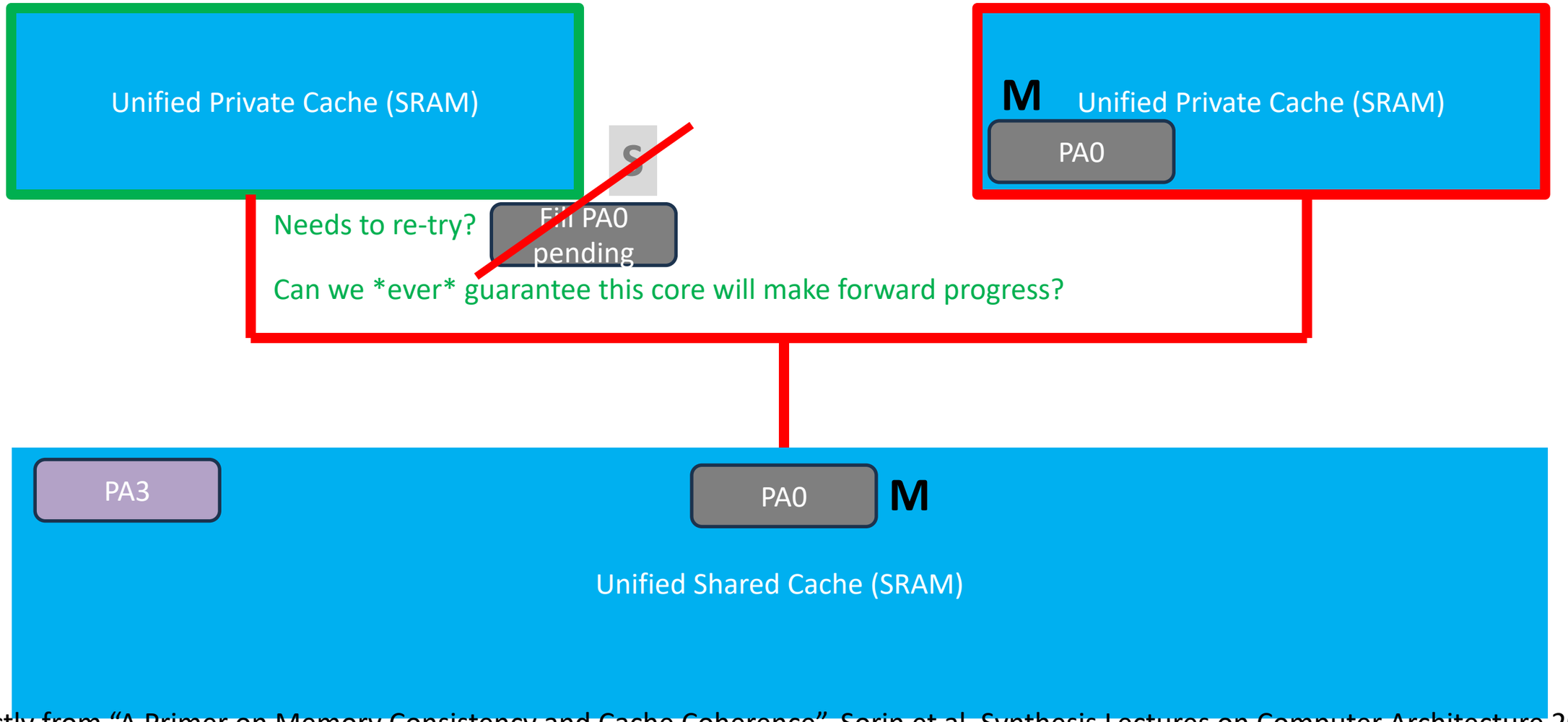
Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example - Req PA0 for Writing



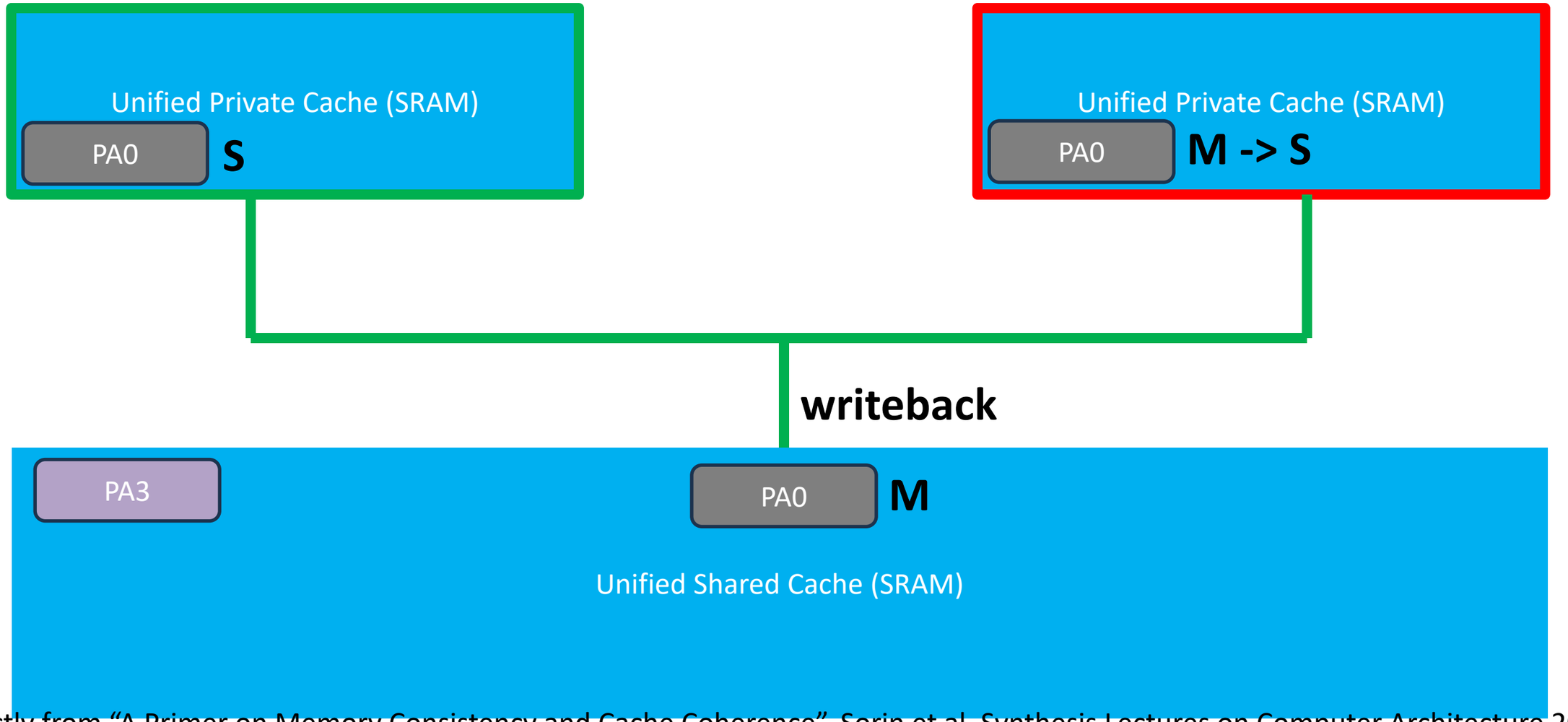
Directly from "A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example - Resp PA0 for Writing ?



Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example - Re-Req PA0 for Reading ?



Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

This gets *really hard*

- ***CANNOT ignore intermediary states***
- ***MUST guarantee every core makes forw***
- And try to minimize hops/latency

TABLE 7.20: Optimized MSI Snooping with Split-Transaction Bus—Cache Controller

	load	store	replacement	OwnGetS	OwnGetM	OwnPutM	OtherGetS	OtherGetM	OtherPutM	Own Data response
I	issue GetS/IS^{AD}	issue GetM/IM^{AD}					-	-	-	
IS^{AD}	stall	stall	stall	-/IS^{D}			-	-	-	**-/IS^{A}**
IS^{D}	stall	stall	stall				-	**-/IS^{D}I**		load hit/S
IS^{A}	**stall**	**stall**	**stall**	**load hit/S**			-	-		
IS^{D}I	stall	stall	stall				-	-		load hit/I
IM^{AD}	stall	stall	stall		-/IM^{D}		-	-	-	**-/IM^{A}**
IM^{D}	stall	stall	stall				**-/IM^{D}S**	**-/IM^{D}I**		store hit//M
IM^{A}	**stall**	**stall**	**stall**		**store hit/M**		-	-	-	
IM^{D}I	**stall**	**stall**	**stall**				-	-		**store hit, send data to GetM requestor/I**
IM^{D}S	**stall**	**stall**	**stall**				-	**-/IM^{D}SI**		**store hit, send data to GetS requestor and mem/S**
IM^{D}SI	**stall**	**stall**	**stall**					-		**store hit, send data to GetS requestor and mem/I**
S	hit	issue GetM/SM^{AD}	-/I				-	-/I		
SM^{AD}	hit	stall	stall		-/SM^{D}		-	-/IM^{AD}		**-/SM^{A}**
SM^{D}	hit	stall	stall				**-/SM^{D}S**	**-/SM^{D}I**		store hit/M
SM^{A}	**hit**	**stall**	**stall**		**store hit/M**		-	**-/IM^{A}**		
SM^{D}I	**hit**	**stall**	**stall**				-	-		**store hit, send data to GetM requestor/I**
SM^{D}S	**hit**	**stall**	**stall**				-	**-/SM^{D}SI**		**store hit, send data to GetS requestor and mem/S**
SM^{D}SI	**hit**	**stall**	**stall**				-	-		**store hit, send data to GetS requestor and mem/I**
M	hit	hit	issue PutM/MI^{A}				send data to requestor and to memory/S	send data to requestor/I		
MI^{A}	hit	hit	stall			send data to requestor/I	send data to requestor and to memory/II^{A}	send data to requestor/II^{A}		
II^{A}	stall	stall	stall			-/I	-	-	-	

Private cache responsibilities

Shared cache responsibilities

TABLE 7.21: Optimized MSI Snooping with Split-Transaction Bus—Memory Controller

	GetS	GetM	PutM from Owner	PutM from Non-Owner	Data
IorS	send data to requestor	send data to requestor, set owner to requestor/M		-	
M	clear Owner/IorS^{D}	set Owner to requestor	clear Owner/IorS^{D}	-	**write data to memory/IorS^{A}**
IorS^{D}	**stall**	**stall**	**stall**	-	write data to memory/IorS
IorS^{A}	**clear Owner/IorS**	-	**clear Owner/IorS**	-	

[1] "A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

May not be a single monolithic bus

- Snoopy coherence assumes a **global totally-ordered transport**
- Does not mean it has to be a single physical bus (think ethernet protocol)

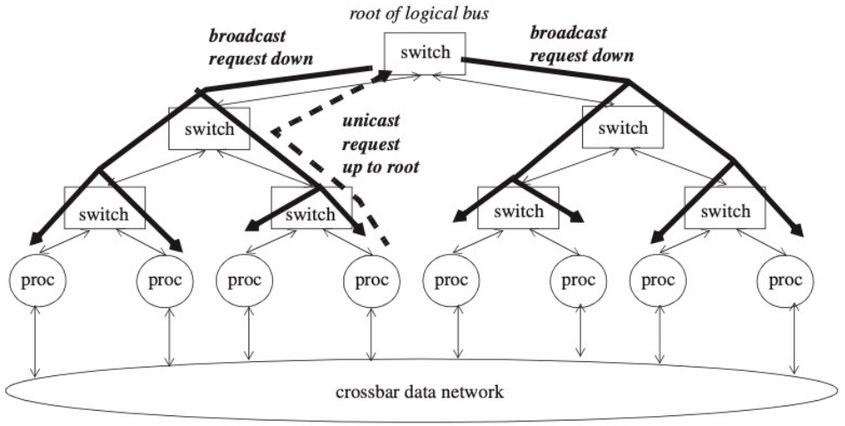


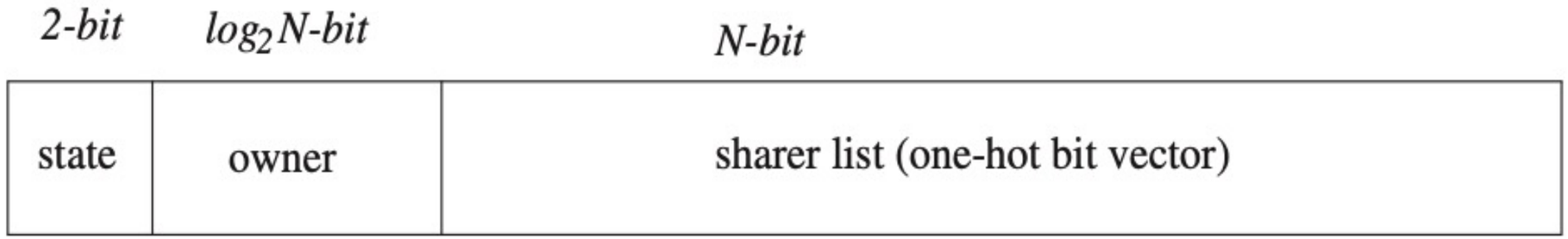
FIGURE 7.12: Starfire E10000 (drawn with only eight processors for clarity). A coherence request is unicast up to the root, where it is serialized, before being broadcast down to all processors.

- But the high-degree of arbitration and coordination may not scale well

[1] "A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

Directory Coherence

- Relax global, total-ordering constraints and move to a distributed model
  - I.e. there is no single global bus and just request packets going over a network
- The ***lower*** maintains a ***Directory*** to track the state of cache lines in ***upper***
  - Ordering happens at the directory
  - Things may arrive OOO at upper caches
  - No global resource "locking", so requests can be processed in parallel
- For every cache line in the ***lower*** cache, keep a directory entry such as



"A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

Directory Coherence

- Relax global, total-ordering constraints and move to a distributed model
  - I.e. there is no single global bus and just request packets going over a network
- The ***lower*** maintains a ***Directory*** to track the state of cache lines in ***upper***
  - Ordering happens at the directory
  - Things may arrive OOO at upper caches
  - No global resource "locking", so requests can be processed in parallel
- For every cache line in the ***lower*** cache, keep a directory entry such as

2-bit	*log_2N-bit*	*N-bit*
S	N/A	0 1 1 (cores 1 and 2)

"A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

Directory Coherence

- Relax global, total-ordering constraints and move to a distributed model
  - I.e. there is no single global bus and just request packets going over a network
- The ***lower*** maintains a ***Directory*** to track the state of cache lines in ***upper***
  - Ordering happens at the directory
  - Things may arrive OOO at upper caches
  - No global resource "locking", so requests can be processed in parallel
- For every cache line in the ***lower*** cache, keep a directory entry such as

2-bit	*log_2N-bit*	*N-bit*
M	2	0 0 1 (core 2)

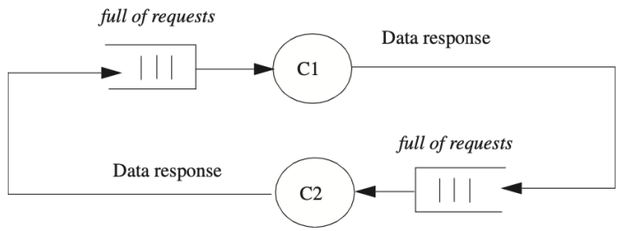
"A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

Scalable inter-cache network[1]

Unified Private Cache (SRAM)

Unified Private Cache (SRAM)

Unified Private Cache (SRAM)



Network queues are finite-width
If end up with circular dependency, can deadlock

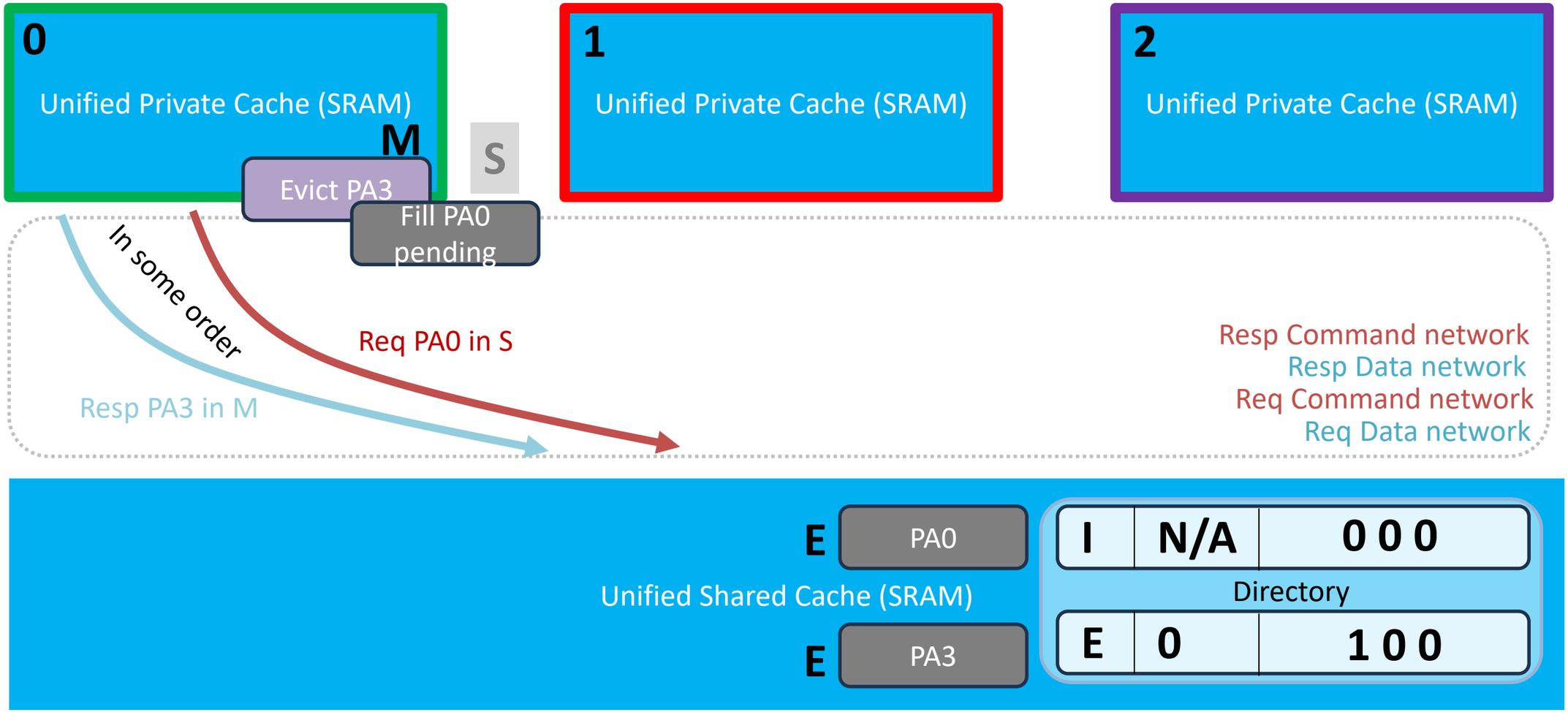
Common approach to split request/response
Onto different network/virtual networks

Resp Command network
Resp Data network
Req Command network
Req Data network

Unified Shared Cache (SRAM)

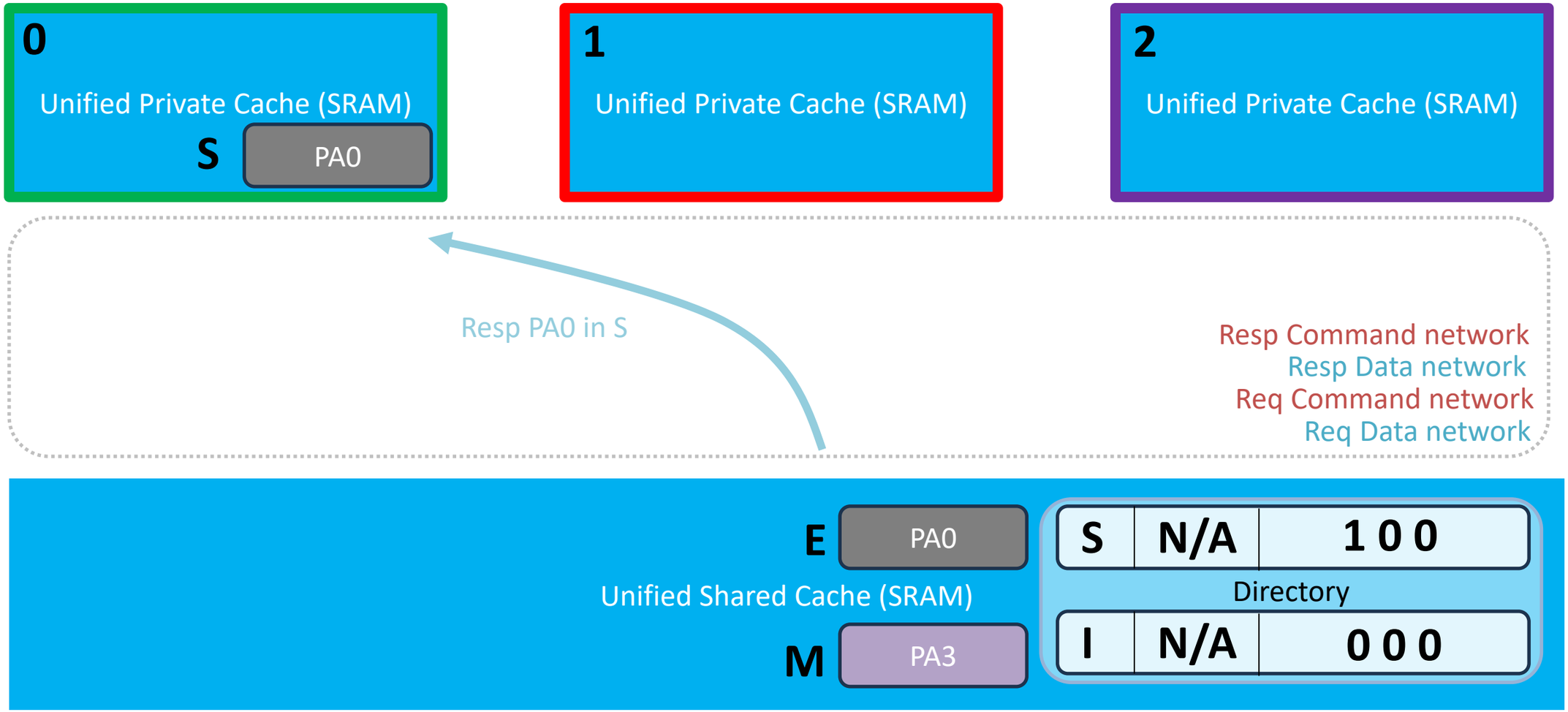
Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example 1 - Req PA0 for Reading



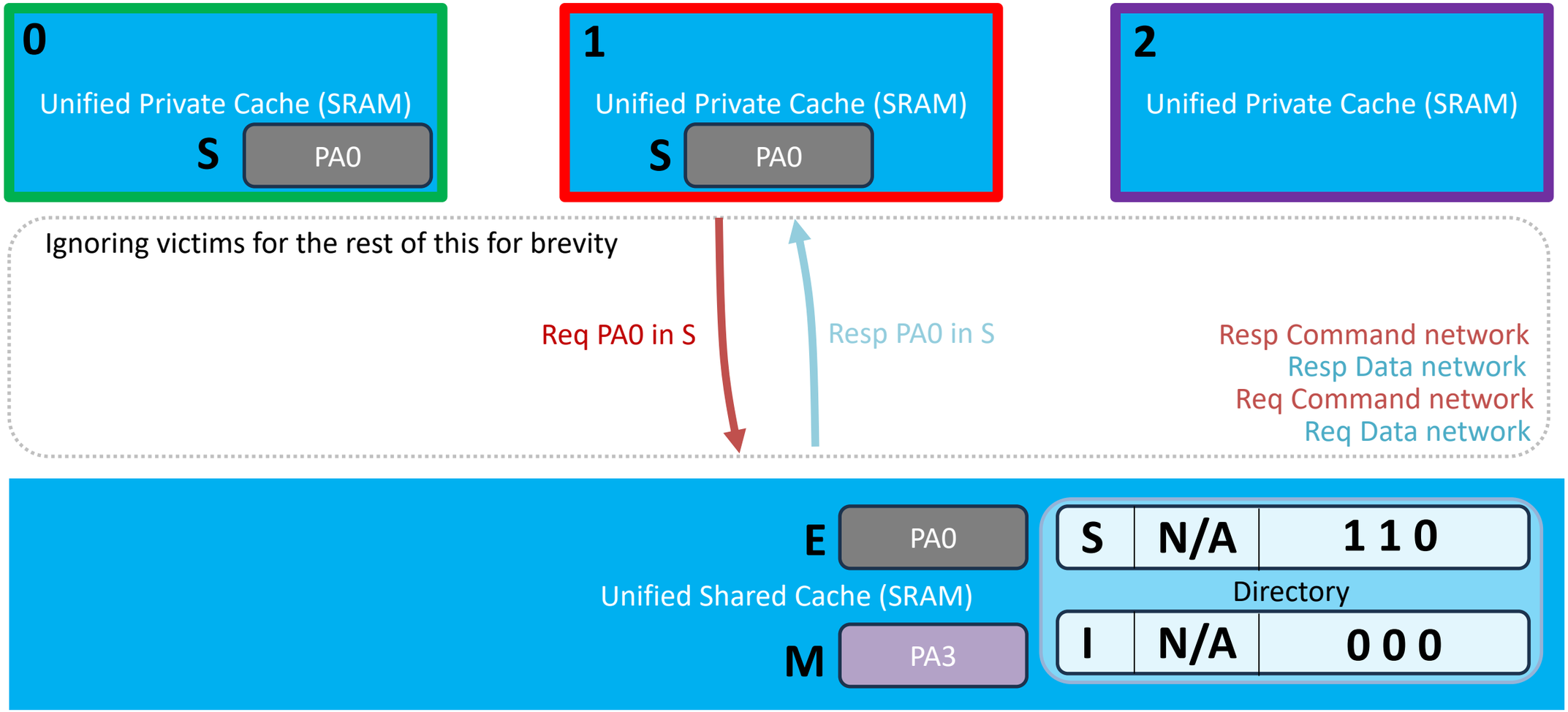
Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example 1 - Req PA0 for Reading



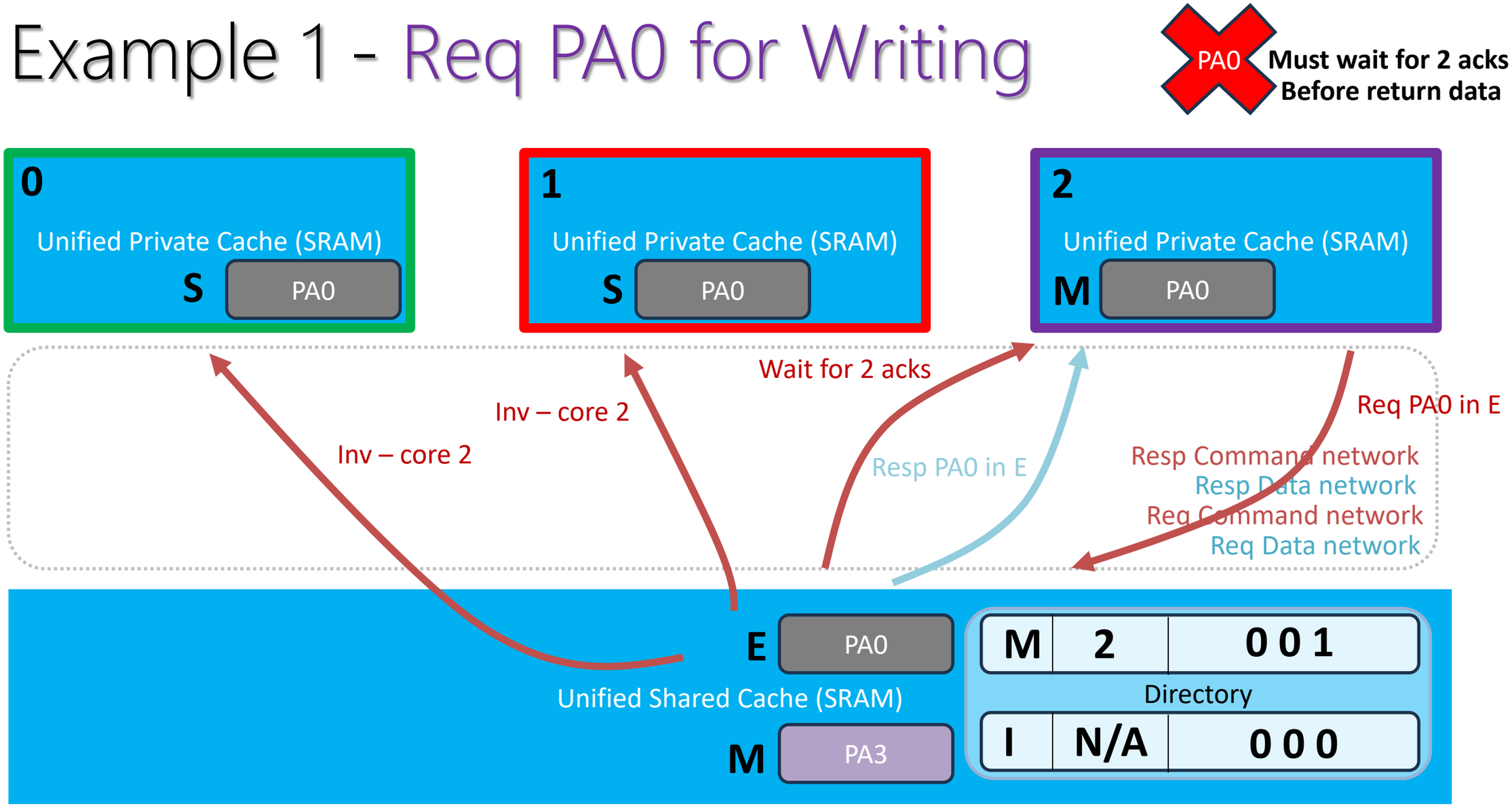
Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example 1 - Req PA0 for Reading



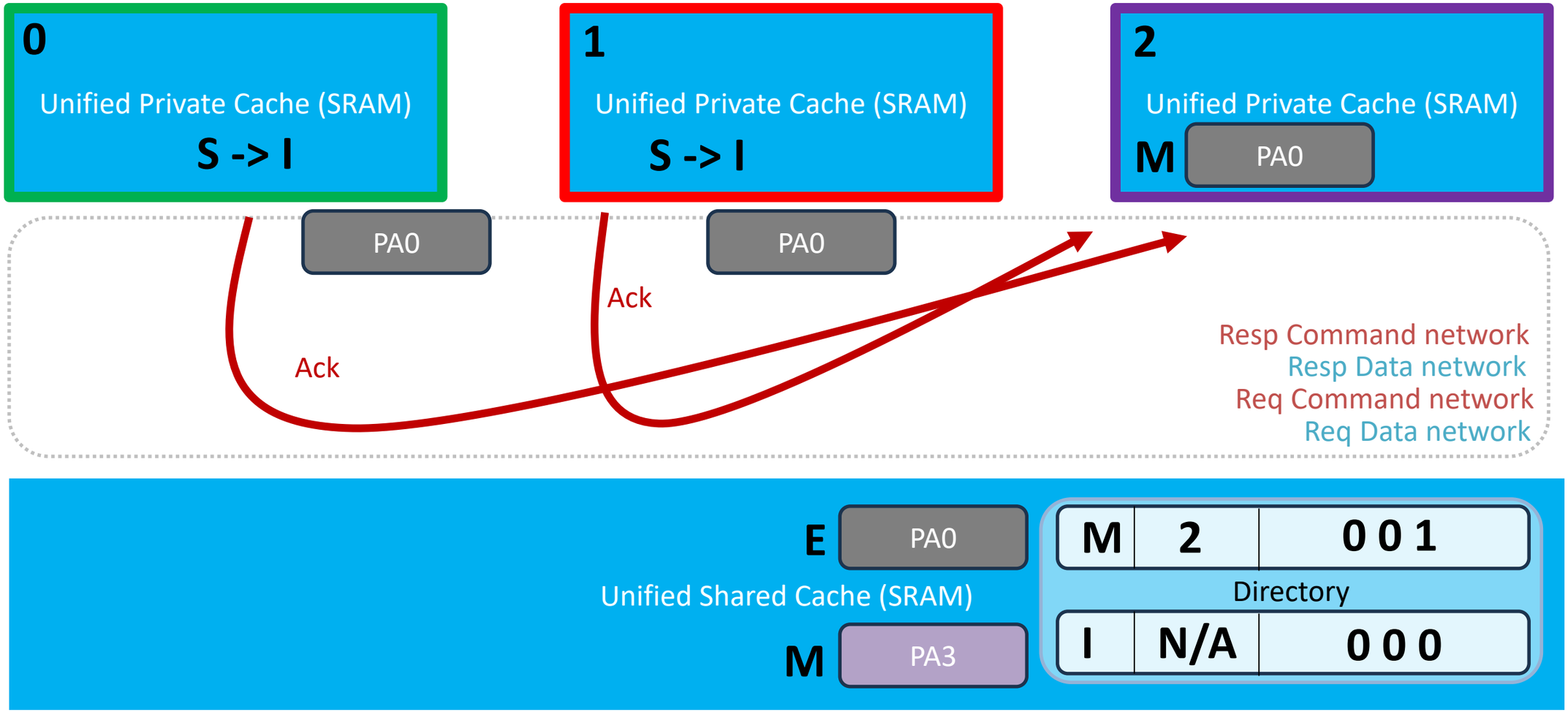
Directly from "A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example 1 - Req PA0 for Writing



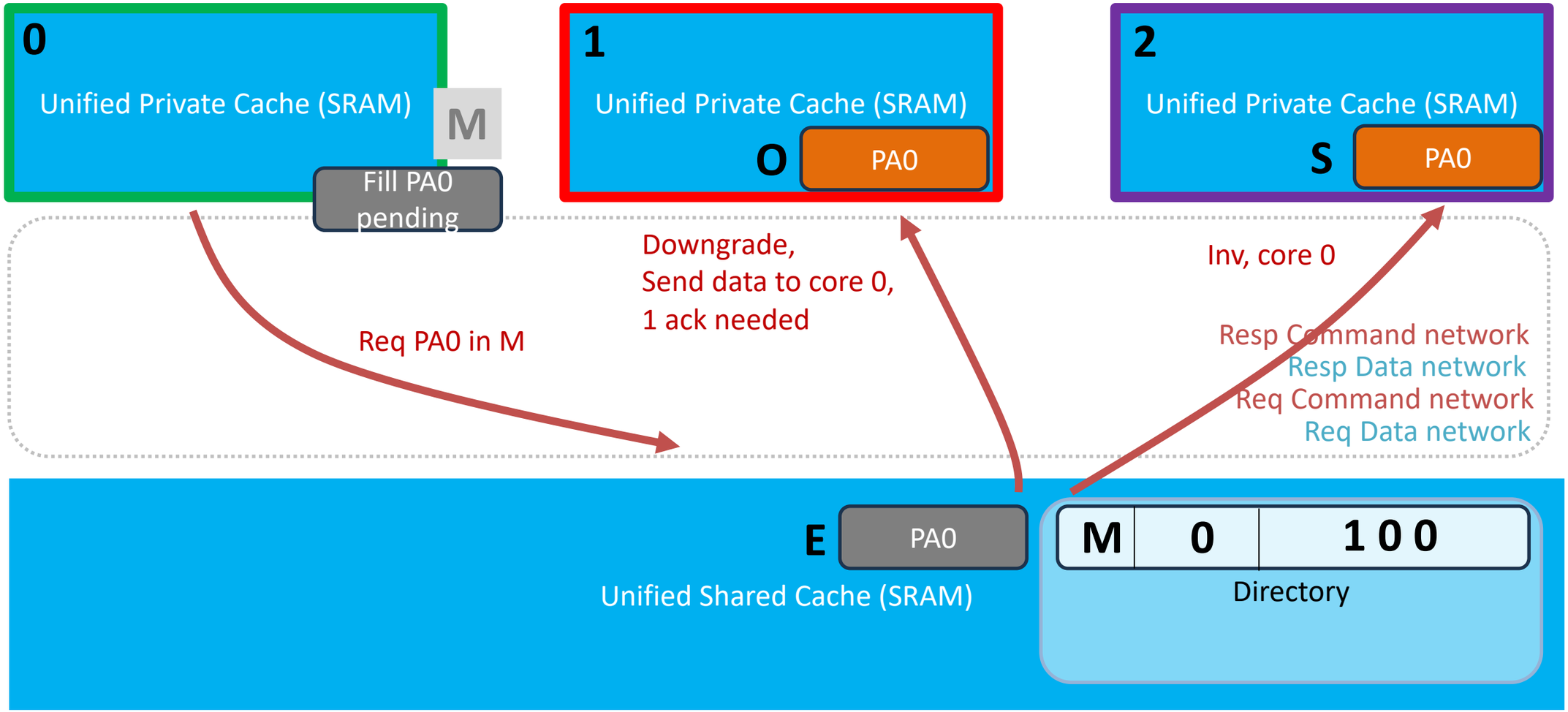
Directly from "A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example 1 - Req PA0 for Writing



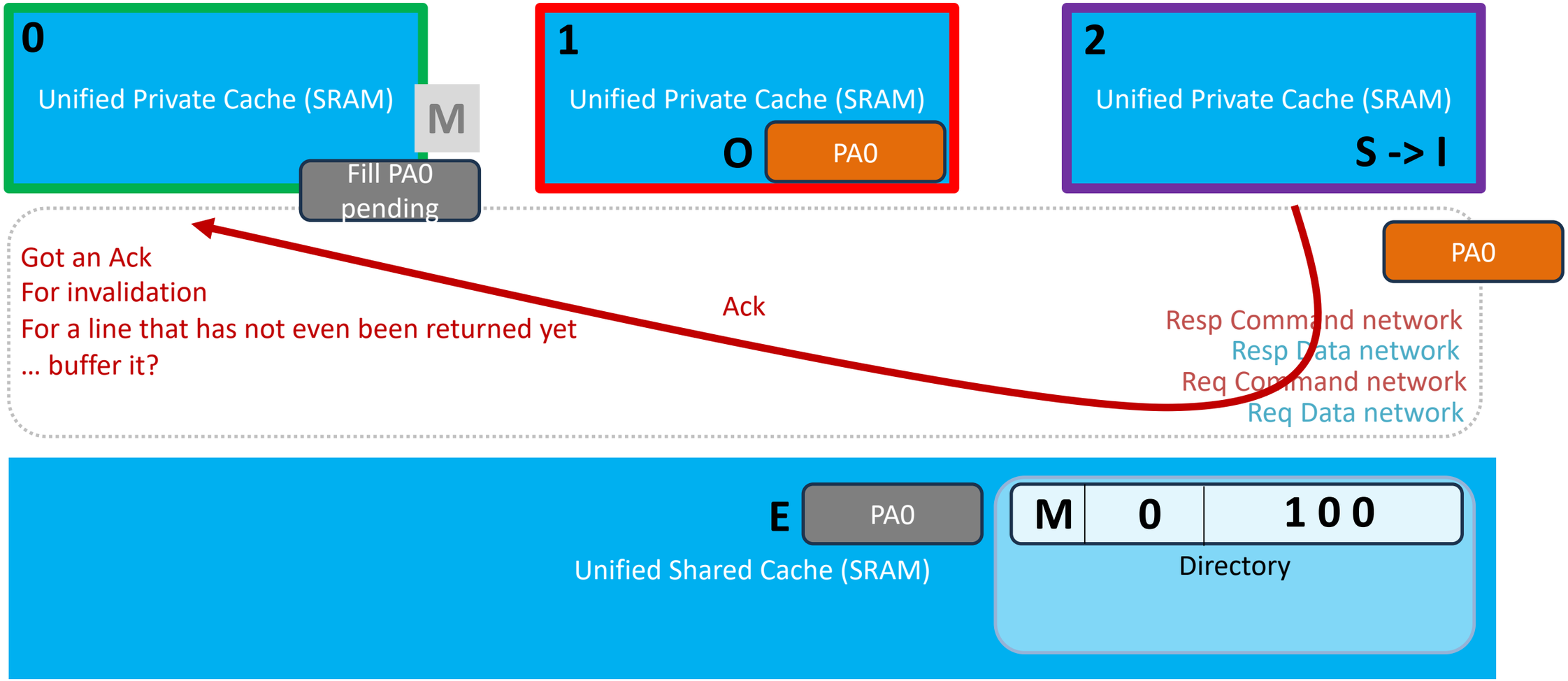
Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example 2 - Req PA0 for Writing



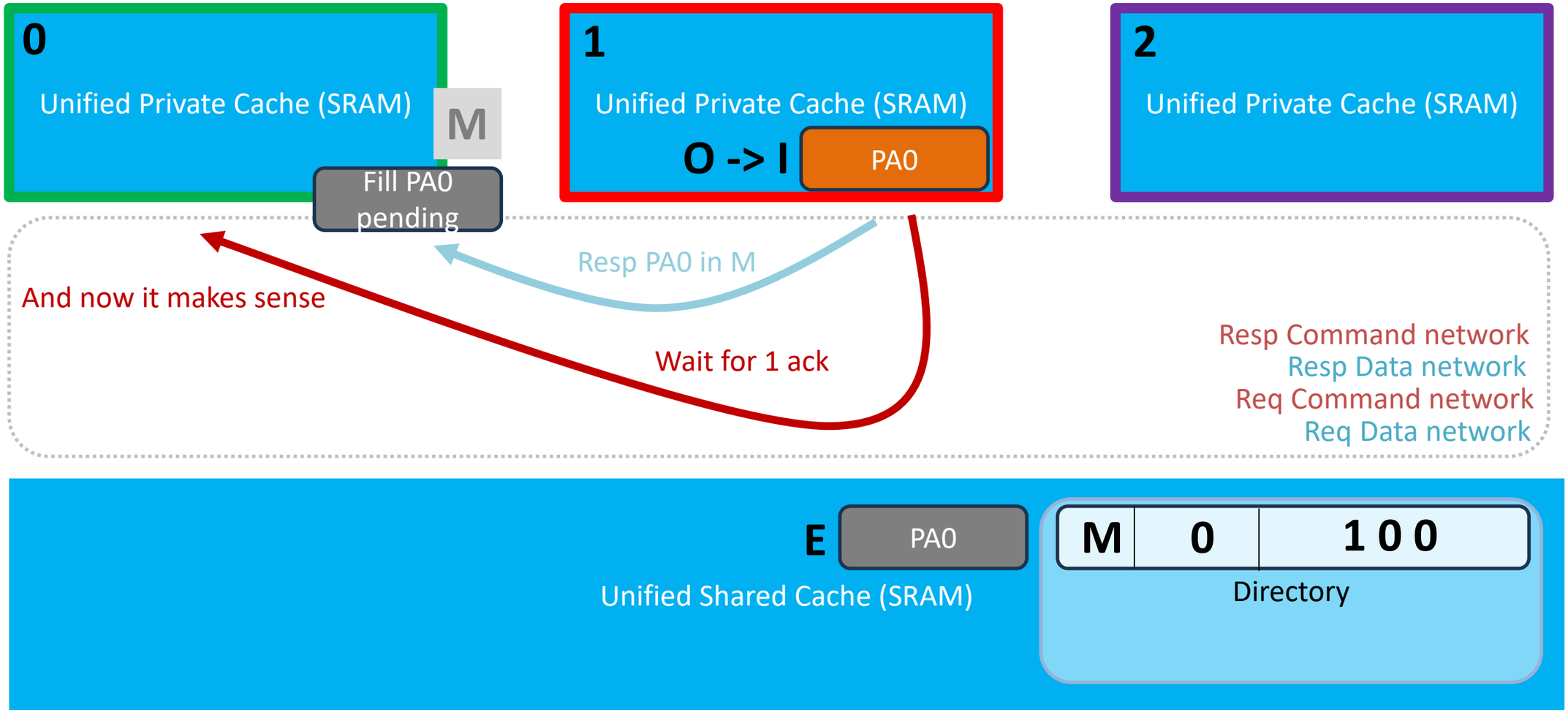
Directly from "A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example 2 - Req PA0 for Writing



Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

Example 2 - Req PA0 for Writing



Directly from “A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

These are **relatively simple** cases

Make up some puzzles… test your friends
There is usually not a single right answer



Electrical & Computer ENGINEERING

Coarse-granular Directory Coherence

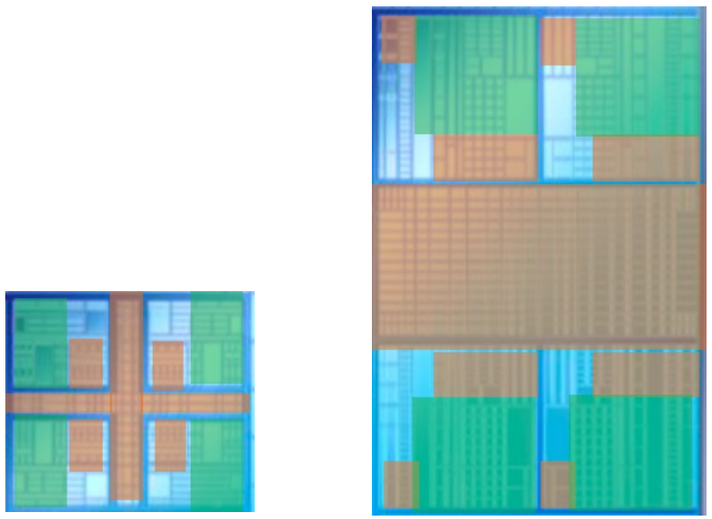
- Can be expensive to track a directory entry for every cacheline
  - Scales linearly with cache size
- Alternatives include aliasing more than one cacheline to a single Directory entry
  - Essentially forcing all the aliased lines into sharing a MESI state
  - Essentially grouping multiple physical cachelines into a single ”logical” cacheline
- Saves Storage, but can exacerbate ***False Sharing*** effects

“A Primer on Memory Consistency and Cache Coherence”, Sorin et al, Synthesis Lectures on Computer Architecture 2011

Coherence overhead in Multicore systems

- Maybe faster than flushing the data cache on load/store (other systems?)
- But coherence can still have a ***non-negligible latency impact***
- ***Workloads may access the same cacheline without actually sharing data***

False Sharing



HotChips 2024, Arik Gihon, Intel Lunar Lake Architecture Session

		Compute Cluster Cores				Low Power Island Cores			
Compute Cluster Cores			24.8	24.65	26	54.35	55.05	54.7	54.3
		23.85		27.25	27.45	54.65	55.2	55.6	54.1
		25.25	27.05		28.2	55.2	55.15	55.3	55.2
		25	27.5	27.9		56	55.15	55.3	56
Low Power Island Cores		55.05	54.45	55.4	55		23.7	23.6	22.7
		54.55	54.85	55.5	55.35	22.75		23.7	23.2
		55.1	55.05	55.55	55.15	23.4	23.6		23.5
		55.15	55.1	54.9	56.3	22.9	22.95	23.9	

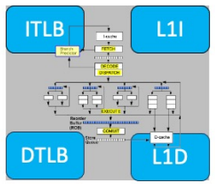
~55 ns on LNL , 3X better vs Prev Gen

HotChips 2024, Arik Gihon, Intel Lunar Lake Architecture Session

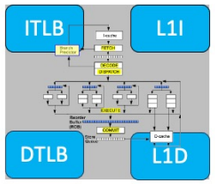
Snoop overhead scales with M and N

- ***N*** – number of cores
- ***M*** – depth of the cache hierarchy
- Rule of thumb – ***O(M*N)***
- ***There is implicitly a tradeoff between helping a single core with a deeper cache hierarchy, and helping multicore performance with a shallower one***

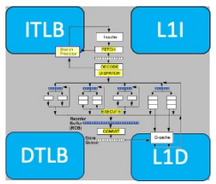
I.e. this can take a "long time"



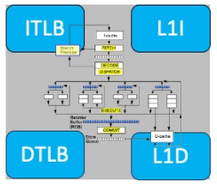
Wants to write



S



S



S

Unified Private Cache (SRAM)

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

S

Unified Private Cache (SRAM)

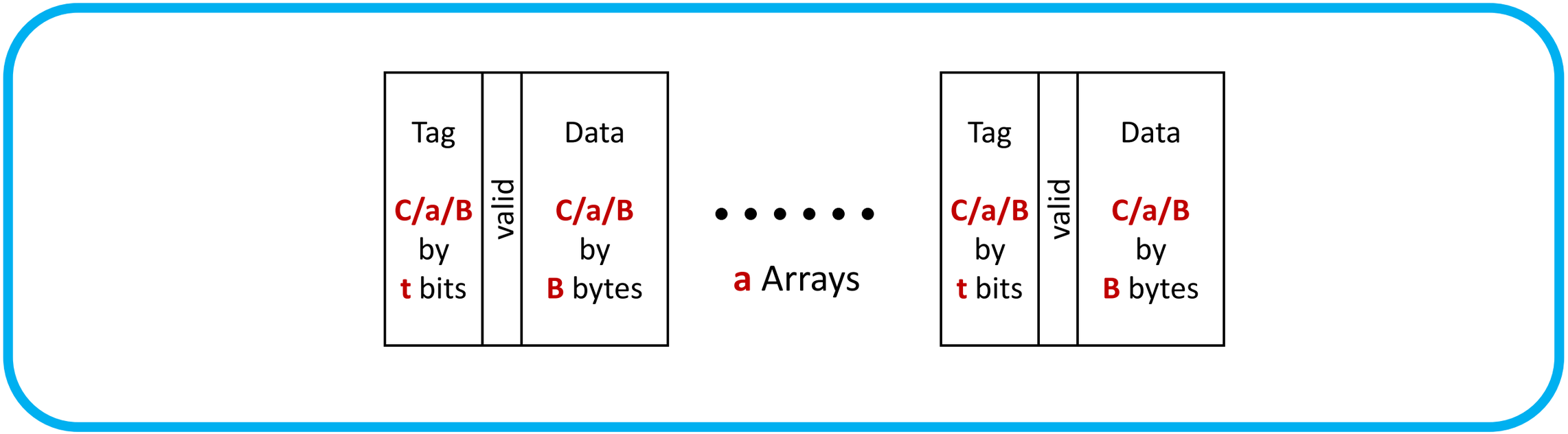
S

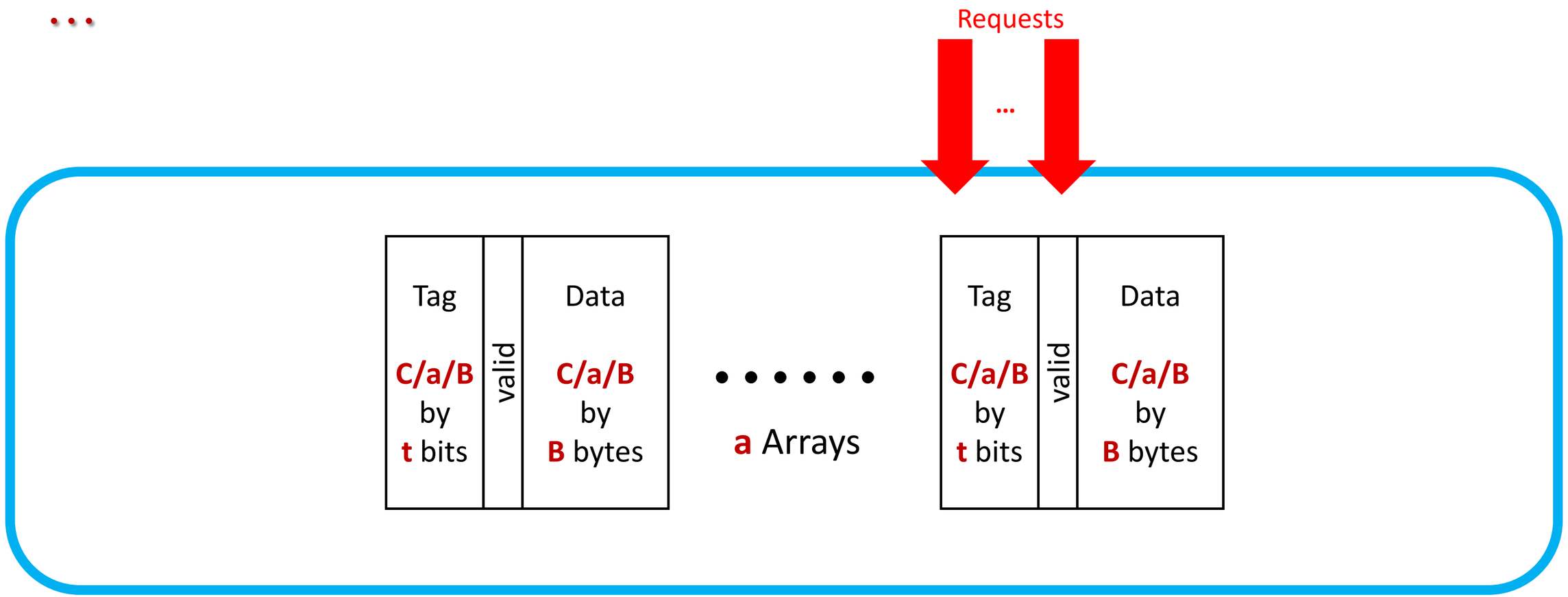
Unified Shared Cache (SRAM) S + something else in this PA space also has it

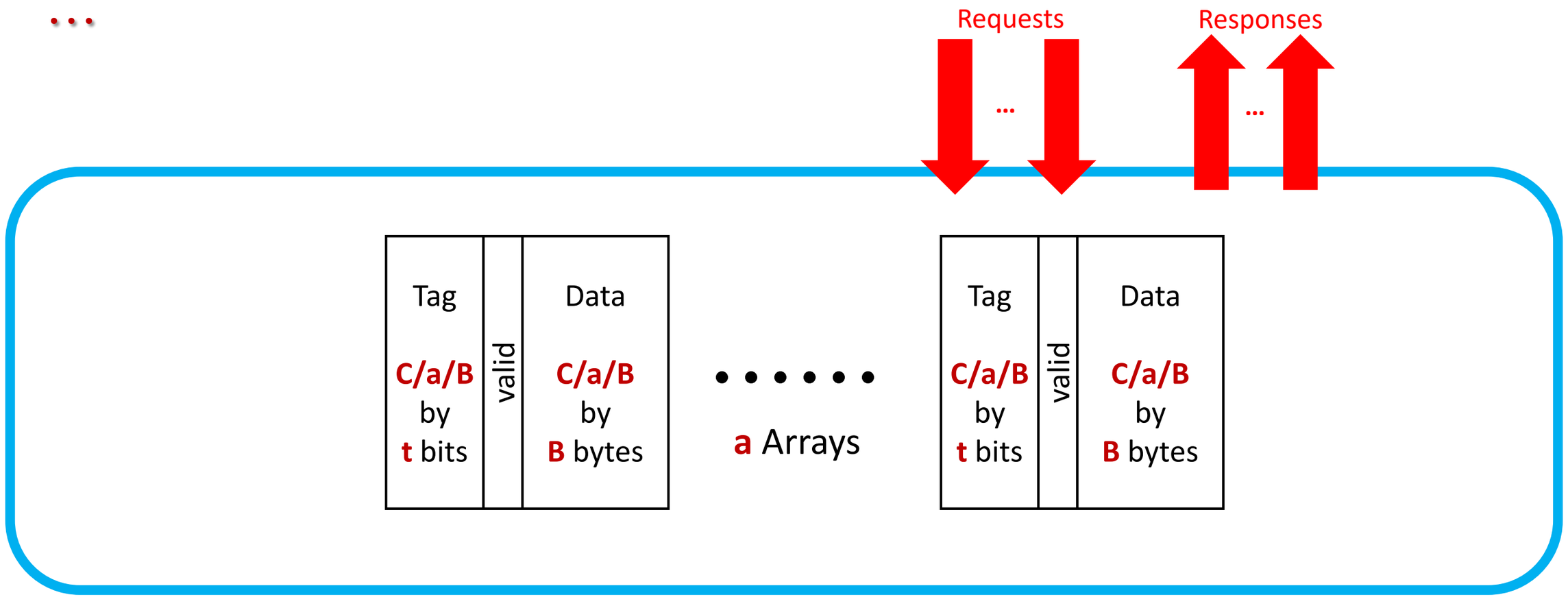
These examples were with inclusive caches

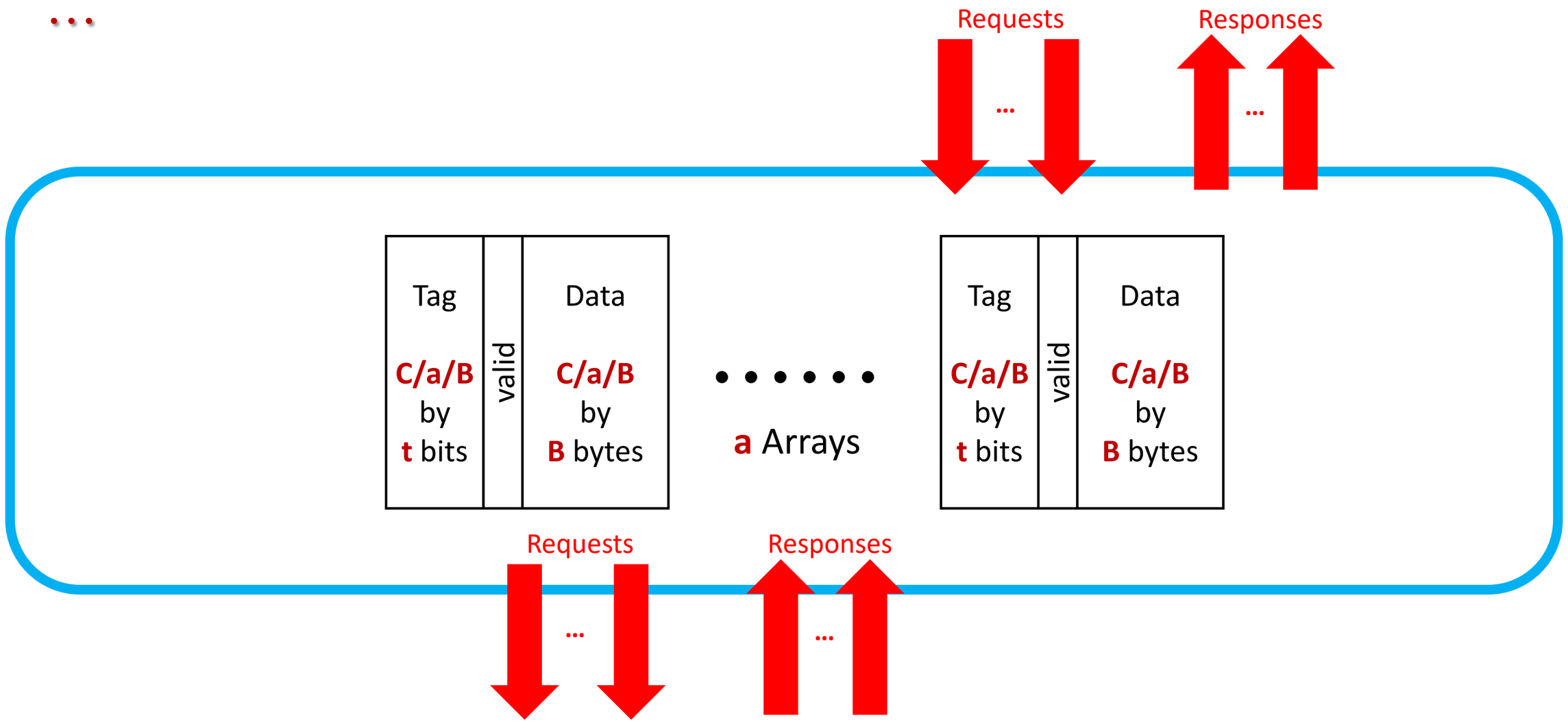
- Directory maintenance is significantly different with
  - **Non-inclusive** cache hierarchies
  - **Exclusive** cache hierarchies
- If you want to discuss these, come to OH

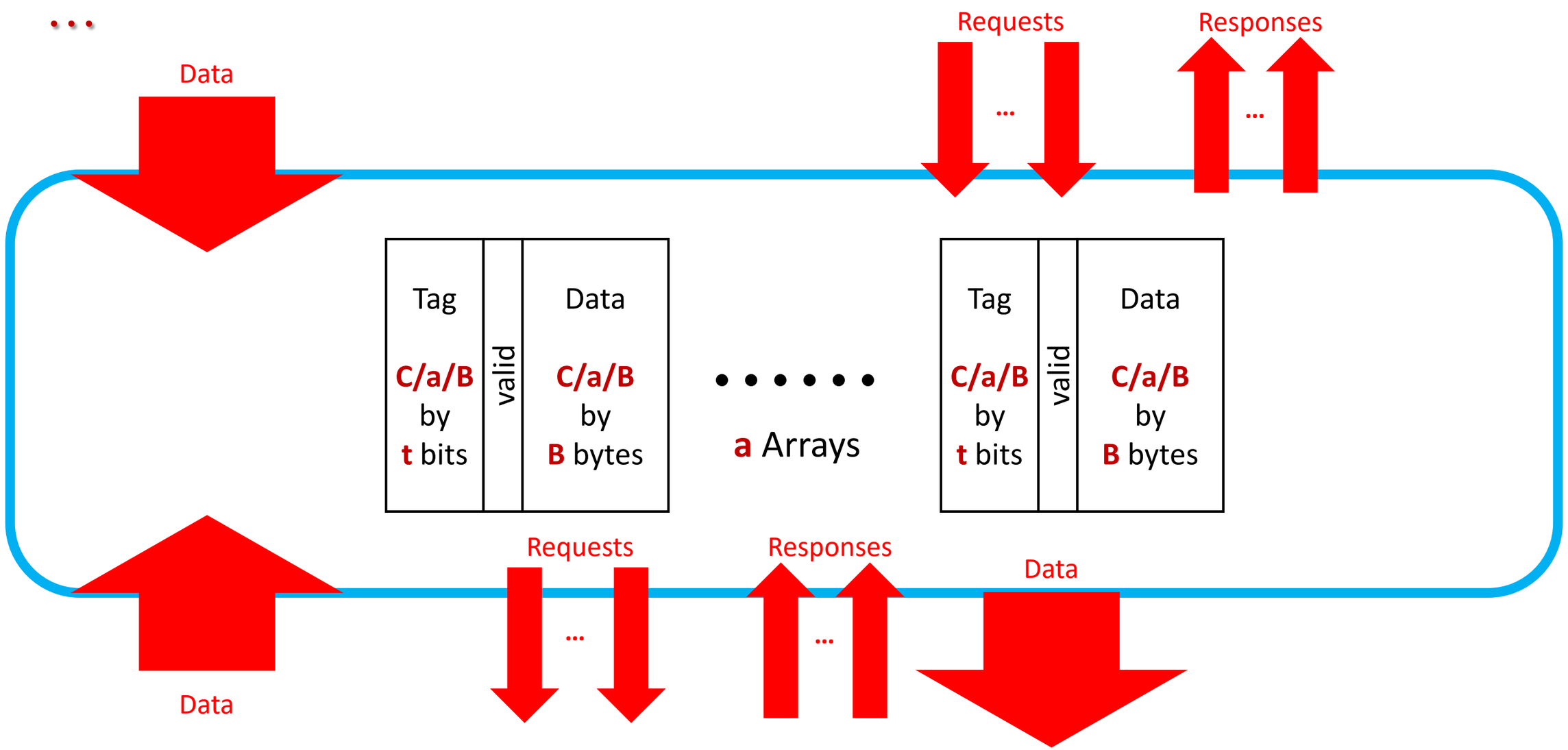
In short… caches are complicated

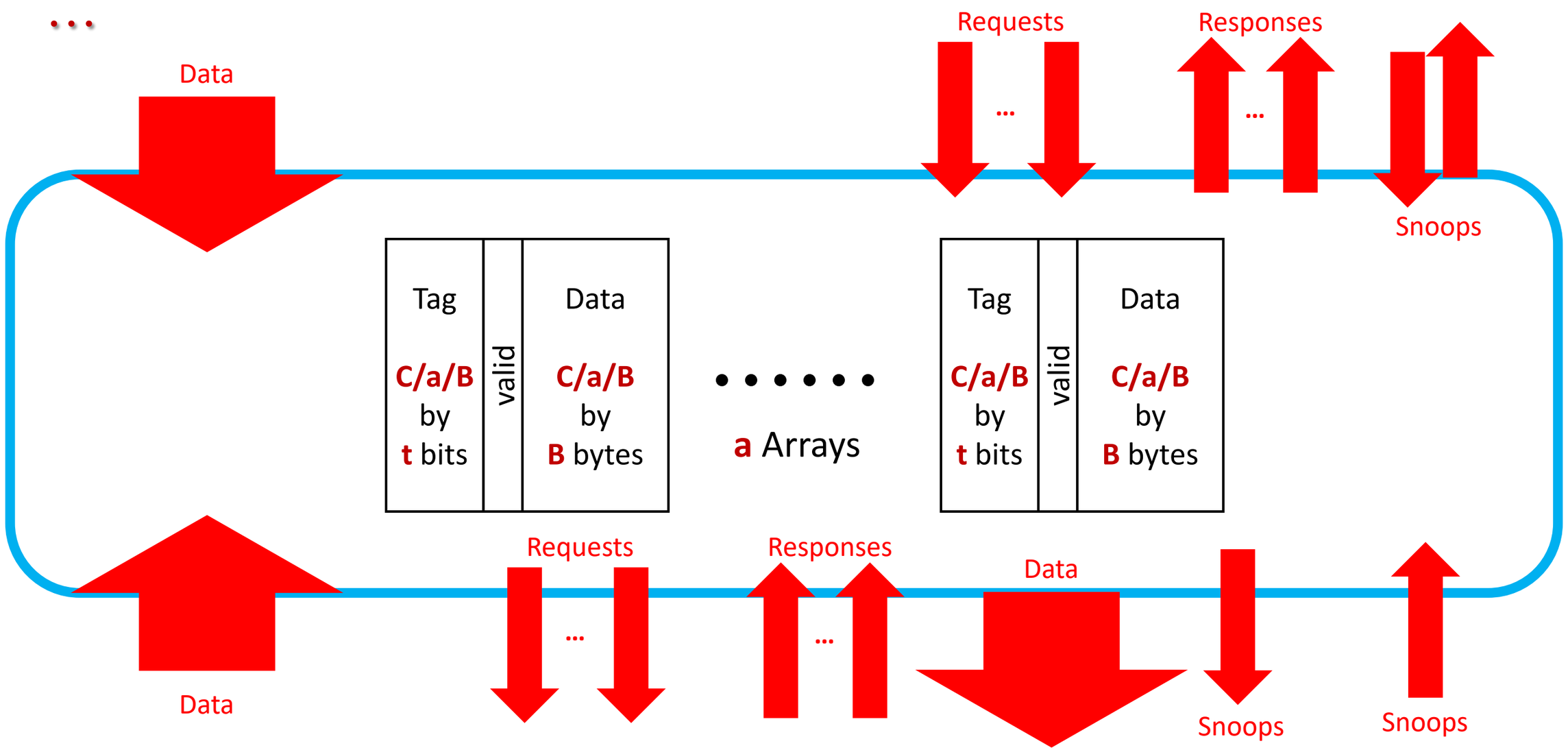



Requests
Tag
C/a/B
by
t bits
valid
Data
C/a/B
by
B bytes
a Arrays
Tag
C/a/B
by
t bits
valid
Data
C/a/B
by
B bytes

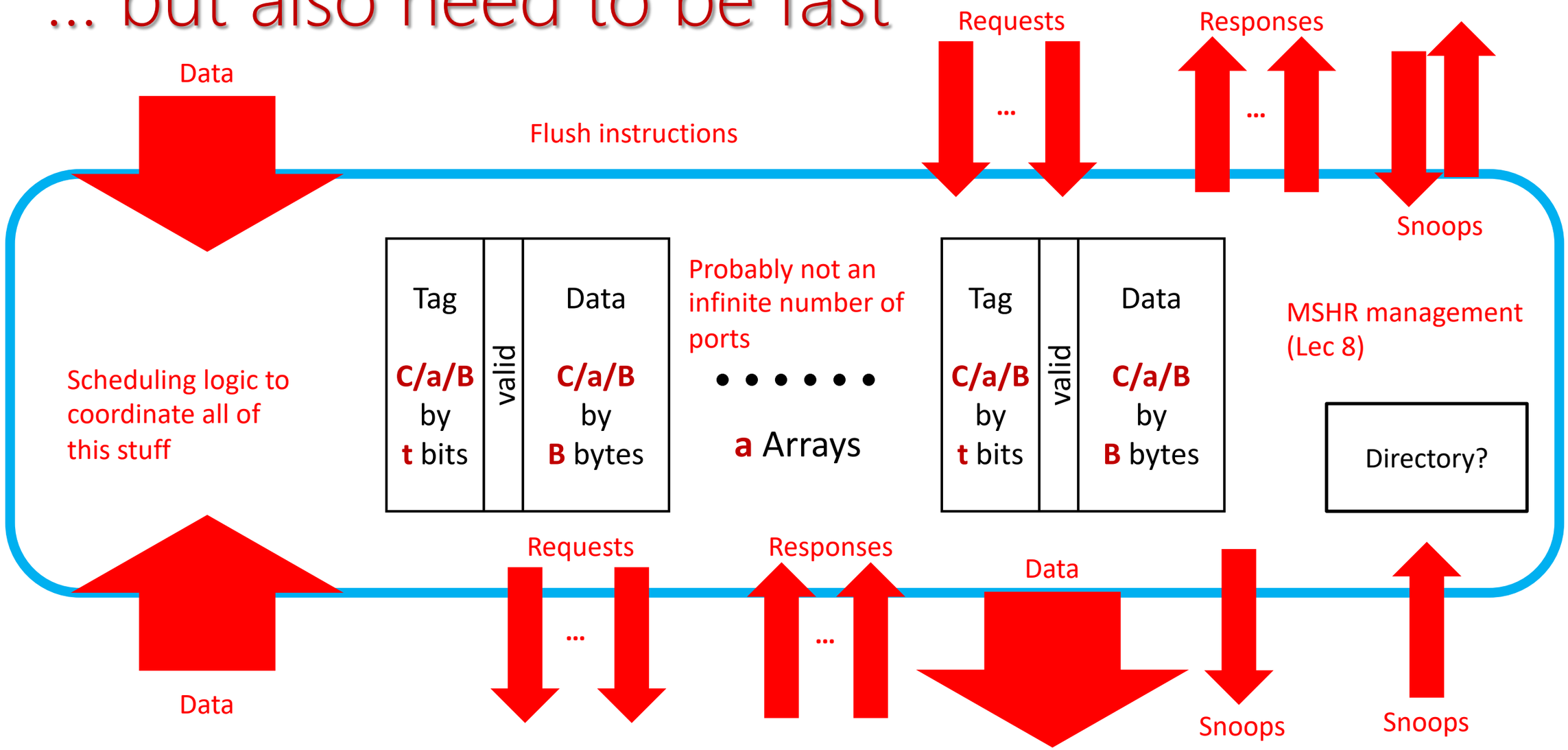

Requests
Responses
Tag
C/a/B
by
t bits
valid
Data
C/a/B
by
B bytes
a Arrays
Tag
C/a/B
by
t bits
valid
Data
C/a/B
by
B bytes


Requests
Responses
Tag
C/a/B
by
t bits
valid
Data
C/a/B
by
B bytes
a Arrays
Tag
C/a/B
by
t bits
valid
Data
C/a/B
by
B bytes
Requests
Responses


Requests
Responses
Data
Tag
C/a/B
by
t bits
valid
Data
C/a/B
by
B bytes
a Arrays
Tag
C/a/B
by
t bits
valid
Data
C/a/B
by
B bytes
Requests
Responses
Data
Data

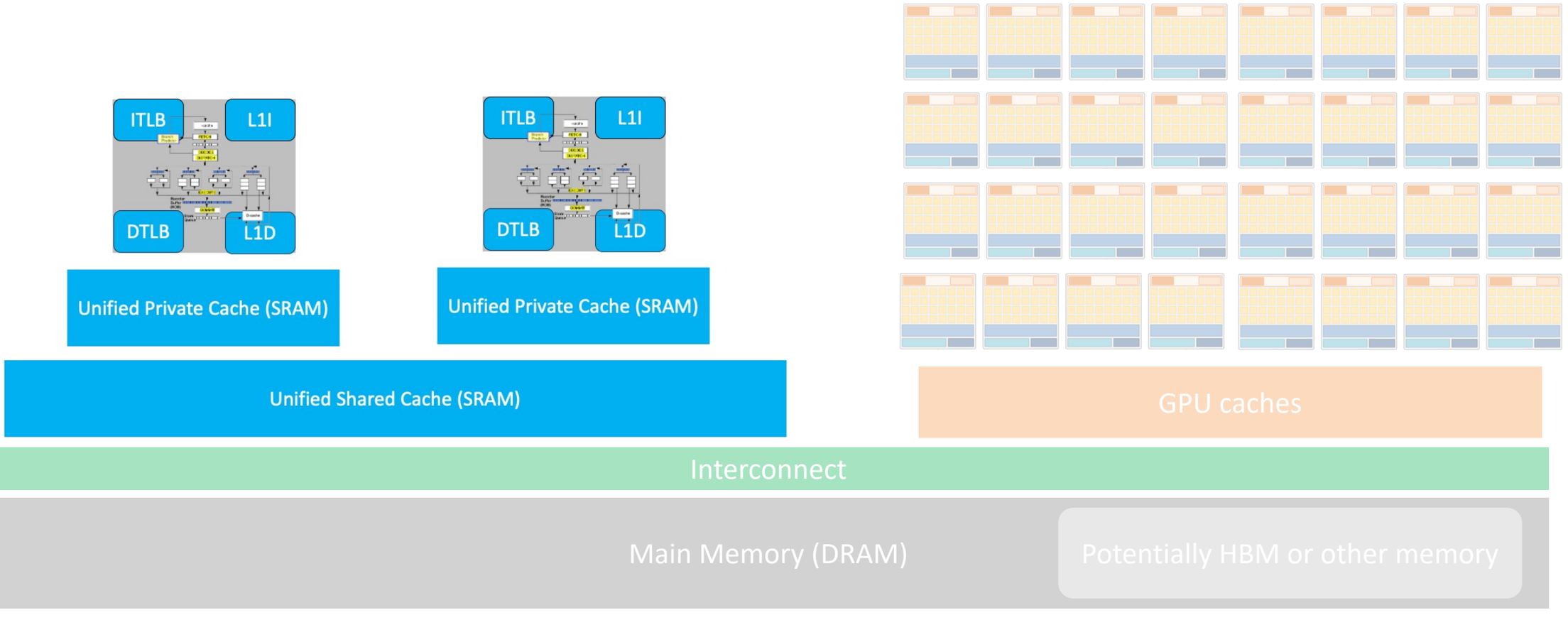

Requests
Responses
Data
Snoops
Tag
C/a/B
by
t bits
valid
Data
C/a/B
by
B bytes
a Arrays
Requests
Responses
Data
Data
Snoops
Snoops

… but also need to be fast



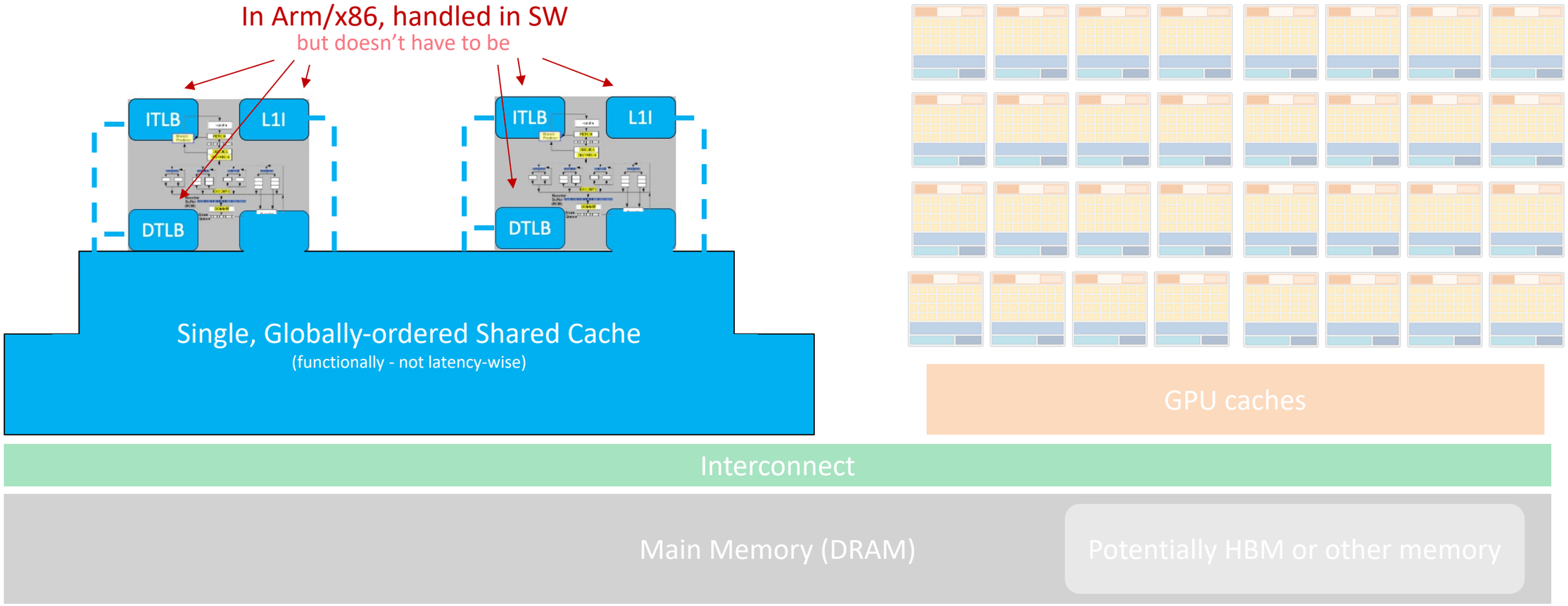
Leaving this topic for now

We've turned non-coherent caches



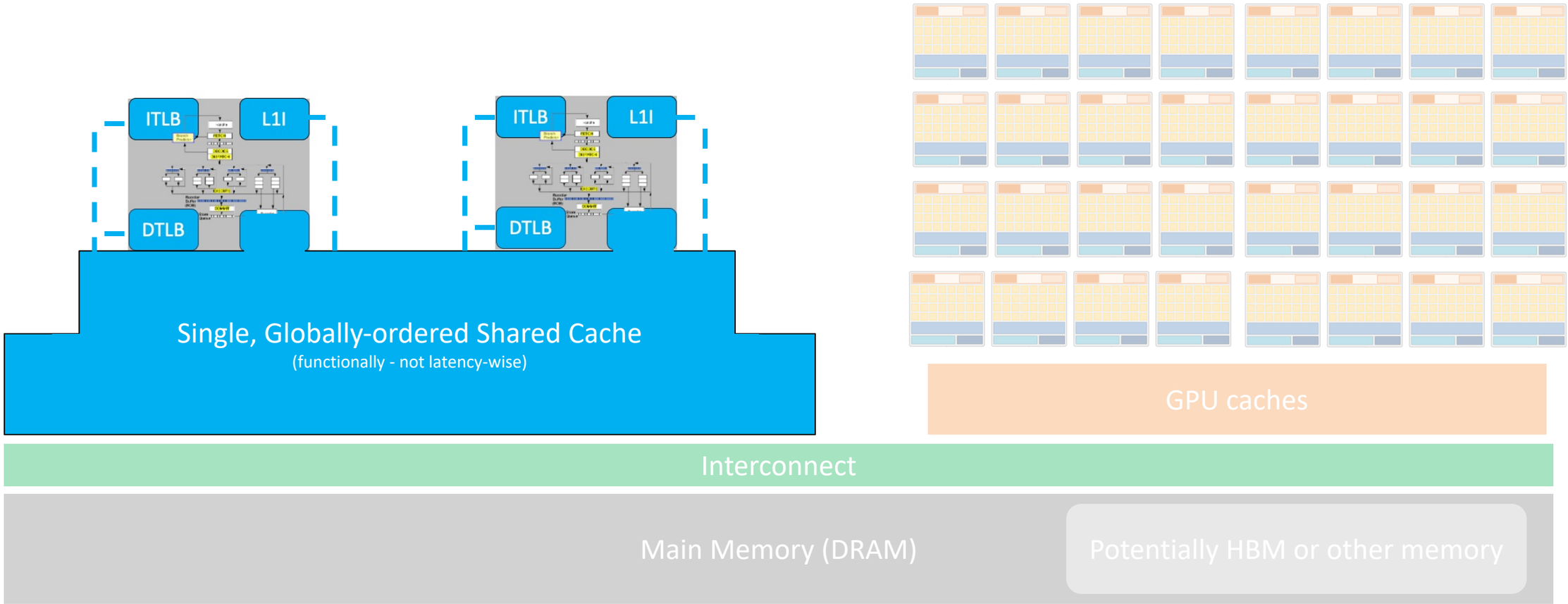
Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

… into a coherent monolith (software view)



Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

Does memory now “behave reasonably”?



Showing a genericized version of the Lunar Lake cluster hierarchy (split L1s + TLBs, unified L2 & LLC) – Ignoring the L0 as the 18740 processor does not have an L0

Depends on how you define "reasonable"

- Two threads, running in parallel on two cores, sharing an address space
- Common to synchronize via shared memory (Dekker's algorithm)

Extending what we know about single-threaded correctness to multiple cores… NO

TABLE 3.3: Can Both r1 and r2 be Set to 0?

Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0*/

But… just try this on Arm (a lot of times)

"A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

The "right answer" is "what the ISA says"

- This code is inherently non-deterministic (multicore – running in parallel)
- But you can't write parallel programs correctly without knowing
- Generally ISA's job to specify which pairs of operations can be ***"reordered"***
  - ***in this context, means "becomes globally visible in architectural state in reverse order"***
  - Microarchitecturally you can still do whatever you want
- But.. how did we end up with nonsensical results, if caches are coherent?

The things we think should happen

TABLE 3.3: Can Both r1 and r2 be Set to 0?		
Core C1	**Core C2**	**Comments**
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0*/

S1, S2, L1, L2	S1, L1, S2, L2	S2, S1, L2, L1	S2, L2, S1, L1	S1, S2, L2, L1	S2, S1, L1, L2
NEW,NEW	0,NEW	NEW,NEW	NEW,0	NEW,NEW	NEW,NEW

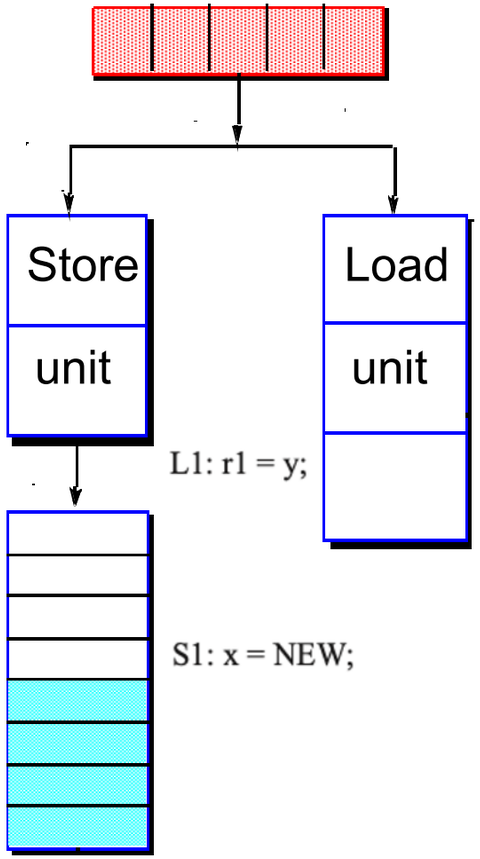
But we are fully-steeped in OOO execution at this point

Nobody told the processor it couldn't reorder S1,L1 or S2,L2. The addresses do not conflict!

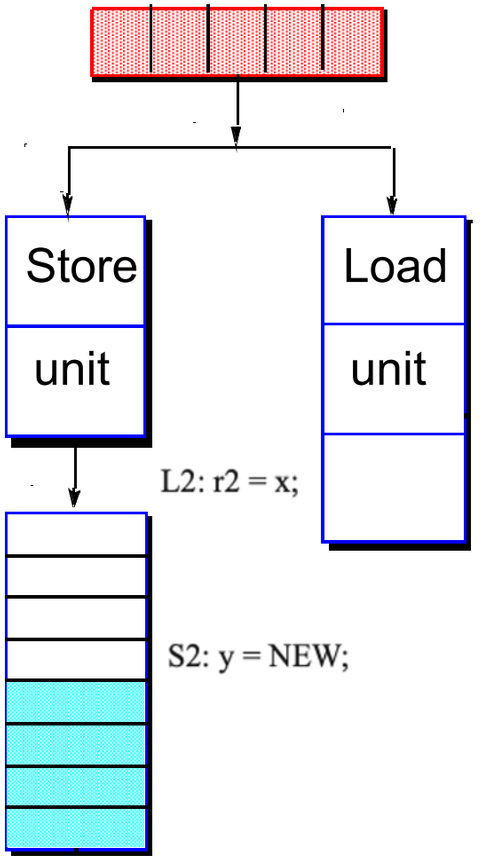
L1,L2,S1,S2
A perfectly valid sequence

"Time, Clocks, and the Ordering of Events in a Distributed System", Leslie Lamport, 1978
"A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

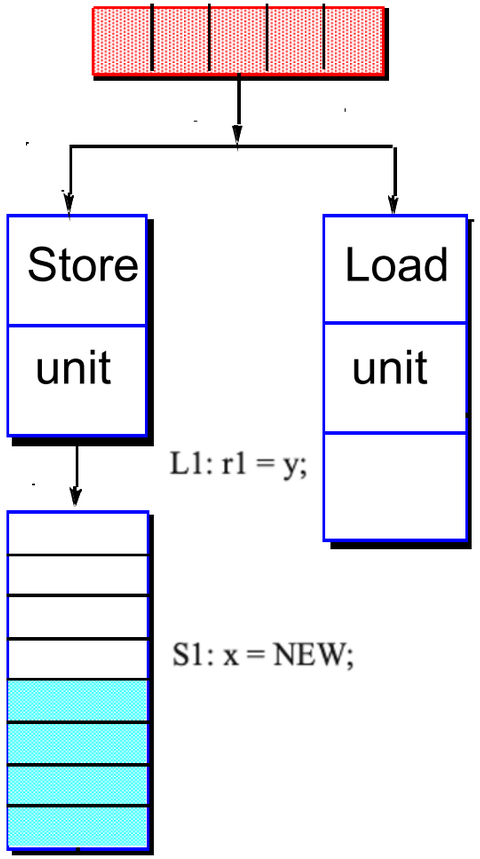
Load Bypassing



Core C1	Core C2
S1: x = NEW;	S2: y = NEW;
L1: r1 = y;	L2: r2 = x;



Load Forwarding



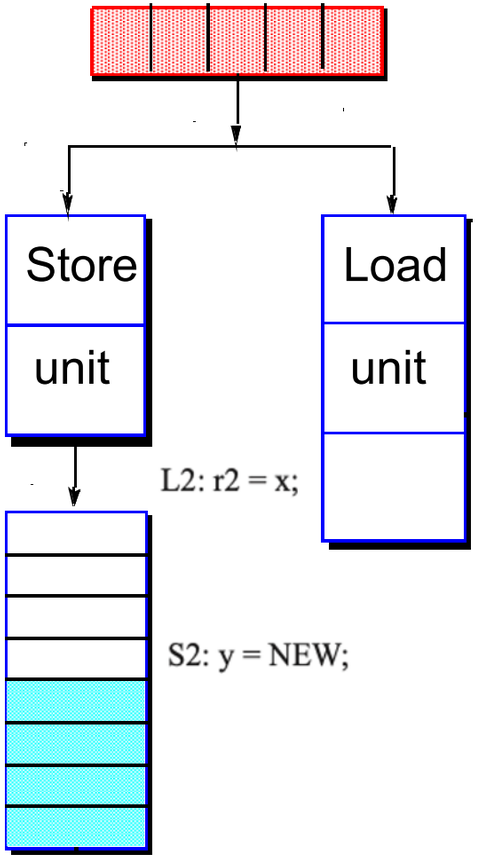
Core C1	Core C2
S1: x = NEW;	S2: y = NEW;
L1: r1 = y;	L2: r2 = x;

```
L3: r2 = x;          L4: r2 = y;
S4: y = NEW2;        S5: x = NEW2;
```

With a store buffer,
Writes implicitly ”become visible” to the issuing core first



At some point…
Need to be able to reason about when updates are visible
And in what order



Consistency models

- **Strict consistency** **(Why don't more people build this? Recall Lecture 8)**
  - No reordering:
    - Load->Load
    - Store->Load
    - Store->Store
    - Load->Store
- **Processor consistency**
  - Guarantees for each core that its stores are seen in program order
- **Cache consistency**
  - Processor consistency but only guarantees for same-address

Consistency models

- **Weak consistency (e.g. Arm + RISC-V)**
  - Load/Load reordering
  - Store/Store reordering
  - Load/Store reordering
  - Store/Load reordering
- **Total-Store-Ordering (e.g. x86)**
  - Load->Load
  - Load->Store
  - Store->Store
  - Store/Load reordering (e.g. store buffer)

What about Atomics?
Device memory accesses?
MMIO accesses?
Flush instructions?
Access to non-cacheable memory?
More on this later…

Are there *any* guarantees on a WC system

- Yes
- But… explicitly annotate code where need to enforce ordering
- Performance in common case vs code complexity/I$

Core C1	Core C2	Comments
F11: FENCE		/* Initially, data1 & data2 = 0 */
A11: acquire(lock)		
F12: FENCE		
S1: data1 = NEW;		
S2: data2 = NEW;		
F13: FENCE		
R11: release(lock)	**F21: FENCE**	
F14: FENCE	A21: acquire(lock)	
	F22: FENCE	
	L1: r2 = data2;	
	L2: r1 = data1;	/* Two Possible Outcomes under XC:
	F23: FENCE	(r1, r2) =
	R22: release(lock)	(0, 0) or (NEW, NEW)
	F24: FENCE	Same as with SC */

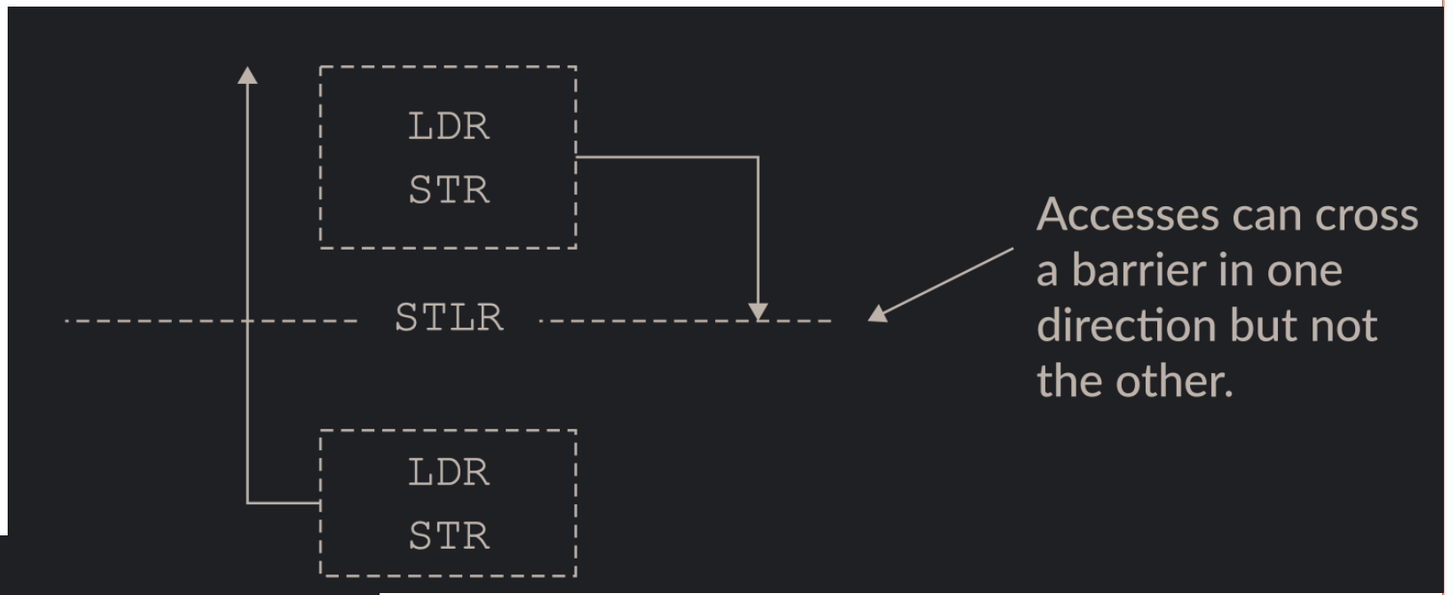
Textbook example

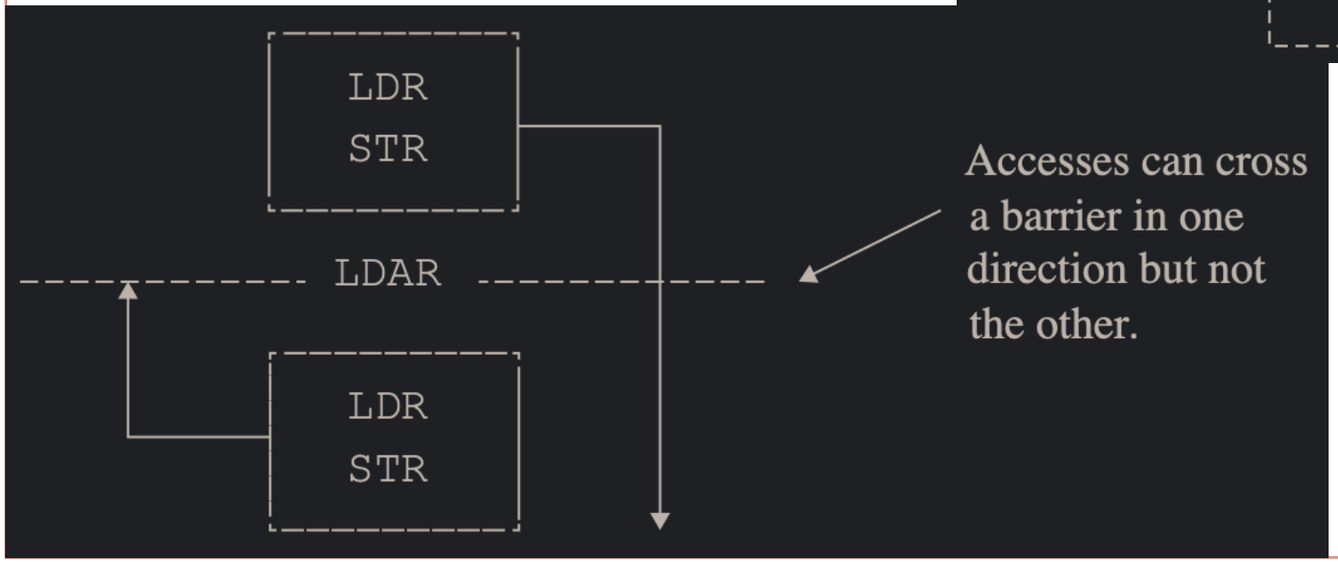
Arm has 3 main types
DSB - data sync barrier
DMB - data memory barrier
ISB – instruction sync barrier
Plus other ordering syntax…

"A Primer on Memory Consistency and Cache Coherence", Sorin et al, Synthesis Lectures on Computer Architecture 2011

Acquire/Release semantics

Finer-grained ways to surgically ONLY
Prevent specific kinds of reorderings





https://developer.arm.com/documentation/102336/0100/Load-Acquire-and-Store-Release-instructions

Inner/Outer-shareable domains

- Can also “make visible” at different granularities
- **Inner-Shareable –** Say, within a single cluster
- **Outer-Shareable –** Say, across clusters
- Choosing the right barrier is *really hard*

Recall I$ inv instruction

This kind of code often requires a tight collaboration between the OS developer and the Architect

```
_platform_pre_stackheap_init:
    dsb ish      // ensure all previous stores have completed
                 // before invalidating
    ic ialluis   // I cache invalidate all inner shareable to PoU
                 // (which includes secondary cores)
    dsb  ish     // ensure completion on inner shareable domain
                 // (which includes secondary cores)
    isb
    b InvalidateUDCaches    // only needed if the MMU is on at this point
```

From Arm developer website - https://developer.arm.com/documentation/dui0808/j/the-c-and-c---library-functions-reference/-platform-pre-stackheap-init--

18-740: Modern Computer Architecture & Design

Lecture 12: "Parallel Programming and Threading Models"

Olivia Hsu

October 1, 2026

Next Time ...

- From **Synthesis Lectures on Computer Architecture**:
- Core Reading:
  - Sec. 5: Bhattacharjee et al, "Architectural and Operating System Support for Virtual Memory," 2017
  - Skim: Scott Michael L., "Shared Memory Synchronization", 2013
- Elective Reading:
  - Sec. 3: Bugnion et al, "Hardware and Software Support for Virtualization", 2017

Electrical & Computer ENGINEERING